

**TỔNG LIÊN ĐOÀN LAO ĐỘNG VIỆT NAM
TRƯỜNG ĐẠI HỌC TÔN ĐỨC THẮNG
KHOA CÔNG NGHỆ THÔNG TIN**



**TRẦN PHẠM HỮU PHÚC - 52200219
VŨ THIÊN HOÀNG - 52200321**

PHÂN LOẠI LỖI CỦA MỘT HỆ THỐNG

**(FAULT-TYPES
CLASSIFICATION)**

DỰ ÁN CÔNG NGHỆ THÔNG TIN

KHOA HỌC MÁY TÍNH

THÀNH PHỐ HỒ CHÍ MINH, NĂM 2025

**TỔNG LIÊN ĐOÀN LAO ĐỘNG VIỆT NAM
TRƯỜNG ĐẠI HỌC TÔN ĐỨC THẮNG
KHOA CÔNG NGHỆ THÔNG TIN**



**TRẦN PHẠM HỮU PHÚC - 52200219
VŨ THIÊN HOÀNG - 52200321**

PHÂN LOẠI LỖI CỦA MỘT HỆ THỐNG

**(FAULT-TYPES
CLASSIFICATION)**

DỰ ÁN CÔNG NGHỆ THÔNG TIN

KHOA HỌC MÁY TÍNH

Người hướng dẫn
TS. Trịnh Hùng Cường

THÀNH PHỐ HỒ CHÍ MINH, NĂM 2025

LỜI CẢM ƠN

Nhóm xin gửi lời cảm ơn chân thành và sâu sắc nhất đến TS. Trịnh Hùng Cường, người thầy đã tận tình hướng dẫn, định hướng và hỗ trợ nhóm trong suốt quá trình thực hiện đề tài. Nhờ sự chỉ bảo tận tâm và những góp ý quý báu từ thầy, nhóm đã có thể hoàn thiện dự án một cách khoa học, đúng hướng và đạt được nhiều kết quả tốt hơn mong đợi.

Thầy không chỉ truyền đạt kiến thức chuyên môn mà còn giúp nhóm rèn luyện tư duy nghiên cứu, tinh thần trách nhiệm và phương pháp làm việc hiệu quả. Đây là những hành trang vô cùng quan trọng đối với mỗi thành viên trong quá trình học tập và phát triển sau này.

Nhóm xin kính chúc thầy luôn dồi dào sức khỏe, thành công trong công việc giảng dạy và nghiên cứu.

TP. Hồ Chí Minh, ngày 19 tháng 11 năm 2025

Tác giả

(Ký tên và ghi rõ họ tên)

Trần Phạm Hữu Phúc

Vũ Thiên Hoàng

CÔNG TRÌNH ĐƯỢC HOÀN THÀNH TẠI TRƯỜNG ĐẠI HỌC TÔN ĐỨC THẮNG

Tôi xin cam đoan đây là công trình nghiên cứu của riêng tôi và được sự hướng dẫn khoa học của TS. Trịnh Hùng Cường. Các nội dung nghiên cứu, kết quả trong đề tài này là trung thực và chưa công bố dưới bất kỳ hình thức nào trước đây. Những số liệu trong các bảng biểu phục vụ cho việc phân tích, nhận xét, đánh giá được chính tác giả thu thập từ các nguồn khác nhau có ghi rõ trong phần tài liệu tham khảo.

Ngoài ra, trong Dự án còn sử dụng một số nhận xét, đánh giá cũng như số liệu của các tác giả khác, cơ quan tổ chức khác đều có trích dẫn và chú thích nguồn gốc.

Nếu phát hiện có bất kỳ sự gian lận nào tôi xin hoàn toàn chịu trách nhiệm về nội dung Dự án của mình. Trường Đại học Tôn Đức Thắng không liên quan đến những vi phạm tác quyền, bản quyền do tôi gây ra trong quá trình thực hiện (nếu có).

TP. Hồ Chí Minh, ngày ... tháng ... năm 20..

Tác giả

(Ký tên và ghi rõ họ tên)

Trần Phạm Hữu Phúc

Vũ Thiên Hoàng

PHÂN LOẠI LỖI CỦA MỘT HỆ THỐNG

TÓM TẮT

Báo cáo "Phân loại lỗi của một hệ thống" tập trung nghiên cứu và ứng dụng các thuật toán học máy (Machine Learning) bao gồm Random Forest, SVM, kNN và MLP để tự động nhận diện và phân loại các dạng lỗi trong hệ thống điện (như lỗi ngắn mạch 1 pha, 2 pha, 3 pha) dựa trên dữ liệu dòng điện và điện áp. Nhóm đã xây dựng quy trình hoàn chỉnh từ thu thập, tiền xử lý dữ liệu, huấn luyện đến đánh giá và tối ưu hóa tham số mô hình bằng kỹ thuật GridSearchCV. Kết quả thực nghiệm cho thấy các mô hình đều đạt hiệu suất tốt trên tập kiểm thử. Trong đó, kNN có sự cải thiện vượt bậc sau khi tối ưu (Độ chính xác tăng từ 79.7% lên 88.5%) và MLP (Sử dụng lần lượt các thư viện Sklearn và PyTorch) thể hiện khả năng xử lý tốt các dữ liệu phi tuyến. Tuy nhiên, báo cáo cũng chỉ ra thách thức chung là việc phân biệt giữa lỗi 3 pha (LLL) và lỗi 3 pha chạm đất (LLLG) vẫn còn hạn chế do sự tương đồng lớn về đặc trưng tín hiệu.

FAULT-TYPES CLASSIFICATION

ABSTRACT

The report "Fault-Types Classification of a System" focuses on researching and applying machine learning algorithms, including Random Forest, SVM, kNN, and MLP, to automatically detect and classify fault types in electrical systems (such as single-line, line-to-line, and three-phase faults) based on current and voltage data. The team established a complete process ranging from data collection and preprocessing to training, evaluation, and hyperparameter optimization using GridSearchCV. Experimental results indicate that all models achieved good performance on the test set. Notably, kNN showed significant improvement after optimization (with accuracy increasing from 79.7% to 88.5%), and MLP (implemented in both Sklearn and PyTorch) demonstrated strong capabilities in handling non-linear data. However, the report also noted a common challenge: distinguishing between Three-Phase faults (LLL) and Three-Phase to Ground faults (LLLG) remains limited due to the high similarity in their signal characteristics.

MỤC LỤC

DANH MỤC HÌNH VẼ	vii
DANH MỤC BẢNG BIỂU	ix
DANH MỤC CÁC CHỮ VIẾT TẮT.....	x
CHƯƠNG 1. TỔNG QUAN.....	1
1.1 Lý do chọn đề tài.....	1
1.2 Mục tiêu thực hiện đề tài.....	1
1.3 Phạm vi và đối tượng nghiên cứu	1
1.4 Phương pháp nghiên cứu.....	2
CHƯƠNG 2. CƠ SỞ LÝ THUYẾT.....	2
2.1 Lỗi của một hệ thống.....	2
2.1.1 <i>Khái niệm lỗi trong hệ thống</i>	<i>2</i>
2.1.2 <i>Các loại lỗi thường gặp trong hệ thống.....</i>	<i>2</i>
2.1.3 <i>Dấu hiệu nhận biết lỗi trong dữ liệu.....</i>	<i>3</i>
2.1.4 <i>Tại sao cần học máy để phân loại lỗi?</i>	<i>4</i>
2.2 Tổng quan về học máy	4
2.2.1 <i>Khái niệm</i>	<i>4</i>
2.2.2 <i>Các phương pháp học máy</i>	<i>5</i>
2.2.3 <i>Các thành phần chính trong bài toán học máy.....</i>	<i>5</i>
2.2.4 <i>Các chỉ số đánh giá mô hình.....</i>	<i>6</i>
2.3 Các mô hình học máy được sử dụng.....	6
2.3.1 <i>Random Forest.....</i>	<i>6</i>
2.3.2 <i>SVM.....</i>	<i>10</i>

2.3.3 <i>kNN</i>	13
2.3.4 <i>MLP</i>	16
CHƯƠNG 3. THỰC NGHIỆM	21
3.1 Dataset.....	21
3.2 Random Forest	28
3.3 SVM.....	36
3.4 <i>kNN</i>	47
3.5 <i>MLP</i>	52
3.5.1 <i>MLP</i> (sklearn - <i>MLPClassifier</i>)	52
3.5.2 <i>MLP</i> (PyTorch).....	64
CHƯƠNG 4. ĐÁNH GIÁ VÀ KẾT LUẬN	78
4.1 Tổng hợp kết quả các mô hình.....	78
4.2 So sánh hiệu năng theo các chỉ số chính.....	79
4.3 Nhận xét quan trọng: Lớp lỗi khó (<i>LLL</i> vs <i>LLLG</i>)	80
4.4 Kết luận mô hình tốt nhất.....	81
4.5 Tổng kết cuối cùng.....	81
TÀI LIỆU THAM KHẢO.....	83

DANH MỤC HÌNH VẼ

Hình 3.1 : Histogram phân phối các đặc trưng	22
Hình 3.2 : Boxplot các đặc trưng	23
Hình 3.3 : Ma trận tương quan giữa các đặc trưng	24
Hình 3.4 : Phân bố số mẫu theo từng loại lỗi.....	25
Hình 3.5 : Phân cụm các loại lỗi	26
Hình 3.6 : Mối quan hệ cặp giữa các đặc trưng	27
Hình 3.7 : Violin plot các đặc trưng.....	28
Hình 3.8 : Các đặc trưng quan trọng.....	30
Hình 3.9 : ROC Curve.....	31
Hình 3.10 : Confusion Matrix - Baseline	33
Hình 3.11 : Confusion Matrix - Optimized.....	33
Hình 3.12 : Baseline vs Optimized	34
Hình 3.13 : Minh họa dự đoán của RF trên tập kiểm tra	36
Hình 3.14 : Learning Curve	38
Hình 3.15 : Confusion Matrix - Baseline	40
Hình 3.16 : Confusion Matrix - Optimized.....	40
Hình 3.17 : Normalized Confusion Matrix - Baseline	42
Hình 3.18 : Normalized Confusion Matrix - Optimized.....	42
Hình 3.19 : Precision-Recall Curve	43
Hình 3.20 : Minh họa dự đoán trên một số mẫu kiểm tra	44
Hình 3.21 : Baseline vs Optimized	45
Hình 3.22 : Confusion Matrix - Baseline	49

Hình 3.23 : Confusion Matrix - Optimized	49
Hình 3.24 : Baseline vs Optimized	50
Hình 3.25 : Minh họa dự đoán của kNN trên tập kiểm tra	52
Hình 3.26 : Baseline vs Optimized	54
Hình 3.27 : Learning Curve MLP (Optimized).....	57
Hình 3.28 : Confusion Matrix MLP (Baseline)	59
Hình 3.29 : Normalized Confusion Matrix MLP (Baseline)	60
Hình 3.30 : Confusion Matrix MLP (Optimized)	61
Hình 3.31 : Normalized Confusion Matrix MLP (Optimized)	62
Hình 3.32 : Precision-Recall Curve MLP (Optimized).....	63
Hình 3.33 : Minh họa dự đoán của MLP trên tập kiểm tra	64
Hình 3.34 : Baseline vs Optimized	67
Hình 3.35 : Confusion Matrix - MLP (PyTorch, Baseline)	71
Hình 3.36 : Normalized Confusion Matrix - MLP (PyTorch, Baseline)	72
Hình 3.37 : Confusion Matrix - MLP (PyTorch, Optimized)	73
Hình 3.38 : Normalized Confusion Matrix - MLP (PyTorch, Optimized)	74
Hình 3.39 : ROC Curve - MLP (PyTorch, Optimized, OvR)	75
Hình 3.40 : Precision-Recall Curve - MLP (PyTorch, Optimized, OvR).....	76
Hình 3.41 : Minh họa dự đoán của MLP PyTorch trên tập kiểm tra	77
Hình 3.42 : Learning Curve - MLP (PyTorch) Baseline vs Optimized	78

DANH MỤC BẢNG BIỂU

Bảng 2.1 : Dấu hiệu nhận biết lỗi.....	3
Bảng 3.1 : Kết quả AUC	32
Bảng 3.2 : Accuracy	35
Bảng 3.3 : F1 Macro.....	35
Bảng 3.4 : Precision Macro	35
Bảng 3.5 : Recall Macro.....	36
Bảng 3.6 : Accuracy	45
Bảng 3.7 : F1 Macro.....	45
Bảng 3.8 : Precision Macro	46
Bảng 3.9 : Recall Macro.....	46
Bảng 3.10 : Accuracy	50
Bảng 3.11 : F1 Macro.....	50
Bảng 3.12 : Precision Macro	51
Bảng 3.13 : Recall Macro.....	51
Bảng 3.14 : Accuracy	55
Bảng 3.15 : F1 Macro.....	55
Bảng 3.16 : Precision Macro	55
Bảng 3.17 : Recall Macro.....	56
Bảng 3.18 : Accuracy	68
Bảng 3.19 : F1 Macro.....	68
Bảng 3.20 : Precision Macro	68
Bảng 3.21 : Recall Macro.....	69

DANH MỤC CÁC CHỮ VIẾT TẮT

ML	Machine Learning
RF	Random Forest
SVM	Support Vector Machine
kNN	k-Nearest Neighbors
MLP	Multi-Layer Perceptron
TN	True Negative
TP	True Positive
FN	False Negative
FP	False Positive
ANN	Artificial Neural Networks
AUC	Area Under the Curve
ROC	Receiver Operating Characteristic
OOB	Out-of-Bag
PCA	Principal Component Analysis
OvR	One-vs-Rest

CHƯƠNG 1. TỔNG QUAN

1.1 Lý do chọn đề tài

Trong các hệ thống hiện đại (như dây chuyền sản xuất, hệ thống cảm biến, hay máy chủ mạng,...), việc phát hiện và phân loại lỗi đóng vai trò quan trọng để đảm bảo hoạt động ổn định, giảm thiểu thời gian ngừng máy và chi phí bảo trì.

Trước đây, việc xác định lỗi thường được thực hiện thủ công hoặc dựa vào các quy tắc cố định, dẫn đến hạn chế khi hệ thống trở nên phức tạp và dữ liệu đa dạng.

Sự phát triển của Machine Learning (Học máy) đã mở ra hướng tiếp cận mới cho việc tự động phát hiện và phân loại lỗi dựa trên dữ liệu thực tế, giúp phát hiện bất thường nhanh hơn, chính xác hơn và có khả năng mở rộng.

Chính vì vậy, nhóm quyết định chọn đề tài “**Áp dụng các mô hình học máy để phân loại lỗi của một hệ thống**” nhằm nghiên cứu, thử nghiệm và so sánh hiệu quả của các mô hình học máy trong việc nhận diện và phân loại lỗi.

1.2 Mục tiêu thực hiện đề tài

- Xây dựng mô hình học máy có khả năng phân loại lỗi hệ thống dựa trên dữ liệu đầu vào.
- So sánh hiệu năng của các mô hình phổ biến như **Random Forest, SVM, kNN, MLP**.
- Tối ưu hóa tham số của mô hình nhằm nâng cao độ chính xác phân loại.
- Đưa ra quy trình huấn luyện, đánh giá mô hình có thể áp dụng cho các hệ thống khác.

1.3 Phạm vi và đối tượng nghiên cứu

- **Đối tượng:** Dữ liệu lỗi hệ thống (log, thông số cảm biến hoặc dữ liệu mô phỏng).
- **Phạm vi:** Nghiên cứu tập trung vào mô hình học máy truyền thống, không đi sâu vào Deep Learning phức tạp.

- **Kết quả:** Mô hình có khả năng phân loại chính xác $\geq 80\%$ trên dữ liệu thử nghiệm.

1.4 Phương pháp nghiên cứu

1. Thu thập và tiền xử lý dữ liệu.
2. Huấn luyện các mô hình học máy cơ bản.
3. Đánh giá mô hình theo Accuracy, Precision, Recall, F1-score.
4. Tối ưu hóa tham số bằng GridSearchCV .
5. So sánh, phân tích và rút ra kết luận.

CHƯƠNG 2. CƠ SỞ LÝ THUYẾT

2.1 Lỗi của một hệ thống

2.1.1 Khái niệm lỗi trong hệ thống

Trong bất kỳ hệ thống nào, từ thiết bị cơ điện tử, hệ thống cảm biến, dây chuyền sản xuất cho đến mạng máy tính hoặc phần mềm thì lỗi (fault) được định nghĩa là một trạng thái bất thường khiến hệ thống hoạt động sai lệch so với hành vi mong muốn.

Lỗi có thể xuất hiện:

- Đột ngột
- Xuất hiện từ từ
- Ngẫu nhiên
- Lặp đi lặp lại theo chu kỳ

2.1.2 Các loại lỗi thường gặp trong hệ thống

Tùy loại hệ thống, lỗi có thể được chia thành:

a. Lỗi cảm biến (Sensor Fault)

- Sensor hỏng $\rightarrow$ tín hiệu sai
- Tín hiệu bị nhiễu, lệch chuẩn, mất tín hiệu

Ví dụ: cảm biến nhiệt độ trả về giá trị = 0 trong khi máy đang hoạt động.

b. Lỗi phần cứng (Hardware Fault)

- Quá nhiệt
- Ma sát cao
- Cháy mô tơ
- Bộ phận cơ khí mòn

Ví dụ: động cơ bị kẹt làm dòng điện tăng đột biến.

c. Lỗi vận hành hoặc tải (Load Fault)

- Vận hành quá tải
- Tác nghẽn hệ thống
- Áp suất tăng bất thường

d. Lỗi truyền thông hoặc mạng (Communication Fault)

- Packet loss
- Độ trễ cao
- Mất kết nối

e. Lỗi phần mềm / logic (Software Fault)

- Ngoại lệ không xử lý
- Logic sai
- Memory leak
- Deadlock

2.1.3 Dấu hiệu nhận biết lỗi trong dữ liệu

Khi lỗi xuất hiện, dữ liệu đo từ hệ thống sẽ có những thay đổi đặc trưng:

Bảng 2.1: Dấu hiệu nhận biết lỗi

Loại lỗi	Dấu hiệu trong dữ liệu
Lỗi cảm biến	Giá trị nhảy đột ngột, nằm ngoài giới hạn bình thường

Lỗi cơ khí	Biên độ rung tăng mạnh, tần số bất thường
Lỗi tải	Dòng điện tăng, nhiệt độ tăng
Lỗi mạng	Throughput giảm, latency tăng
Lỗi phần mềm	Log xuất hiện lỗi theo pattern nhất định

Điểm quan trọng: Mỗi loại lỗi để lại “dấu vết” trong dữ liệu → đó chính là thứ mô hình học máy sẽ học.

2.1.4 Tại sao cần học máy để phân loại lỗi?

Học máy được ứng dụng vì:

- Học được mẫu lỗi phức tạp từ dữ liệu, không cần mô hình vật lý.
 - Phù hợp khi hệ thống không thể mô tả bằng công thức toán học.
- Phát hiện lỗi nhanh và chính xác hơn con người
 - Giúp giảm downtime, tăng năng suất.
- Xử lý dữ liệu đa chiều. Ví dụ: với 20 cảm biến → Random Forest hoặc MLP xử lý tốt.
- Có khả năng tổng quát hóa lỗi mới
 - Mô hình vẫn dự đoán tốt khi hệ thống thay đổi nhẹ.

2.2 Tổng quan về học máy

2.2.1 Khái niệm

Học máy là một nhánh của trí tuệ nhân tạo, cho phép máy tính tự học từ dữ liệu, từ đó dự đoán hoặc ra quyết định mà không cần lập trình quy tắc cụ thể.

Trong bài toán phân loại lỗi, học máy có thể học từ các mẫu lỗi trong quá khứ để phát hiện hoặc phân loại lỗi mới trong hệ thống.

Các bước cơ bản trong quy trình học máy:

1. Thu thập dữ liệu.
2. Tiền xử lý và chuẩn hóa.
3. Chọn mô hình và huấn luyện.
4. Đánh giá và tối ưu mô hình.
5. Triển khai và theo dõi.

2.2.2 Các phương pháp học máy

Có bốn loại chính:

- **Học có giám sát (Supervised Learning):**
 - Dữ liệu có nhãn (ví dụ: lỗi A, lỗi B, bình thường).
 - Ví dụ: kNN, SVM, Random Forest, Neural Network
- **Học không giám sát (Unsupervised Learning):**
 - Dữ liệu không có nhãn, mô hình tự tìm ra cấu trúc hoặc nhóm (clustering).
 - Ví dụ: K-means, DBSCAN.
- **Học bán giám sát (Semi-supervised Learning):**
 - Dữ liệu vừa có nhãn vừa không có nhãn (thường là ít dữ liệu có nhãn, nhiều dữ liệu không nhãn)
 - Ví dụ: Self-training, Label Propagation, Semi-supervised SVM
- **Học tăng cường (Reinforcement Learning):**
 - Mô hình học qua tương tác môi trường, nhận phần thưởng (reward) cho hành động tốt.
 - Ví dụ: Q-learning, Deep Q-Network, Policy Gradient

2.2.3 Các thành phần chính trong bài toán học máy

- **Dữ liệu đầu vào (Input):** Các thông số hoặc tín hiệu từ hệ thống (ví dụ: nhiệt độ, dòng điện, tốc độ quay).
- **Dữ liệu đầu ra (Output):** Loại lỗi hoặc trạng thái tương ứng (ví dụ: “quá nhiệt”, “kẹt trục”, “hoạt động bình thường”).
- **Mô hình (Model):** Thuật toán học máy được chọn.

- **Hàm mất mát (Loss Function):** Đo sai số giữa giá trị dự đoán và thực tế.
- **Thuật toán tối ưu (Optimizer):** Tìm bộ tham số giúp giảm sai số tối đa.

2.2.4 Các chỉ số đánh giá mô hình

Accuracy: Là tỷ lệ dự đoán đúng trên tổng số mẫu.

$$Accuracy = \frac{TP + TN}{TP + TN + FP + FN}$$

(***TP:** True Positive, **TN:** True Negative, **FP:** False Positive, **FN:** False Negative)

Precision: Cho biết mô hình dự đoán đúng bao nhiêu trong số các dự đoán là “lỗi”.

$$Precision = \frac{TP}{TP + FP}$$

Recall: Cho biết tỷ lệ lỗi thực tế mà mô hình phát hiện được.

$$Recall = \frac{TP}{TP + FN}$$

F1-score: Cân bằng giữa Precision và Recall.

$$F1 - score = 2 \times \frac{Precision \times Recall}{Precision + Recall}$$

2.3 Các mô hình học máy được sử dụng

2.3.1 Random Forest

2.3.1.1 Giới thiệu chung

Random Forest là một thuật toán học máy Học có giám sát (Supervised learning) thuộc nhóm Học tập tổ hợp (Ensemble learning), kết hợp nhiều Cây quyết định (Decision Trees) để tạo thành một mô hình mạnh hơn, ổn định hơn và có độ chính xác cao hơn.

Được phát triển bởi Leo Breiman và Adele Cutler (2001), Random Forest khắc phục nhược điểm của một cây quyết định đơn lẻ (Dễ bị overfitting) bằng cách:

- Tạo nhiều cây huấn luyện độc lập,

- Cho các cây “bỏ phiếu” để đưa ra kết quả cuối cùng.

Random Forest thường được xem là một trong những mô hình hiệu quả nhất cho các bài toán phân loại lỗi hệ thống do khả năng xử lý nhiễu, dữ liệu phi tuyến và sự đa dạng đặc trưng.

2.3.1.2 Nguyên lý hoạt động của Random Forest

Random Forest dựa trên hai yếu tố “Ngẫu nhiên” sau:

1) Ngẫu nhiên về dữ liệu (Bagging / Bootstrapping)

Khi xây dựng mỗi cây quyết định, thuật toán không dùng toàn bộ tập dữ liệu gốc. Thay vào đó, nó tạo ra một tập con dữ liệu mới bằng phương pháp **Sampling with replacement** (Lấy mẫu có hoàn lại).

- Điều này có nghĩa là một mẫu dữ liệu có thể xuất hiện nhiều lần trong một cây, nhưng có thể không xuất hiện lần nào ở cây khác.
- Những mẫu không được chọn cho một cây cụ thể được gọi là Out-of-Bag (OOB).

2) Ngẫu nhiên về đặc trưng (Feature Randomness)

Khi phân chia một nút (node) trong cây quyết định:

- Cây quyết định thông thường sẽ xét tất cả các đặc trưng (features) để tìm ra điểm cắt tốt nhất.
- Random Forest chỉ xét một tập hợp ngẫu nhiên các đặc trưng (ví dụ: chỉ xét $\sqrt{m}$ đặc trưng trong tổng số m đặc trưng).
- Mục đích: Ngăn chặn việc một vài đặc trưng quá mạnh chi phối tất cả các cây, giúp các cây trong rừng có sự đa dạng (diversity).

2.3.1.3 Quy trình hoạt động chi tiết

Giả sử chúng ta có tập dữ liệu huấn luyện với N quan sát và M đặc trưng.

1. **Khởi tạo:** Chọn số lượng cây trong rừng (k cây).
2. **Huấn luyện từng cây:** Với mỗi cây thứ i (từ 1 đến k):
 - Bước 1 (Bootstrapping): Lấy ngẫu nhiên N mẫu từ tập dữ liệu gốc (có hoàn lại).

- Bước 2 (Xây dựng cây): Xây dựng cây quyết định nhưng không cắt tỉa (pruning). Tại mỗi nút phân chia:
 - + Chọn ngẫu nhiên d đặc trưng ($d < M$, thường chọn $d = \sqrt{M}$).
 - + Dùng tiêu chuẩn (Gini hoặc Entropy) để tìm đặc trưng và ngưỡng phân chia tốt nhất trong số d đặc trưng này.

3. Dự đoán (Aggregation):

- Với bài toán Phân loại: Áp dụng cơ chế Voting (Bầu chọn). Mỗi cây đưa ra một nhãn lớp (class), nhãn nào được nhiều cây bầu nhất sẽ là kết quả cuối cùng (Hard Voting).
- Với bài toán Hồi quy: Áp dụng cơ chế Averaging (Trung bình). Kết quả là trung bình cộng giá trị dự đoán của tất cả các cây.

2.3.1.4 Tiêu chuẩn phân chia

Để quyết định cách chia nhánh tại các nút, các cây con trong Random Forest sử dụng các độ đo về độ tinh khiết của dữ liệu.

a. Gini Impurity (Độ vẩn đục Gini)

Đây là tiêu chuẩn phổ biến nhất (mặc định trong Scikit-learn). Nó đo lường xác suất một phần tử được chọn ngẫu nhiên bị phân loại sai.

Công thức cho một nút t với C lớp:

$$Gini(t) = 1 - \sum_{i=1}^C (p_i)^2$$

Trong đó, p_i là xác suất của lớp i tạo nút đó.

- Nếu nút thuần khiết (chỉ chứa 1 lớp): $Gini = 0$
- **Mục tiêu:** Giảm thiểu Gini Impurity.

b. Entropy và Information Gain (Lượng tin thu được)

Entropy đo lường độ hỗn loạn của dữ liệu:

$$Entropy(t) = - \sum_{i=1}^C p_i \log_2(p_i)$$

Information Gain là lượng Entropy giảm đi sau khi phân chia nút cha thành các nút con:

$$IG(Parent, Children) = Entropy(Parent) - \sum \frac{N_{child}}{N_{parent}} \times Entropy(Child)$$

Mục tiêu: Tối đa hóa Information Gain.

2.3.1.5 Out-of-Bag (OOB) Error

Đây là một tính năng đặc biệt của Random Forest dùng để đánh giá mô hình mà không cần tập Validation riêng biệt.

- Trong quá trình Bootstrapping, khoảng 36.8% dữ liệu sẽ không được chọn vào một cây cụ thể (tập OOB).
- Ta có thể dùng chính những mẫu OOB này làm dữ liệu kiểm thử cho cây đó.
- OOB Score: Là độ chính xác trung bình của các dự đoán trên các mẫu OOB của toàn bộ rừng. Nó là một ước lượng không thiên lệch (unbiased estimate) về khả năng tổng quát hóa của mô hình.

2.3.1.6 Các siêu tham số (Hyperparameters) quan trọng

Khi tinh chỉnh (tuning) Random Forest, bạn cần chú ý các tham số sau:

- **n_estimators:** Số lượng cây trong rừng. Càng nhiều càng tốt (ổn định hơn) nhưng tốn tài nguyên và thời gian.
- **max_depth:** Độ sâu tối đa của mỗi cây. Giới hạn độ sâu giúp tránh overfitting.
- **min_samples_split:** Số lượng mẫu tối thiểu cần thiết để tách một nút.
- **max_features:** Số lượng đặc trưng tối đa được xem xét để tìm ra sự phân chia tốt nhất.
- **criterion:** Hàm đo lường chất lượng phân chia (Gini hoặc Entropy).

2.3.1.7 Ưu điểm và Nhược điểm

- **Ưu điểm**

- **Độ chính xác cao:** Thường tốt hơn cây quyết định đơn lẻ và nhiều thuật toán khác.
- **Chống Overfitting:** Nhờ cơ chế Bagging và Random Feature, nó khó bị overfitting hơn cây đơn lẻ (trừ khi dữ liệu quá nhiều).
- **Feature Importance:** Có khả năng tự đánh giá tầm quan trọng của từng đặc trưng (biết được biến nào ảnh hưởng nhất đến kết quả).
- **Xử lý tốt dữ liệu thiếu (Missing values):** Có thể duy trì độ chính xác ngay cả khi thiếu một phần dữ liệu.
- **Không cần chuẩn hóa dữ liệu:** Không yêu cầu Feature Scaling (Standardization/Normalization).

- **Nhược điểm**

- **Black Box (Hộp đen):** Khó giải thích trực quan cụ thể tại sao mô hình đưa ra quyết định đó (so với cây quyết định đơn lẻ rất dễ hiểu).
- **Tốc độ dự đoán chậm:** Do phải chạy qua hàng trăm cây con để tổng hợp kết quả (chậm hơn Linear Regression hay SVM).
- **Tốn tài nguyên:** Yêu cầu bộ nhớ lớn để lưu trữ toàn bộ các cây.

2.3.2 SVM

2.3.2.1 Giới thiệu chung

Support Vector Machine (SVM) là một thuật toán học máy thuộc nhóm học có giám sát (Supervised Learning), được sử dụng phổ biến cho các bài toán phân loại (Classification) và hồi quy (Regression). SVM được giới thiệu bởi Vapnik và các cộng sự vào những năm 1990, và được xem là một trong những mô hình mạnh mẽ nhất cho các bài toán phân loại khi dữ liệu có tính phân tách rõ ràng.

Ý tưởng chính của SVM là tìm ra một siêu phẳng (Hyperplane) có khả năng phân tách các lớp dữ liệu sao cho khoảng cách biên (Margin) giữa siêu phẳng và các điểm dữ liệu gần nhất của mỗi lớp là lớn nhất. Những điểm gần nhất này được gọi là Support Vectors - các điểm dữ liệu quan trọng nhất quyết định ranh giới phân lớp.

2.3.2.2 Nguyên lý hoạt động của SVM

a) Siêu phẳng phân lớp (Separating Hyperplane)

Trong không gian 2 chiều, siêu phẳng là một đường thẳng; trong không gian 3 chiều, là một mặt phẳng; và trong không gian nhiều chiều hơn, đó là một mặt siêu phẳng (hyperplane).

Đối với bài toán phân loại nhị phân (binary classification), nhiệm vụ của SVM là tìm ra một siêu phẳng:

- Phân chia hai lớp dữ liệu sao cho chúng nằm ở hai phía khác nhau.
- Có khoảng cách đến các điểm của hai lớp là **lớn nhất (max-margin)**.

Một siêu phẳng tối ưu giúp mô hình tổng quát hóa tốt hơn, giảm khả năng bị overfitting.

b) Support Vectors

Support Vectors là các điểm dữ liệu nằm sát biên (margin).

Những điểm này “định hình” siêu phẳng tối ưu của SVM.

- Nếu Support Vectors thay đổi → siêu phẳng thay đổi.
- Nếu các điểm khác thay đổi → siêu phẳng không đổi.

Do đó SVM mang tính “ổn định” hơn so với nhiều thuật toán khác.

c) Margin (Biên phân tách)

Margin là khoảng cách giữa siêu phẳng và các support vector.

SVM tìm siêu phẳng có **margin lớn nhất** vì:

- Margin lớn → mô hình tổng quát hóa tốt
- Ít nhạy cảm với nhiễu
- Hạn chế overfitting

SVM cho dữ liệu không thể phân tách tuyến tính:

Trong nhiều bài toán thực tế bao gồm phân loại lỗi hệ thống điện, dữ liệu không thể phân tách bằng một đường thẳng.

Để xử lý trường hợp này, SVM sử dụng **Kernel Trick**.

2.3.2.3 Kernel Trick

Kernel Trick là kỹ thuật biến đổi dữ liệu từ không gian đầu vào ban đầu sang một không gian đặc trưng có số chiều cao hơn, trong đó dữ liệu có thể tuyến tính phân tách.

Một số loại kernel phổ biến:

- a. Linear Kernel: Sử dụng cho dữ liệu tuyến tính phân tách được.
- b. Polynomial Kernel: Áp dụng khi mối quan hệ giữa các biến mang bản chất đa thức.
- c. RBF (Radial Basis Function) Kernel - Gaussian Kernel: Là kernel phổ biến nhất, dùng để phân tách các cụm dữ liệu phi tuyến.

RBF tạo ra ranh giới phân chia cong hoặc phức tạp hơn.

Trong bài toán phân loại lỗi hệ thống điện, nhóm sử dụng **RBF kernel** vì dữ liệu dòng/áp mang tính phi tuyến cao.

2.3.2.4 Các siêu tham số quan trọng của SVM

a) C (Regularization Parameter)

- Điều khiển mức phạt với các điểm phân loại sai.
- C lớn $\rightarrow$ ưu tiên phân loại đúng từng điểm nhưng dễ overfit.
- C nhỏ $\rightarrow$ tăng margin nhưng chấp nhận phân loại sai một số điểm.

b) Gamma (γ) trong RBF Kernel

- Gamma điều khiển độ cong của biên phân tách.
- Gamma lớn $\rightarrow$ mô hình “nhìn gần”, ranh giới phức tạp, dễ overfit.
- Gamma nhỏ $\rightarrow$ ranh giới mềm, dễ underfit.

c) Kernel

- Chọn loại kernel: linear, poly, rbf...

2.3.2.5 Ưu điểm và Nhược điểm của SVM

● Ưu điểm:

- Hiệu quả cao với dữ liệu có số chiều lớn.
- Hoạt động tốt khi khoảng cách giữa các lớp rõ ràng.

- Phù hợp với dữ liệu phi tuyến nhờ Kernel Trick.
- Không bị ảnh hưởng nhiều bởi outliers (nhờ max-margin).

- **Nhược điểm**

- Tốc độ huấn luyện chậm với dataset lớn.
- Việc lựa chọn kernel và siêu tham số C , γ cần thử nghiệm nhiều.
- Không tạo ra xác suất đầu ra tự nhiên (phải bật thêm `probability=True`).

2.3.2.6 Ứng dụng SVM trong phân loại lỗi hệ thống

Trong hệ thống điện ba pha, các loại lỗi (LG, LL, LLG, LLL, LLLG) tạo ra các mẫu dòng và áp rất khác nhau ở từng pha.

Những mẫu này hình thành **các cụm dữ liệu phi tuyến**:

- Dòng/áp tăng mạnh ở pha lỗi
- Pha lành ít thay đổi
- Lỗi nhiều pha tạo thành biên dạng phức tạp

Do đó:

- SVM với **RBF Kernel** có khả năng phân tách các cụm lỗi một cách hiệu quả
- Sau khi chuẩn hóa dữ liệu, SVM đạt độ chính xác cao và ổn định.

SVM là lựa chọn mạnh mẽ cho bài toán phân loại lỗi vì:

- Có khả năng học biên dạng phi tuyến
- Tổng quát hóa tốt
- Tập trung vào những điểm “quan trọng nhất” (support vectors)
- Ít bị ảnh hưởng bởi nhiễu so với nhiều thuật toán khác

2.3.3 *kNN*

2.3.3.1 Giới thiệu

k-Nearest Neighbors (kNN) là một trong những thuật toán học máy đơn giản và phổ biến nhất thuộc nhóm Học có giám sát (Supervised Learning). Thuật toán này

có thể được sử dụng cho cả bài toán phân loại (Classification) và hồi quy (Regression), nhưng được ứng dụng rộng rãi nhất trong phân loại.

kNN được gọi là thuật toán "Học lười" (Lazy Learning) hoặc phi tham số (Non-parametric):

- **Lazy Learning:** Thuật toán không thực sự "học" một mô hình cụ thể (như bộ trọng số trong mạng nơron hay các quy tắc trong cây quyết định) trong giai đoạn huấn luyện. Thay vào đó, nó chỉ lưu trữ toàn bộ dữ liệu huấn luyện. Quá trình tính toán chỉ thực sự diễn ra khi cần dự đoán cho một điểm dữ liệu mới.
- **Non-parametric:** Thuật toán không đưa ra các giả định về phân phối của dữ liệu nền (như giả định dữ liệu tuân theo phân phối chuẩn).

2.3.3.2 Nguyên lý hoạt động

Nguyên lý cơ bản của kNN dựa trên giả định: "Những điểm dữ liệu tương đồng nhau thường nằm gần nhau trong không gian đặc trưng".

Quy trình dự đoán nhãn cho một điểm dữ liệu mới x_{new} gồm các bước sau:

1. Tính khoảng cách: Tính khoảng cách từ điểm x_{new} đến tất cả các điểm trong tập dữ liệu huấn luyện.
2. Tìm láng giềng: Chọn ra k điểm dữ liệu gần nhất với x_{new} (được gọi là k láng giềng).
3. Bầu chọn (Voting):
 - Đối với bài toán Phân loại: Nhãn của x_{new} được quyết định bằng cách lấy theo đa số (majority voting) từ nhãn của k láng giềng này.
 - Đối với bài toán Hồi quy: Giá trị của x_{new} là trung bình cộng giá trị của k láng giềng.

2.3.3.3 Các thành phần chính của thuật toán

a. Hàm đo khoảng cách (Distance Metrics)

- Việc xác định "gần nhất" phụ thuộc vào cách đo khoảng cách giữa hai điểm dữ liệu $x = (x_1, x_2, \dots, x_n)$ và $y = (y_1, y_2, \dots, y_n)$. Các độ đo phổ biến bao gồm:

- **Khoảng cách Euclid (Euclidean Distance):**

$$d(x, y) = \sqrt{\sum_{i=1}^n (x_i - y_i)^2}$$

- **Khoảng cách Manhattan (Manhattan Distance):**

$$d(x, y) = \sqrt{\sum_{i=1}^n |x_i - y_i|}$$

- **Khoảng cách Minkowski:** Là dạng tổng quát của Euclid và Manhattan (Khi $p = 1$ là Manhattan, $p = 2$ là Euclid).

$$d(x, y) = \left(\sum_{i=1}^n |x_i - y_i|^p \right)^{\frac{1}{p}}$$

b. Lựa chọn tham số k

- Giá trị k là siêu tham số quan trọng nhất ảnh hưởng đến hiệu suất mô hình:
- **Nếu k quá nhỏ (ví dụ $k = 1$):** Mô hình rất nhạy cảm với nhiễu (noise) và các điểm ngoại lai (outliers), dễ dẫn đến hiện tượng Overfitting.
- **Nếu k quá lớn:** Mô hình trở nên quá mượt, có thể bao gồm cả các điểm từ các lớp khác ở xa, làm mất đi các chi tiết cục bộ quan trọng, dễ dẫn đến Underfitting.
- **Cách chọn k :** Thường chọn k là một số lẻ (để tránh trường hợp hòa phiếu trong phân loại nhị phân) và sử dụng phương pháp Cross-validation (như GridSearch) để tìm k tối ưu.

2.3.3.4 Ưu điểm và Nhược điểm

- **Ưu điểm:**

- **Đơn giản và trực quan:** Thuật toán dễ cài đặt, dễ giải thích và không đòi hỏi giả định phức tạp về dữ liệu.
- **Không tốn thời gian huấn luyện:** Do đặc thù "Lazy Learning", kNN bỏ qua bước huấn luyện, giúp tiết kiệm thời gian ở giai đoạn đầu.
- **Linh hoạt với dữ liệu mới:** Dễ dàng cập nhật mô hình bằng cách thêm trực tiếp dữ liệu mới vào bộ nhớ mà không cần huấn luyện lại.
- **Hiệu quả với bài toán đa lớp:** Xử lý tự nhiên các bài toán phân loại nhiều lớp (Multi-class) mà không cần kỹ thuật hỗ trợ.

— **Nhược điểm:**

- **Tốc độ dự đoán chậm:** Chi phí tính toán cao khi quy mô dữ liệu lớn do phải quét toàn bộ tập huấn luyện để tìm láng giềng.
- **Tốn tài nguyên bộ nhớ:** Yêu cầu dung lượng lưu trữ lớn để chứa toàn bộ dữ liệu gốc.
- **Nhạy cảm với nhiễu:** Dễ bị ảnh hưởng bởi các điểm ngoại lai (outliers) hoặc các đặc trưng không quan trọng, làm giảm độ chính xác.
- **Bắt buộc chuẩn hóa dữ liệu:** Nhạy cảm với sự chênh lệch đơn vị đo, do đó việc đưa dữ liệu về cùng thang đo (Scaling) là bước bắt buộc.

2.3.4 MLP

2.3.4.1 Giới thiệu

Mạng nơ-ron nhiều lớp (Multi-Layer Perceptron - MLP) là một mô hình học máy thuộc nhóm Artificial Neural Networks (ANN), được sử dụng rộng rãi trong các bài toán hồi quy (regression) và phân loại (classification). MLP là sự mở rộng của Perceptron đơn lớp bằng cách bổ sung thêm một hoặc nhiều lớp ẩn (hidden layers), cho phép mô hình học được các mối quan hệ phi tuyến phức tạp trong dữ liệu.

2.3.4.2 Nguyên lý hoạt động

Cấu trúc của MLP

Một mạng MLP tiêu chuẩn bao gồm ba thành phần chính:

- **Lớp đầu vào (Input layer):** nhận vector đặc trưng đầu vào $\mathbf{x} = [x_1, x_2, \dots, x_d]$
- **Một hoặc nhiều lớp ẩn (Hidden layers):** Bao gồm một hoặc nhiều lớp nơ-ron, thực hiện các phép biến đổi tuyến tính kết hợp với hàm kích hoạt phi tuyến. Đây là thành phần quan trọng nhất giúp MLP học được các mối quan hệ phức tạp trong dữ liệu.
- **Lớp đầu ra (Output layer):** đưa ra giá trị dự đoán hoặc xác suất phân lớp

Mỗi nơ-ron trong mạng thực hiện phép biến đổi tuyến tính kết hợp với hàm kích hoạt phi tuyến.

2.3.4.3 Các thành phần chính của thuật toán

a. Nguyên lý lan truyền xuôi (Forward Propagation)

Tại mỗi nơ-ron trong mạng MLP, đầu vào được xử lý qua hai bước: biến đổi tuyến tính và áp dụng hàm kích hoạt.

Với một nơ-ron tại lớp thứ l , giá trị tổng hợp được tính như sau:

$$z^{(l)} = \sum_{i=1}^n w_i^{(l)} x_i^{(l-1)} + b^{(l)}$$

Sau đó, đầu ra của nơ-ron được tính bằng hàm kích hoạt:

$$a^{(l)} = f(z^{(l)})$$

Một trong những hàm kích hoạt phổ biến nhất trong MLP là ReLU (Rectified Linear Unit):

$$f(z) = \max(0, z)$$

ReLU giúp tăng tốc độ hội tụ của mô hình và giảm hiện tượng mất gradient so với các hàm kích hoạt truyền thống như Sigmoid hoặc Tanh.

trong đó:

- $w_i^{(l)}$ là trọng số,
- $b^{(l)}$ là bias,

- $f(\cdot)$ là hàm kích hoạt (ReLU, Sigmoid, Tanh,...),
- $x_i^{(l-1)}$ là đầu ra của lớp trước đó
- $a^{(l)}$ là đầu ra của nơ-ron tại lớp l .

b. Hàm Softmax

Đối với bài toán phân loại đa lớp, lớp đầu ra của MLP thường sử dụng hàm Softmax để chuyển các giá trị đầu ra thành xác suất:

$$\hat{y}_c = \frac{e^{z_c}}{\sum_{k=1}^c e^{z_k}}$$

trong đó:

- với c là số lớp
- $\hat{y}_c$ là xác suất mẫu thuộc về lớp c

Kết quả dự đoán cuối cùng là lớp có xác suất lớn nhất.

c. Hàm mất mát (Loss Function)

Trong bài toán phân loại, để đo lường mức độ sai lệch giữa dự đoán của mô hình và nhãn thực tế, MLP thường sử dụng Cross-Entropy Loss, được định nghĩa như sau:

$$L = -\frac{1}{N} \sum_{i=1}^N \sum_{c=1}^C y_{i,c} \log(\hat{y}_{i,c})$$

trong đó:

- N là số mẫu huấn luyện,
- $y_{i,c}$ là nhãn thực (one-hot),
- $\hat{y}_{i,c}$ là xác suất dự đoán của mô hình.

Hàm mất mát này phạt nặng các dự đoán sai với độ tin cậy cao, giúp mô hình học hiệu quả hơn trong bài toán phân loại nhiều lớp.

d. Lan truyền ngược và cập nhật trọng số - tối ưu hóa (Backpropagation)

MLP được huấn luyện bằng thuật toán Backpropagation, trong đó gradient của hàm mất mát theo từng tham số được tính bằng quy tắc

dây chuyền. Các trọng số và bias của mạng được cập nhật thông qua thuật toán tối ưu, ví dụ Stochastic Gradient Descent (SGD):

$$w^{(t+1)} = w^{(t)} - \eta \frac{\partial L}{\partial w}$$

với:

- η là learning rate.
- t là số vòng lặp huấn luyện.

Quá trình này được lặp lại qua nhiều epoch cho đến khi hàm mất mát hội tụ.

e. Các siêu tham số quan trọng của MLP

Một số siêu tham số ảnh hưởng mạnh đến hiệu năng của MLP bao gồm:

- Số lớp ẩn và số nơ-ron mỗi lớp: Quyết định độ phức tạp của mô hình.
- Learning rate: Điều khiển tốc độ học của mô hình.
- Batch size: Ảnh hưởng đến độ ổn định và tốc độ huấn luyện.
- Epochs: Số vòng lặp huấn luyện.
- Hàm kích hoạt: ReLU, Sigmoid, Tanh.

Việc lựa chọn các siêu tham số này cần được cân nhắc để tránh hiện tượng overfitting hoặc underfitting.

2.3.4.4 Ưu điểm và Nhược điểm của MLP

— Ưu điểm:

- Có khả năng mô hình hóa các mối quan hệ phi tuyến phức tạp
- Hoạt động tốt với dữ liệu có nhiều đặc trưng
- Linh hoạt, có thể áp dụng cho nhiều dạng bài toán khác nhau

— Nhược điểm:

- Dễ bị overfitting nếu số lớp ẩn và số nơ-ron quá lớn
- Cần nhiều dữ liệu và tài nguyên tính toán
- Khó diễn giải hơn so với các mô hình cây quyết định

2.3.4.5 Lý do lựa chọn MLP trong bài toán

Trong bài toán phân loại lỗi hệ thống điện, dữ liệu đầu vào thường có nhiều đặc trưng và mang tính phi tuyến. MLP với các lớp ẩn và hàm kích hoạt phi tuyến có khả năng học được các mẫu đặc trưng phức tạp này tốt hơn so với các mô hình tuyến tính. Do đó, MLP được lựa chọn như một mô hình mạnh để so sánh hiệu năng với các thuật toán truyền thống như Random Forest, SVM và kNN.

Việc đưa MLP vào so sánh cùng Random Forest, SVM và kNN giúp đánh giá:

- Khả năng học phi tuyến của mô hình học sâu cơ bản.
- Mức độ cải thiện hiệu suất so với các thuật toán học máy truyền thống.
- Tính phù hợp của MLP đối với bài toán phân loại lỗi hệ thống điện.

CHƯƠNG 3. THỰC NGHIỆM

3.1 Dataset

Nhóm sử dụng: <https://www.kaggle.com/datasets/esathyaprakash/electrical-fault-detection-and-classification/data>

Tập này chứa tập dữ liệu để phân loại các loại lỗi.

Đầu vào - [Ia, Ib, Ic, Va, Vb, Vc]

Nhấn đầu ra GCBA được ánh xạ thành các lớp lỗi:

- **'0000': 'No Fault'**

Lỗi 1 pha chạm đất (Single Line to Ground):

- '1000': 'Single Line to Ground A',
- '0100': 'Single Line to Ground B',
- '0010': 'Single Line to Ground C',

Lỗi 2 pha (Line-to-Line):

- '0011': 'Line-to-Line BC',
- '0101': 'Line-to-Line AC',
- '1001': 'Line-to-Line AB',

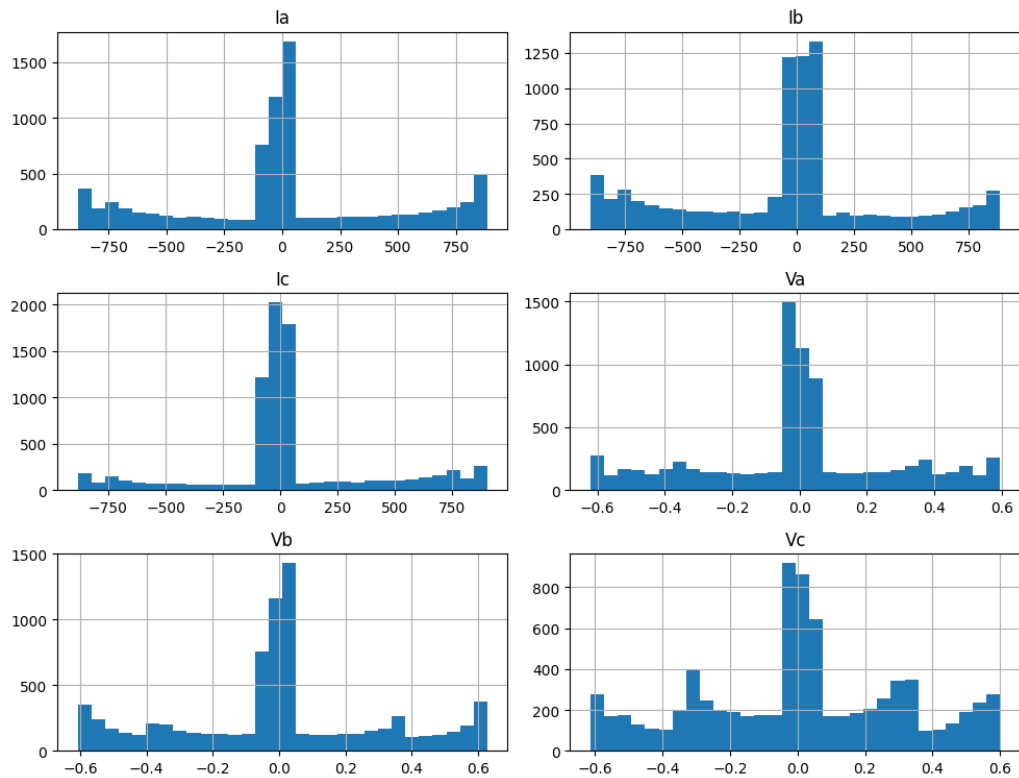
Lỗi 2 pha chạm đất (Line-to-Line with Ground):

- '1010': 'Line-to-Line with Ground AB',
- '1101': 'Line-to-Line with Ground AC',
- '0110': 'Line-to-Line with Ground BC',
- '1011': 'Line A Line B to Ground Fault',

Lỗi 3 pha (Three-Phase):

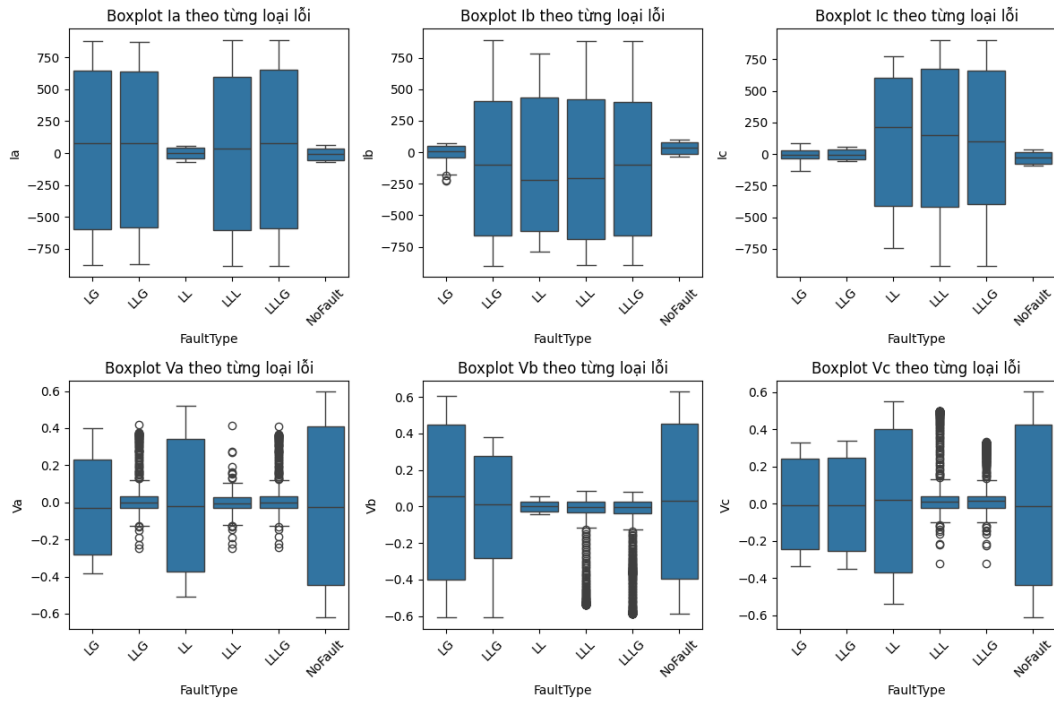
- '0111': 'Three-Phase',
- '1111': 'Three-Phase with Ground'

Histogram phân phối các đặc trưng dòng và áp



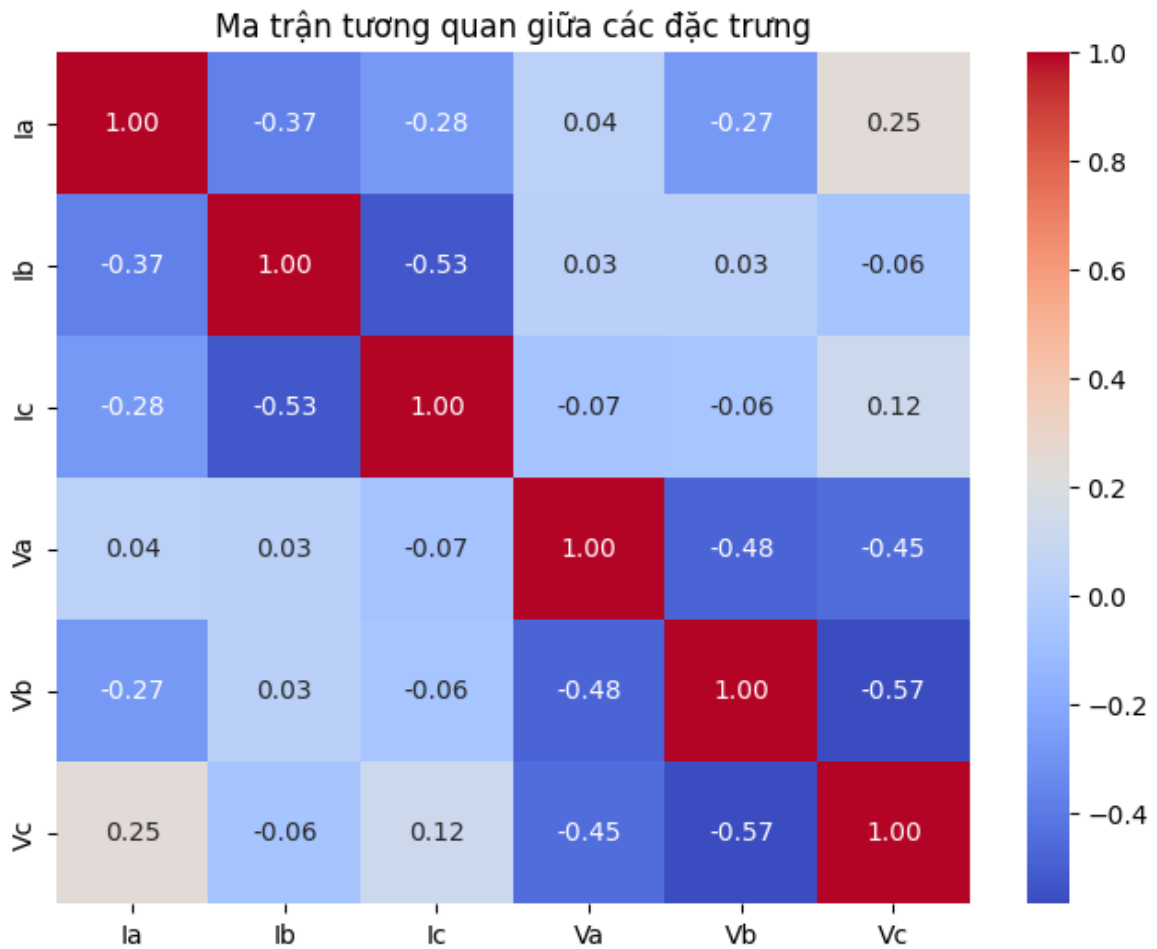
Hình 3.1: Histogram phân phối các đặc trưng

Hình này minh họa histogram phân phối của các đặc trưng dòng điện I_a, I_b, I_c và điện áp V_a, V_b, V_c trong tập dữ liệu. Có thể nhận thấy các đặc trưng dòng điện có miền giá trị rộng và độ biến thiên lớn, trong khi các đặc trưng điện áp có miền giá trị hẹp hơn và tập trung quanh giá trị 0. Sự khác biệt rõ rệt về thang đo giữa các đặc trưng cho thấy dữ liệu cần được chuẩn hóa trước khi đưa vào huấn luyện các mô hình học máy nhằm đảm bảo hiệu quả và độ ổn định trong quá trình huấn luyện.



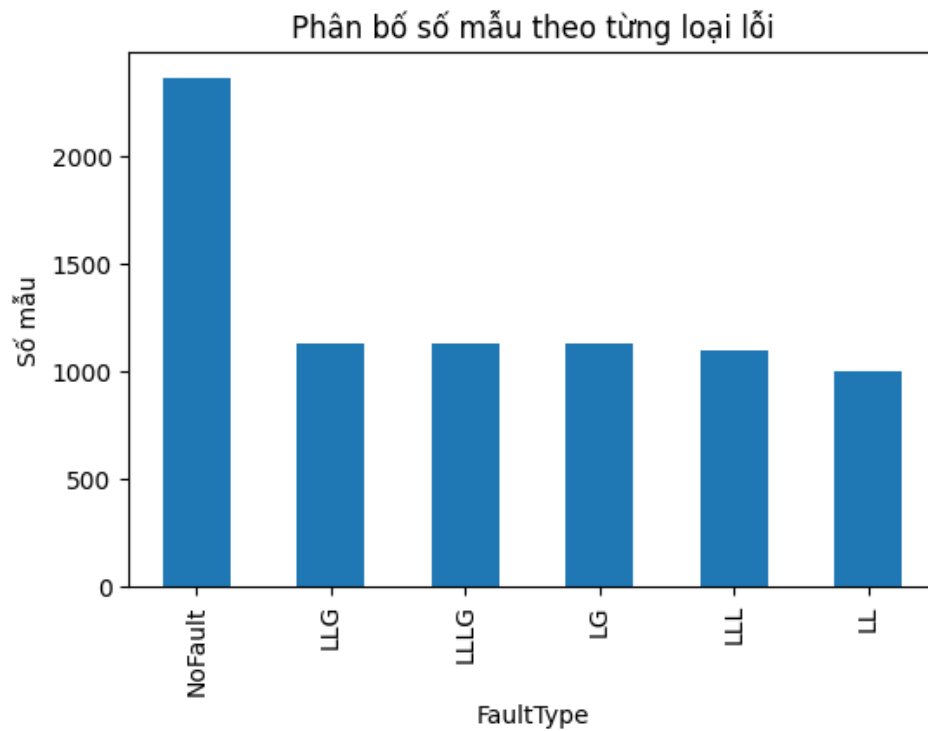
Hình 3.2: Boxplot các đặc trưng

Hình này trình bày boxplot của các đặc trưng dòng điện I_a, I_b, I_c và điện áp V_a, V_b, V_c theo từng loại lỗi. Có thể nhận thấy sự khác biệt rõ rệt về phân bố và độ biến thiên của các đặc trưng giữa các nhóm lỗi, đặc biệt đối với các đặc trưng dòng điện. Các loại lỗi khác nhau tạo ra các khoảng giá trị và median khác nhau, trong khi trạng thái *No Fault* có phân bố ổn định hơn. Điều này cho thấy các đặc trưng đầu vào có khả năng phân biệt các loại lỗi và phù hợp để sử dụng trong các mô hình phân loại.



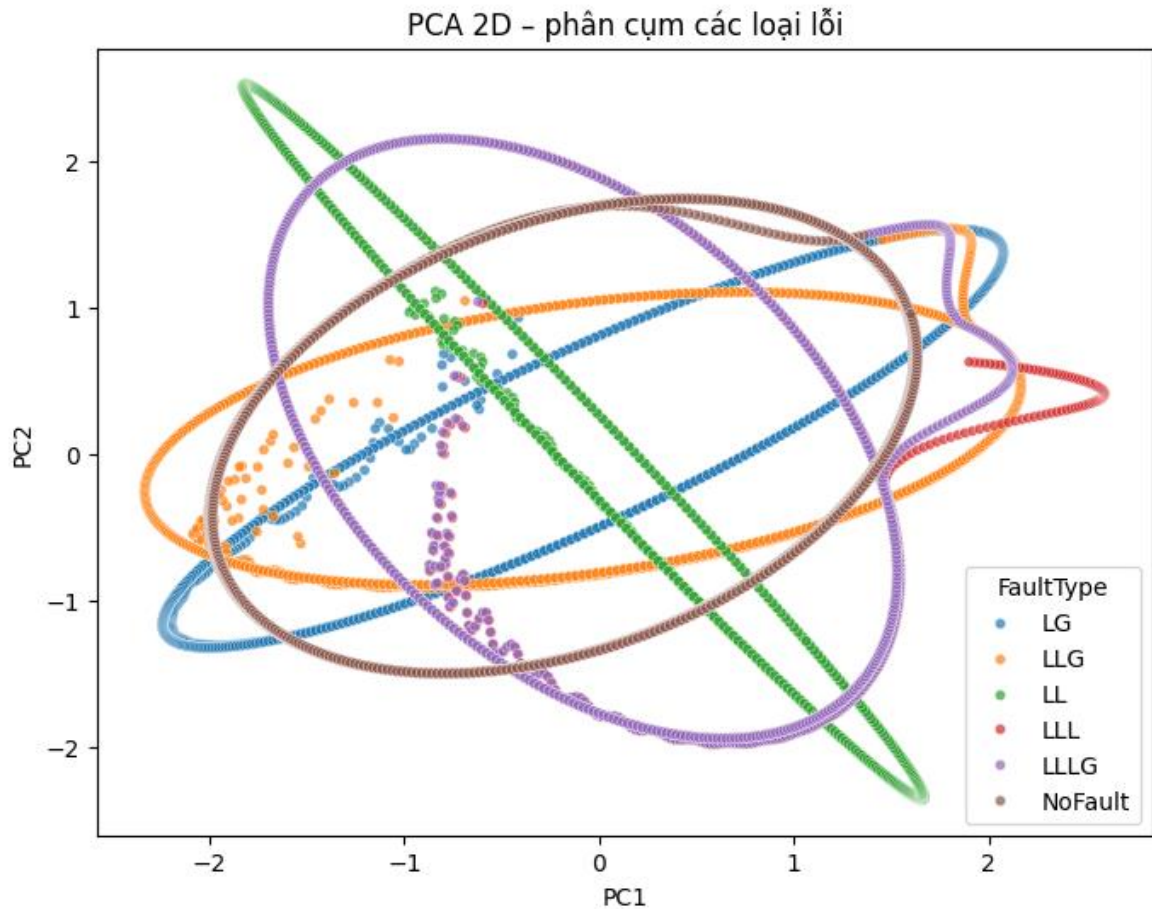
Hình 3.3: Ma trận tương quan giữa các đặc trưng

Hình này thể hiện ma trận tương quan giữa các đặc trưng dòng điện I_a, I_b, I_c và điện áp V_a, V_b, V_c . Có thể quan sát thấy một số cặp đặc trưng có mức độ tương quan trung bình, đặc biệt giữa các pha dòng điện và giữa các pha điện áp, trong khi mối tương quan giữa dòng điện và điện áp nhìn chung không cao. Điều này cho thấy các đặc trưng đầu vào mang thông tin tương đối độc lập, góp phần nâng cao khả năng học và phân biệt của các mô hình phân loại.



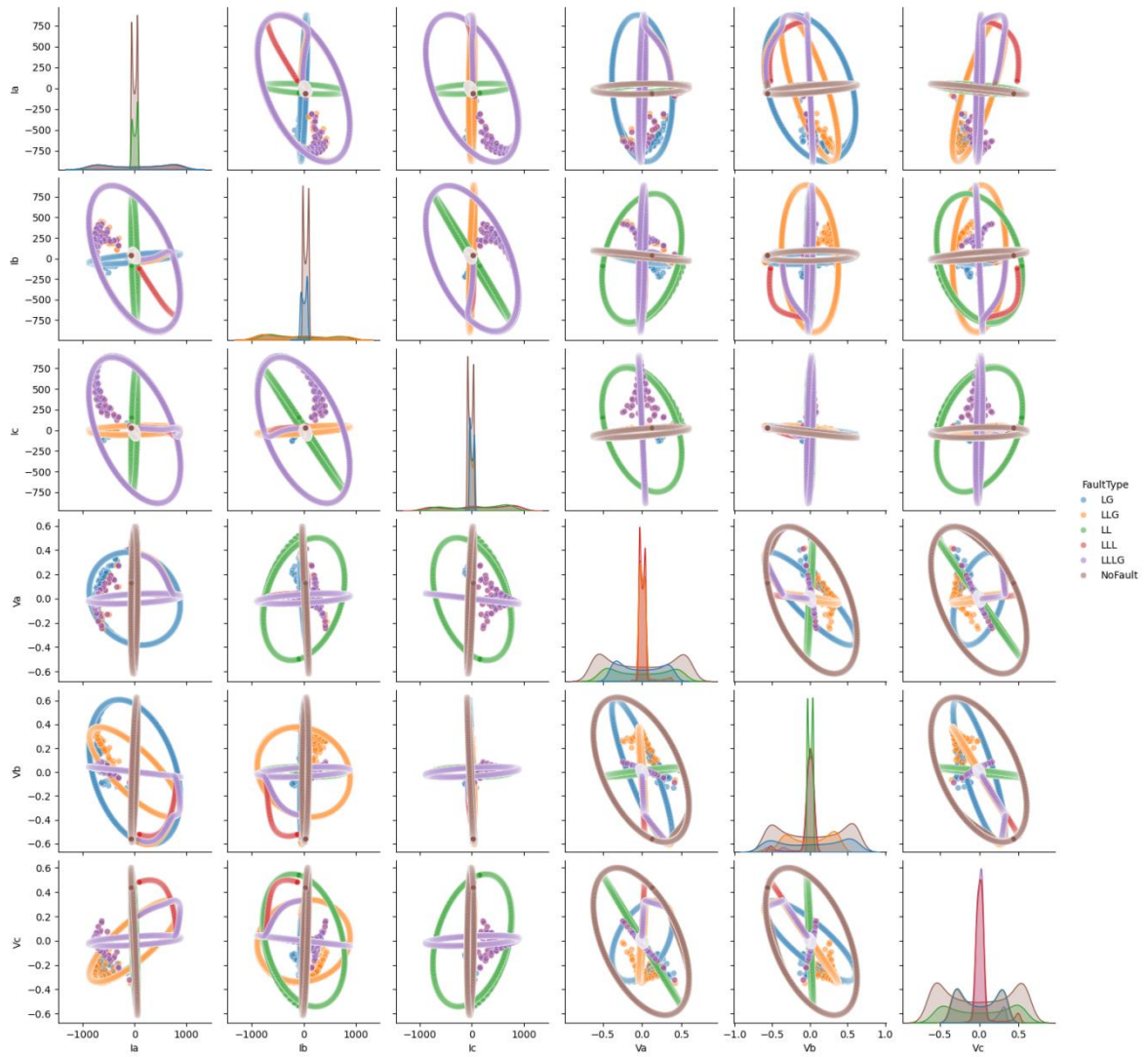
Hình 3.4: Phân bố số mẫu theo từng loại lỗi

Hình này thể hiện phân bố số lượng mẫu theo từng loại lỗi trong tập dữ liệu. Có thể nhận thấy số lượng mẫu giữa các lớp không hoàn toàn đồng đều, trong đó lớp *No Fault* chiếm số lượng lớn hơn so với các loại lỗi còn lại. Sự mất cân bằng nhẹ giữa các lớp có thể ảnh hưởng đến quá trình huấn luyện mô hình, do đó cần được xem xét trong bước xây dựng và đánh giá các mô hình phân loại.



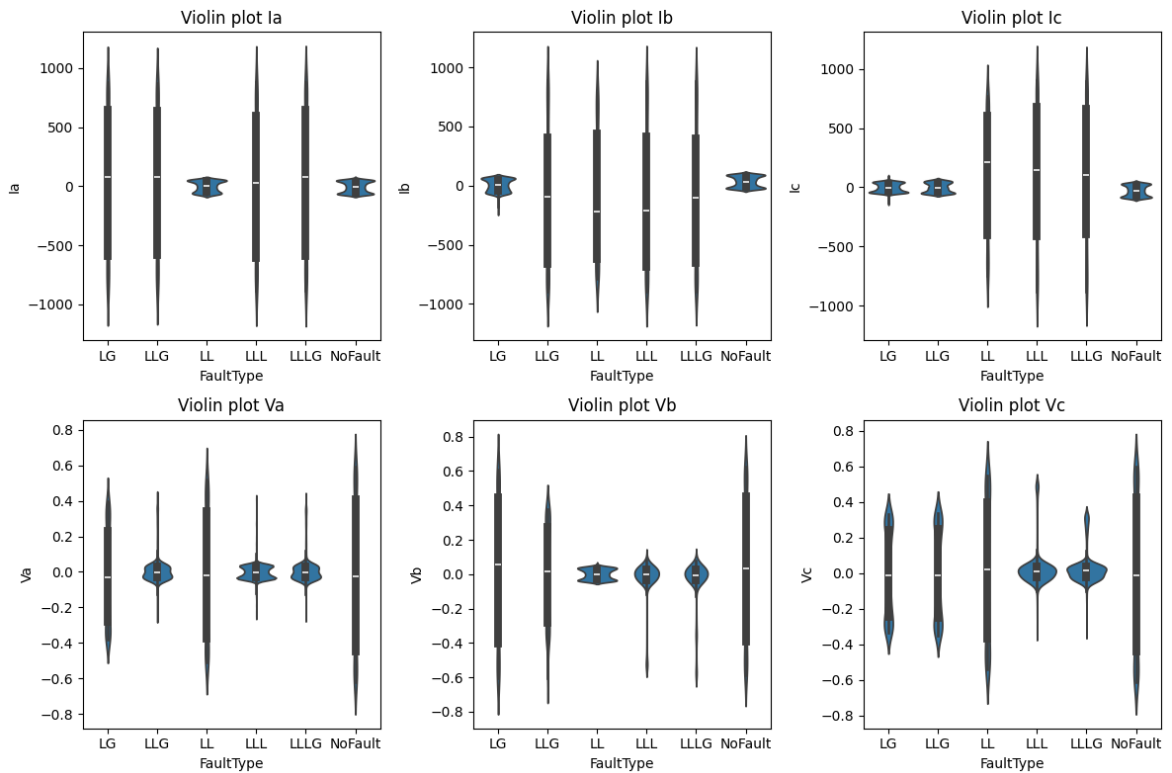
Hình 3.5: Phân cụm các loại lỗi

Hình này minh họa kết quả giảm chiều dữ liệu bằng PCA xuống không gian hai chiều nhằm trực quan hóa sự phân bố của các loại lỗi. Có thể quan sát thấy các mẫu dữ liệu tạo thành các cụm tương đối khác nhau theo từng loại lỗi, tuy vẫn tồn tại sự chồng lấn giữa một số nhóm. Điều này cho thấy dữ liệu có khả năng phân biệt giữa các lớp, đồng thời phản ánh tính chất phi tuyến và phức tạp của bài toán phân loại lỗi hệ thống điện.



Hình 3.6: Mối quan hệ cặp giữa các đặc trưng

Hình này thể hiện mối quan hệ cặp giữa các đặc trưng dòng điện và điện áp theo từng loại lỗi. Quan sát cho thấy các đặc trưng có mối quan hệ phi tuyến và sự chồng lấn giữa các nhóm lỗi ở nhiều không gian hai chiều, đồng thời vẫn tồn tại những vùng phân bố khác biệt giữa các loại lỗi. Điều này cho thấy bài toán phân loại có tính phức tạp và đòi hỏi các mô hình có khả năng học được mối quan hệ phi tuyến giữa các đặc trưng.



Hình 3.7: Violin plot các đặc trưng

Hình này minh họa violin plot của các đặc trưng dòng điện và điện áp theo từng loại lỗi. Có thể quan sát rõ sự khác biệt về phân bố và mật độ giá trị giữa các nhóm lỗi, đặc biệt ở các đặc trưng dòng điện với độ biến thiên lớn. Các violin plot cho thấy mỗi loại lỗi có dạng phân bố đặc trưng riêng, trong khi trạng thái *No Fault* có phân bố tập trung và ổn định hơn, cho thấy dữ liệu đầu vào mang thông tin hữu ích cho bài toán phân loại.

3.2 Random Forest

Trong phần này, nhóm tiến hành xây dựng và đánh giá mô hình Random Forest để phân loại các loại lỗi trong hệ thống điện ba pha dựa trên dữ liệu dòng - áp. Mục tiêu của phần này gồm:

- Huấn luyện mô hình Random Forest để dự đoán chính xác tên các loại lỗi (Fault Types) từ dữ liệu đầu vào.
- Đánh giá mô hình bằng Accuracy, Precision, Recall và F1-score.
- Tối ưu siêu tham số bằng GridSearchCV để cải thiện hiệu suất.

- So sánh hiệu quả giữa mô hình baseline và mô hình tối ưu.

Trước khi đưa vào mô hình, dữ liệu được thiết lập như sau:

- **Train/Test Split:** 80% Huấn luyện (Train) - 20% Kiểm thử (Test).
- **Random State:** 42 (Đảm bảo tính nhất quán khi chia dữ liệu).
- **Stratify:** Phân tầng theo nhãn lỗi để đảm bảo tỷ lệ các lớp lỗi cân bằng giữa tập huấn luyện và kiểm thử.

Mô hình khởi tạo ban đầu được thiết lập với các tham số cơ bản nhằm đánh giá hiệu năng trước khi tối ưu:

- **Thuật toán:** Random Forest Classifier.
- **n_estimators:** 300
- **Random State:** 42
- **Các tham số khác:** Giữ giá trị mặc định của thư viện Scikit-learn.

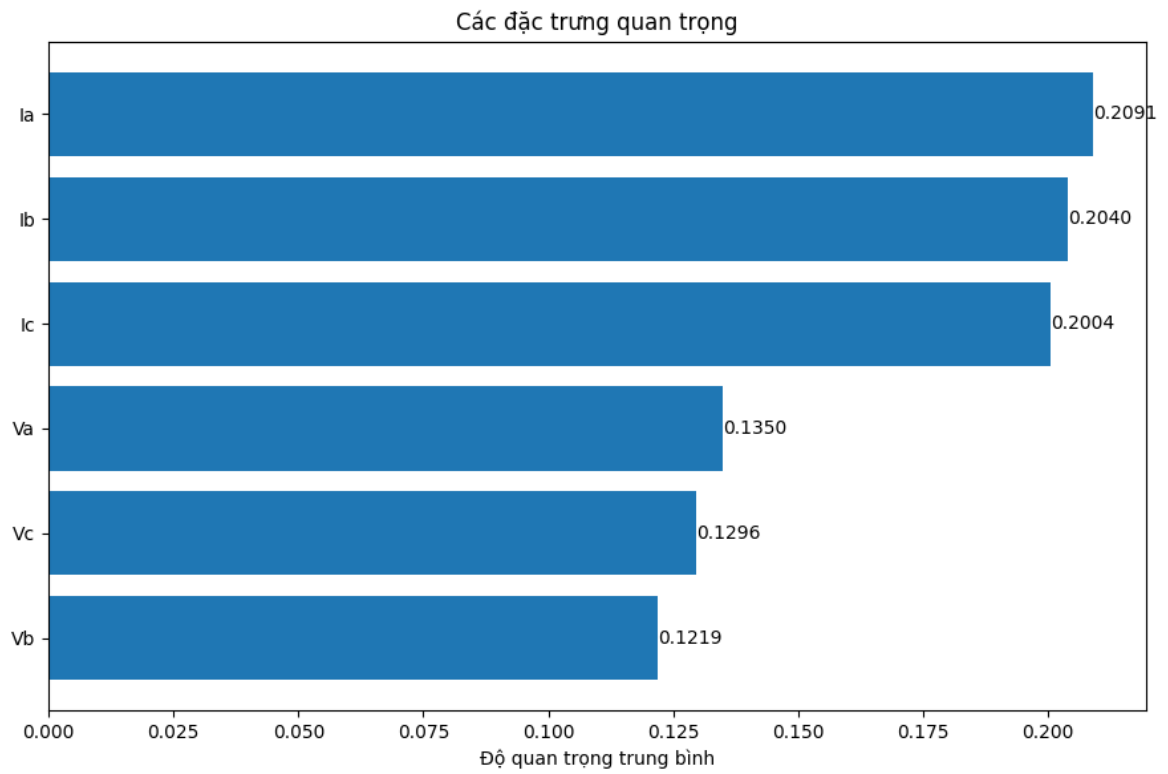
Tối ưu với GridSearchCV, không gian tham số thử nghiệm bao gồm:

- n_estimators [200, 300, 500]
- max_depth [None, 10, 20, 30]
- min_samples_split [2, 5, 10] & min_samples_leaf [1, 2, 4]
- class_weight [None, 'balanced']

Sử dụng:

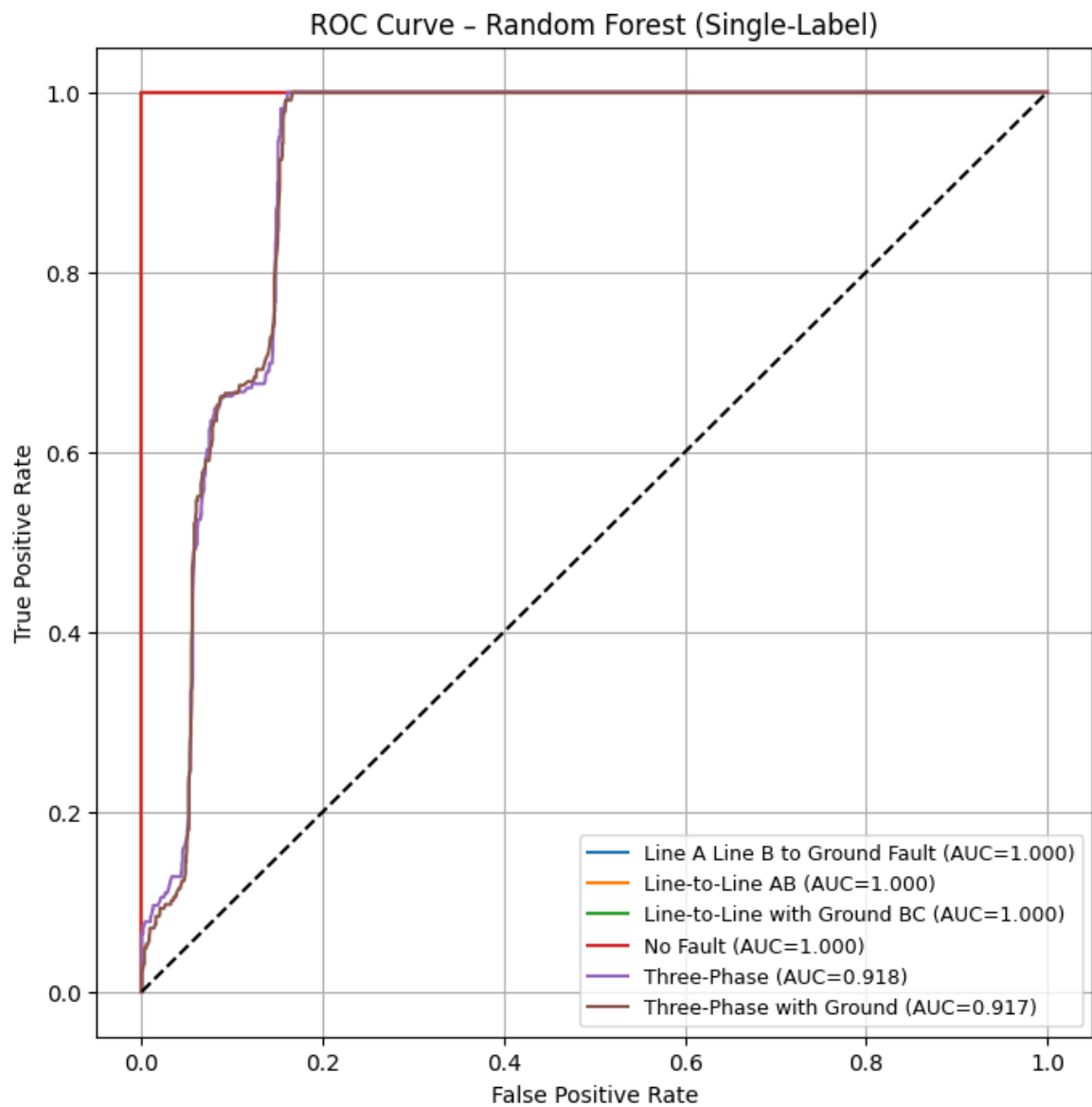
- Cross-validation: **3-fold**
- Thước đo: **F1-weighted**

Kết quả huấn luyện của mô hình Baseline:



Hình 3.8: Các đặc trưng quan trọng

Biểu đồ minh họa mức độ đóng góp trung bình của từng đặc trưng đầu vào đối với quyết định phân loại của mô hình.



Hình 3.9: ROC Curve

Dựa vào biểu đồ ta có:

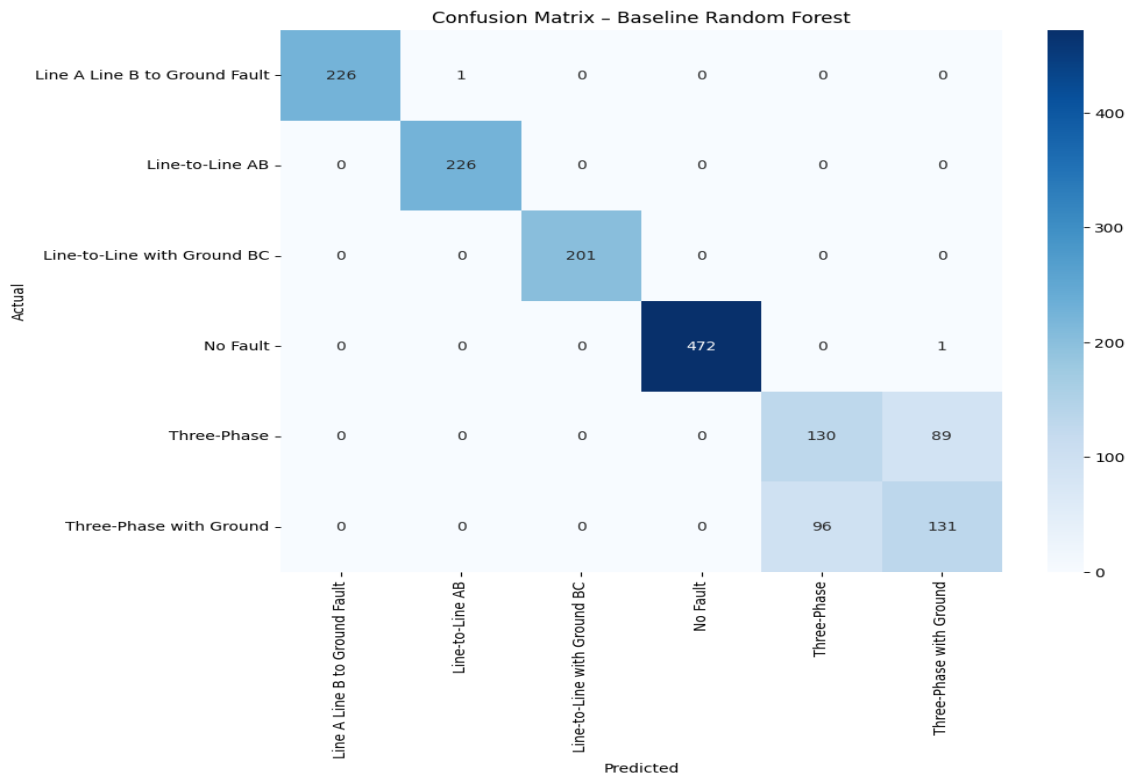
Bảng 3.1: Kết quả AUC

Loại lỗi (Fault Type)	AUC
No Fault (Không lỗi)	1.000
Line-to-Line AB (Lỗi 2 pha AB)	1.000
Line A Line B to Ground Fault (Lỗi 2 pha AB chạm đất)	1.000
Line-to-Line with Ground BC (Lỗi 2 pha BC chạm đất)	1.000
Three-Phase (Lỗi 3 pha cân bằng)	0.918
Three-Phase with Ground (Lỗi 3 pha chạm đất)	0.917

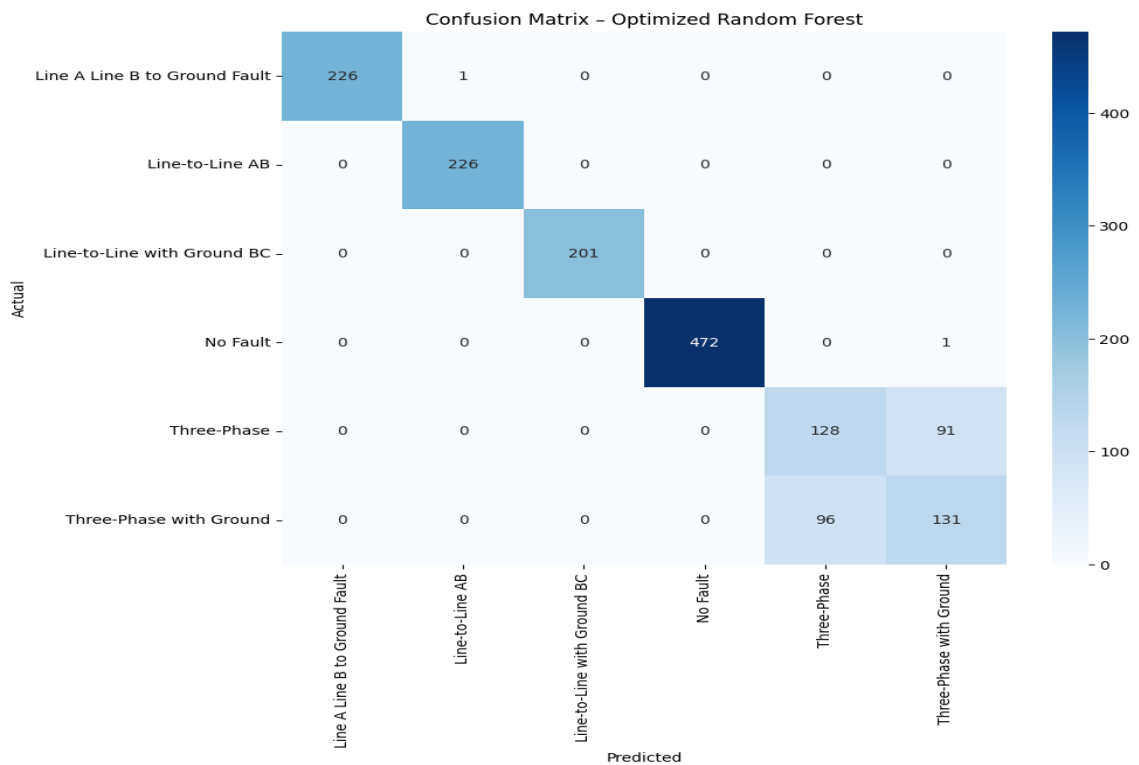
*Nhận xét :

- Dựa trên số liệu, hiệu năng mô hình phân hóa thành hai nhóm rõ rệt. Nhóm đạt độ chính xác tuyệt đối ($AUC = 1.000$) bao gồm trạng thái bình thường và các lỗi bất đối xứng, nhờ các đặc trưng dòng-áp khác biệt giúp phân loại chính xác 100%. Ngược lại, nhóm lỗi 3 pha có AUC thấp hơn (≈ 0.918) do tính chất đối xứng tương đồng khiến việc phân tách tín hiệu dòng chạm đất gặp khó khăn.
- **Kết luận:** Mô hình nhận diện hoàn hảo các lỗi phổ biến và trạng thái an toàn. Đối với sự cố 3 pha nghiêm trọng, dù còn nhầm lẫn nhỏ về yếu tố chạm đất, mô hình vẫn xác định đúng bản chất sự cố để kích hoạt ngắt mạch bảo vệ kịp thời.

Đánh giá: So sánh đánh giá giữa 2 mô hình Baseline và Optimized (Tối ưu bằng GridSearchCV):



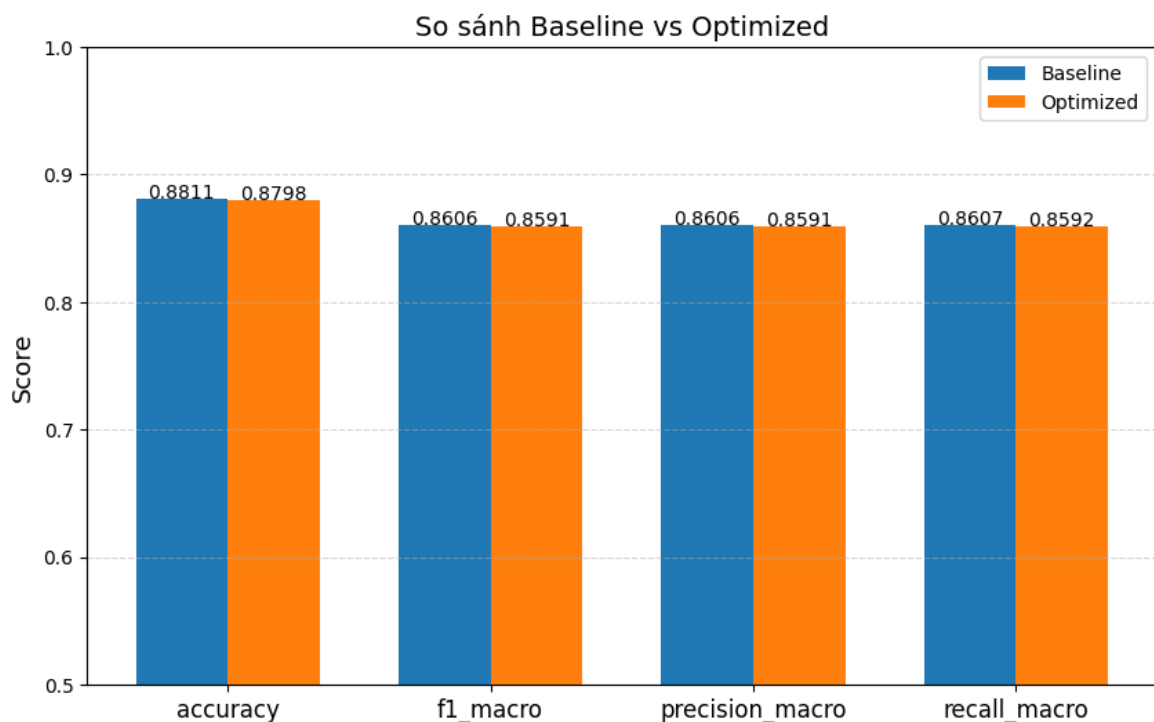
Hình 3.10: Confusion Matrix - Baseline



Hình 3.11: Confusion Matrix - Optimized

Nhận xét:

- **Điểm tốt:** Cả hai mô hình đều nhận diện chính xác tuyệt đối các sự cố 2 pha và trạng thái bình thường.
 - **Điểm yếu:** Cả hai đều bị "lẫn lộn" giữa lỗi 3 pha và 3 pha chạm đất. Máy phát hiện được sự cố 3 pha, nhưng không phân biệt rõ là có chạm đất hay không.
 - **So sánh:** Bản Tối ưu hóa (Optimized) thực tế lại kém hơn bản gốc (Baseline) một chút (số lần đoán đúng giảm từ 130 xuống 127).
- **Kết luận:** Việc tinh chỉnh mô hình không đem lại hiệu quả. Vấn đề nằm ở chỗ dữ liệu của hai loại sự cố 3 pha này quá giống nhau, khiến máy không thể tách biệt rạch ròi.



Hình 3.12: Baseline vs Optimized

- **Accuracy:**

Bảng 3.2: Accuracy

Mô hình	Accuracy
Baseline	0.8811
Optimized	0.8798

- Mô hình mặc định dự đoán đúng tổng số mẫu nhiều hơn mô hình đã qua tối ưu hóa một chút.

- **F1 Macro:**

Bảng 3.3: F1 Macro

Mô hình	F1 Macro
Baseline	0.8606
Optimized	0.8591

- Chỉ số F1 thấp hơn ở mô hình Optimized phản ánh đúng những gì chúng ta đã thấy ở Confusion Matrix trước đó (việc dự đoán sai ở lớp "Three-Phase" nhiều hơn làm kéo điểm F1 trung bình xuống)

- **Precision Macro:**

Bảng 3.4: Precision Macro

Mô hình	Precision Macro
Baseline	0.8606
Optimized	0.8591

- Khi mô hình Baseline dự báo một lỗi cụ thể, tỷ lệ dự báo đó là đúng sự thật cao hơn một chút so với bản Optimized.

- **Recall Macro:**

Bảng 3.5: Recall Macro

Mô hình	Recall Macro
Baseline	0.8607
Optimized	0.8592

- Mô hình Baseline có khả năng phát hiện (không bỏ sót) các lỗi tốt hơn so với mô hình Optimized.

Kết luận: Random Forest với tham số mặc định (Baseline) hoạt động hiệu quả và ổn định hơn. Việc tối ưu hóa có thể đã làm mô hình bị "quá khớp" (overfitting) nhẹ trên tập huấn luyện, dẫn đến kết quả giảm sút nhẹ khi kiểm tra thực tế.

	Ia	Ib	Ic	Va	Vb	Vc	True	Pred	Correct
7831	-71.063267	61.957937	6.229790	-0.141980	-0.429776	0.571755	No Fault	No Fault	True
3223	-21.867403	-552.152034	576.291226	-0.453081	-0.027293	0.480374	Line-to-Line with Ground BC	Line-to-Line with Ground BC	True
935	-21.626458	46.300115	-49.927065	-0.376616	0.573500	-0.196884	Line-to-Line AB	Line-to-Line AB	True
3011	-2.605458	653.546114	-648.352371	0.395122	0.020774	-0.415896	Line-to-Line with Ground BC	Line-to-Line with Ground BC	True
2551	-55.585910	736.729649	-678.585363	-0.020216	-0.017671	0.037887	Line-to-Line with Ground BC	Line-to-Line with Ground BC	True
2411	-10.604064	-651.192568	664.282793	-0.402025	-0.021316	0.423341	Line-to-Line with Ground BC	Line-to-Line with Ground BC	True
3114	56.587412	-459.439157	405.397072	0.287521	0.035916	-0.323437	Line-to-Line with Ground BC	Line-to-Line with Ground BC	True
2669	45.634583	59.726389	-102.650437	0.485733	0.041909	-0.527642	Line-to-Line with Ground BC	Line-to-Line with Ground BC	True
7323	-0.010992	82.194871	-85.203288	-0.583651	0.479995	0.103656	No Fault	No Fault	True
3257	-50.909473	-195.917137	249.066563	-0.507487	-0.039971	0.547458	Line-to-Line with Ground BC	Line-to-Line with Ground BC	True

Hình 3.13: Minh họa dự đoán của RF trên tập kiểm tra

3.3 SVM

Trong phần này, nhóm tiến hành xây dựng và đánh giá mô hình Support Vector Machine (SVM) để phân loại các loại lỗi trong hệ thống điện ba pha dựa trên bộ dữ liệu dòng - áp. Mục tiêu:

- Huấn luyện mô hình SVM với kernel RBF trên dữ liệu đã chuẩn hóa
- Đánh giá mô hình bằng các chỉ số Accuracy, Precision, Recall, F1
- Tối ưu mô hình bằng GridSearchCV
- So sánh hiệu quả giữa mô hình baseline và mô hình đã tối ưu
- Trực quan hóa kết quả bằng Confusion Matrix, ROC-AUC và PR Curve

Do SVM rất nhạy cảm với sự khác biệt về độ lớn giữa các đặc trưng, nhóm áp dụng **StandardScaler** để chuẩn hóa dữ liệu trước khi huấn luyện.

Dữ liệu được chia thành:

- 80% tập huấn luyện
- 20% tập kiểm thử
- Sử dụng **stratify** để giữ tỉ lệ các lớp lỗi như ban đầu.

Nhóm bắt đầu với mô hình SVM mặc định với kernel phi tuyến RBF - phù hợp cho dữ liệu không tuyến tính như dòng/áp ba pha.

Các tham số baseline:

- **Kernel:** RBF
- **C = 10**
- **Gamma = scale**
- **class_weight = balanced** giúp mô hình không thiên lệch lớp

Để tăng hiệu năng mô hình, nhóm tiến hành tối ưu bằng **GridSearchCV** nhằm tìm ra bộ siêu tham số tốt nhất của SVM.

Không gian tìm kiếm:

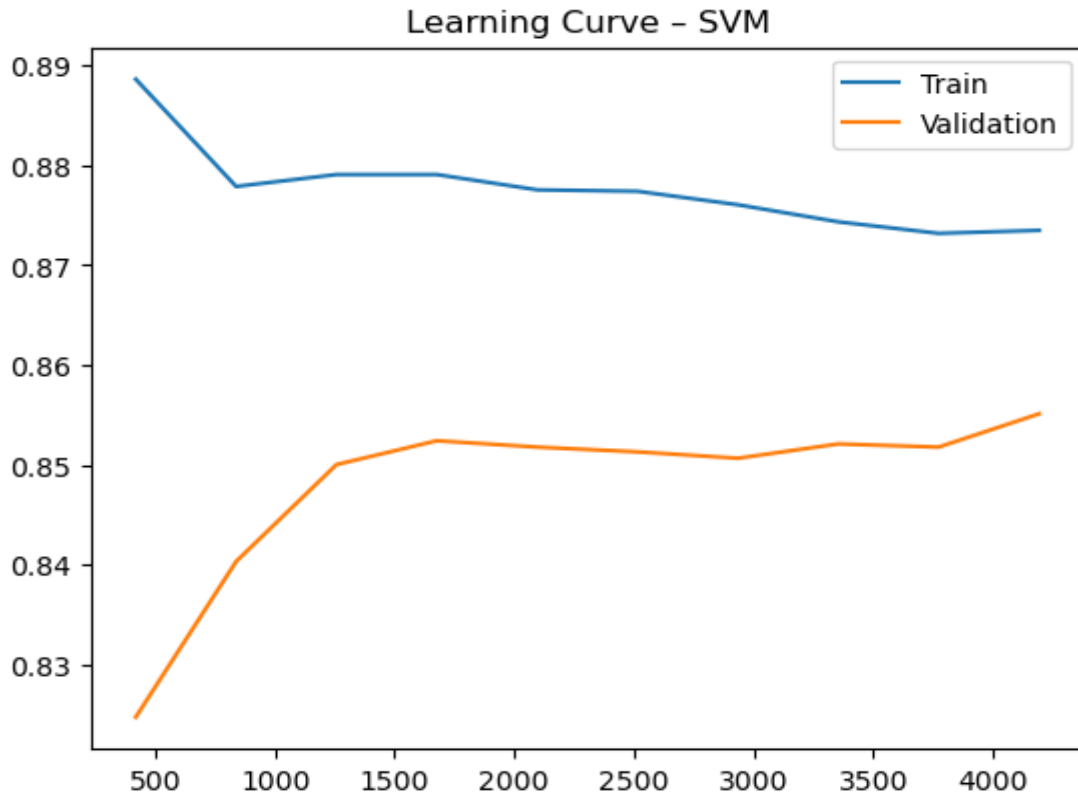
- C: [0.1, 1, 10, 50]
- Gamma: ['scale', 0.1, 0.01]
- Kernel: ['rbf']

GridSearchCV sử dụng:

- Cross-validation: **3-fold**

- Thước đo: **F1-weighted**

Kết quả:



Hình 3.14: Learning Curve

Learning curve cho thấy hai đường Train và Validation tiến gần nhau khi kích thước tập huấn luyện tăng.

Đường Train Accuracy ($\sim 0.88 \rightarrow 0.873$)

- Train score giảm nhẹ khi tăng số mẫu
- Điều này hoàn toàn bình thường, vì mô hình phải khái quát hóa hơn
- Không có dấu hiệu overfitting mạnh (train không quá cao so với validation)

Đường Validation Accuracy ($\sim 0.825 \rightarrow 0.855$)

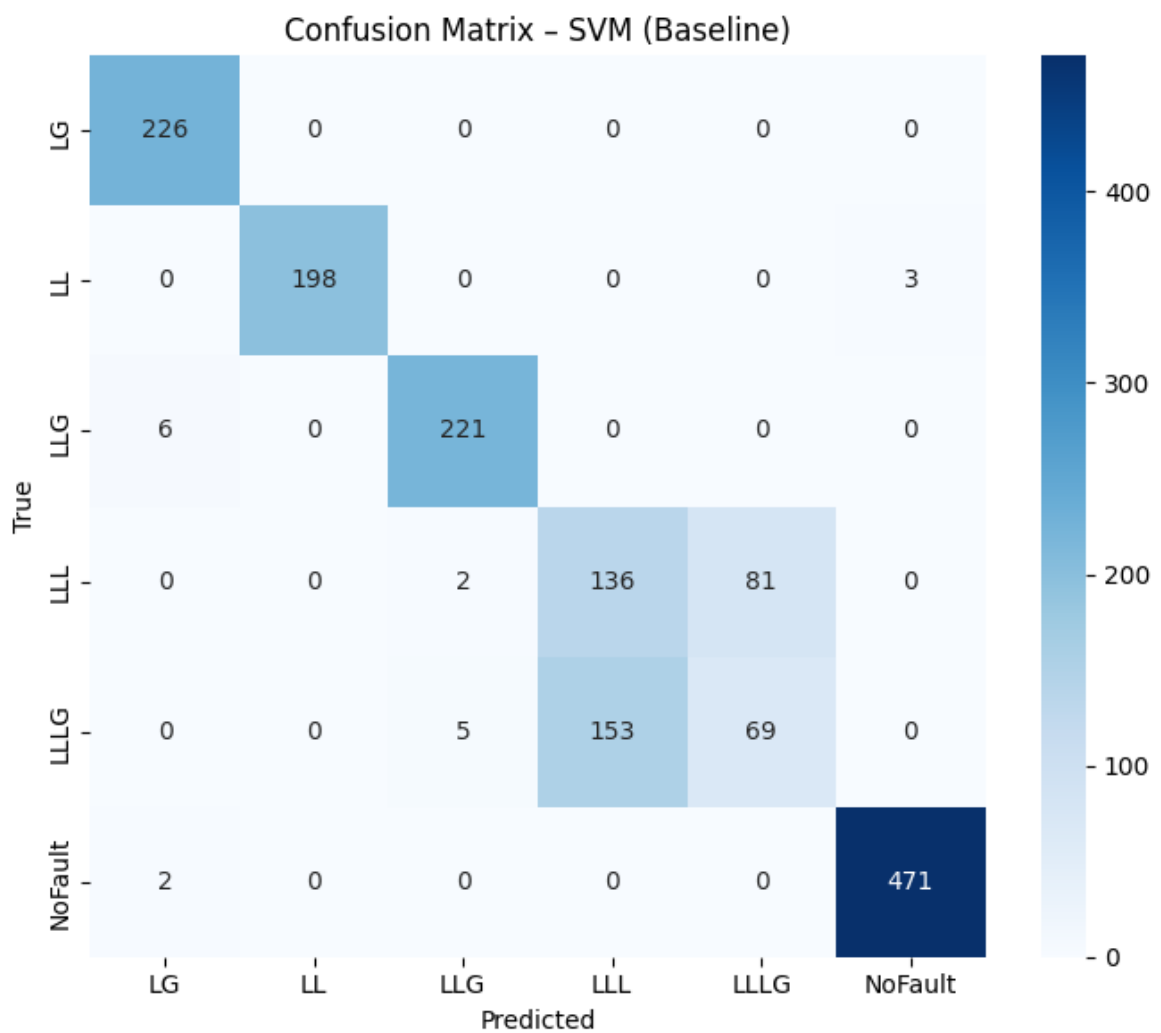
- Validation score tăng dần khi có nhiều dữ liệu hơn
- Cho thấy mô hình học tốt hơn khi dữ liệu lớn hơn
- Không có dấu hiệu underfitting (điểm validation ổn định và khá cao)

Hai đường tiến gần nhau

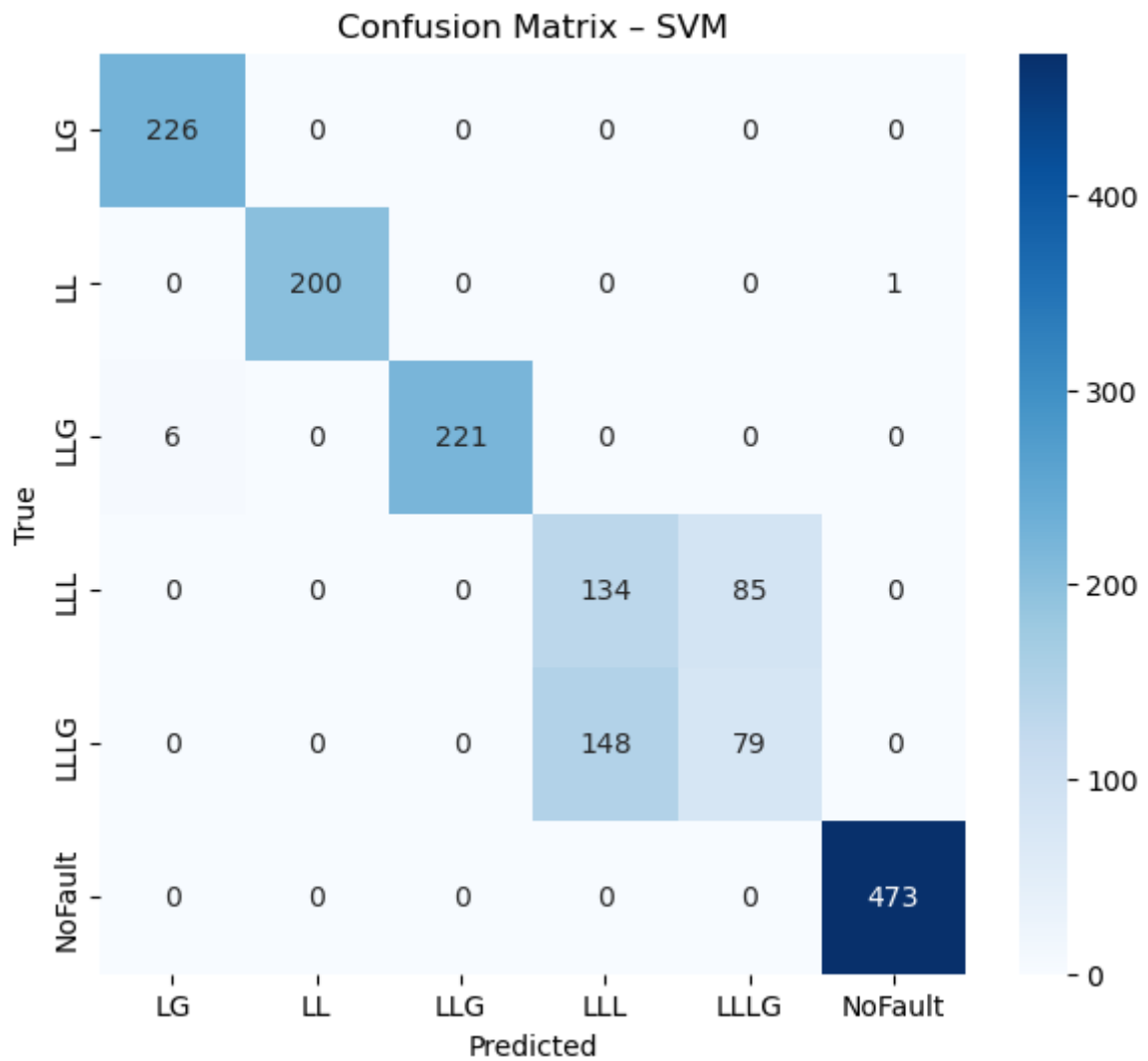
- Train và validation accuracy hội tụ quanh 0.85 - 0.87
- Khoảng cách nhỏ → mô hình tổng quát hóa tốt
- Mô hình không bị overfitting cũng không bị underfitting

Kết luận Learning Curve:

- Mô hình SVM hoạt động ổn định và học tốt khi kích thước dữ liệu tăng.
- Không có hiện tượng overfitting hoặc underfitting nghiêm trọng.
- Validation accuracy tăng dần chứng tỏ mô hình lợi ích rõ rệt khi có nhiều dữ liệu.
- Learning curve cho thấy SVM là lựa chọn phù hợp và đủ mạnh cho bài toán phân loại lỗi hệ thống điện.



Hình 3.15: Confusion Matrix - Baseline

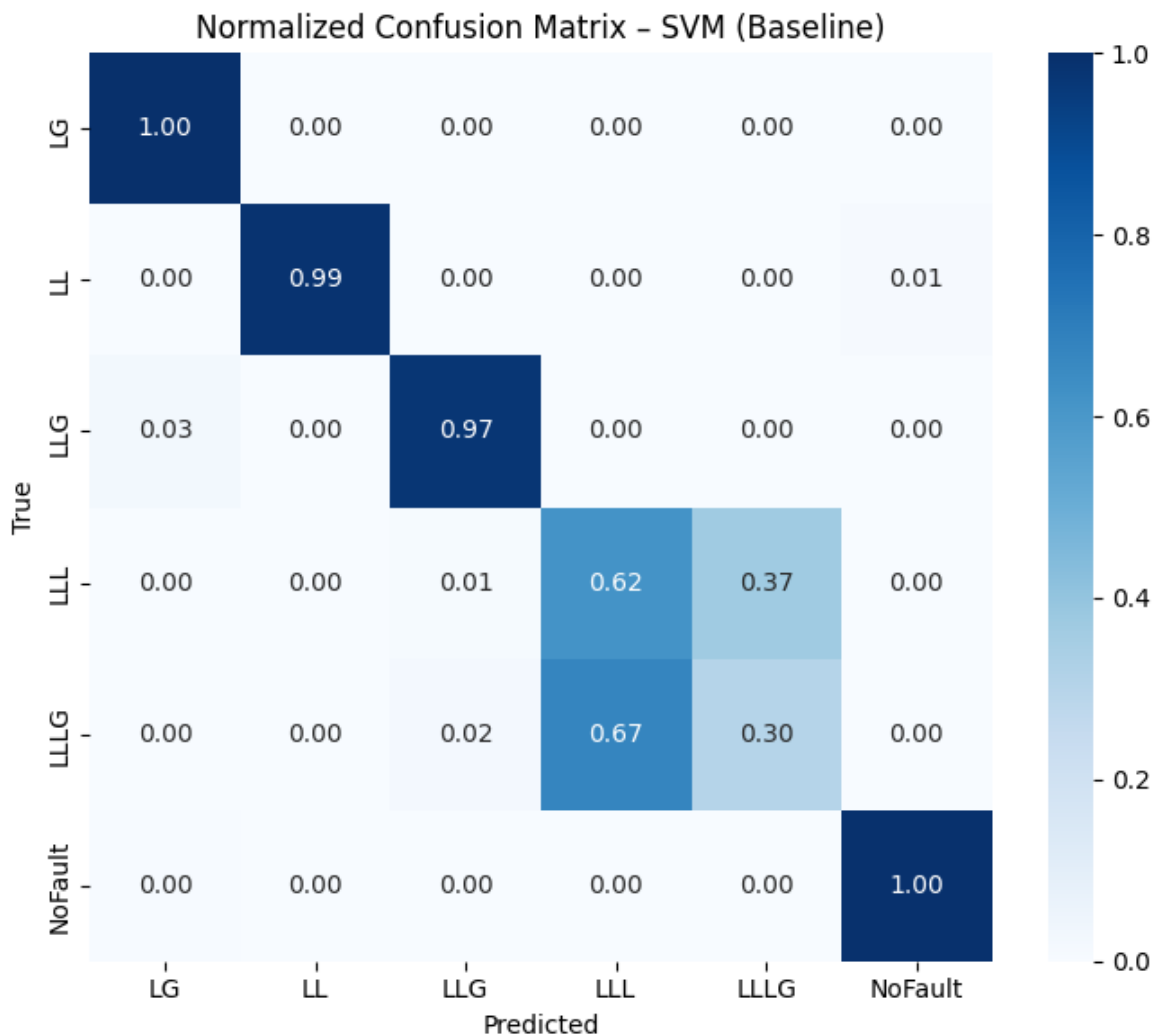


Hình 3.16: Confusion Matrix - Optimized

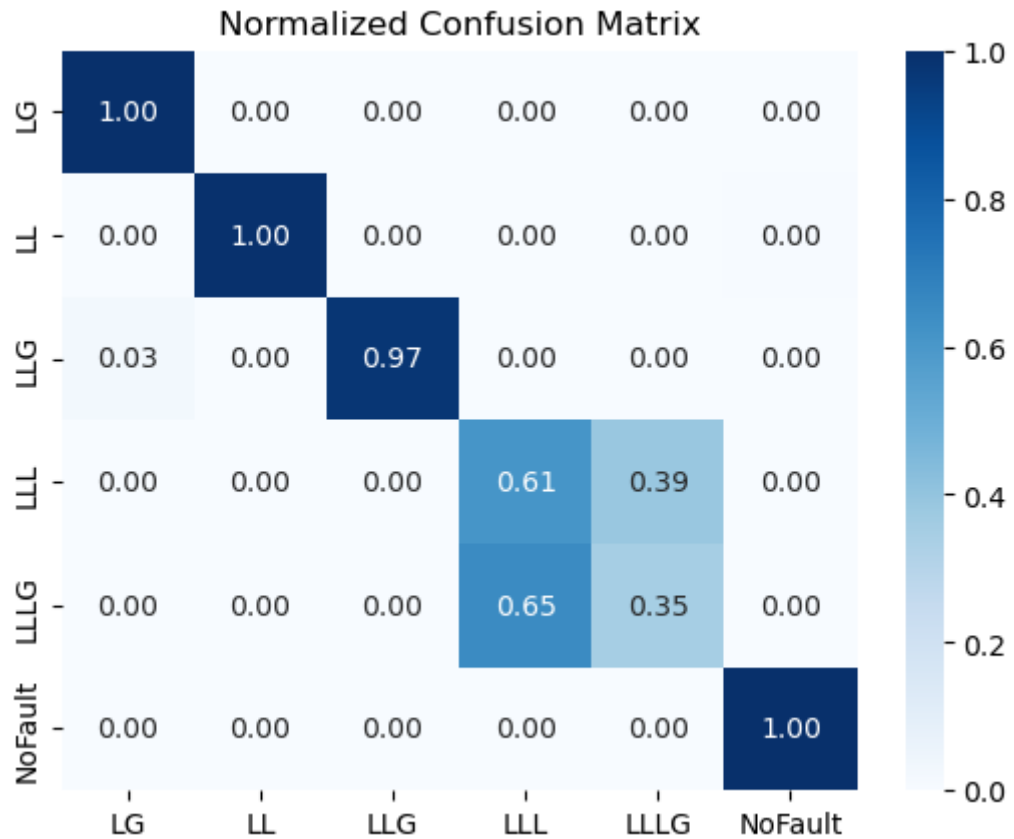
Dựa trên Confusion Matrix, mô hình SVM thể hiện khả năng phân loại rất tốt đối với các lớp lỗi đơn giản. Các lớp LG, LL và LLG đều đạt độ chính xác gần như tuyệt đối, lần lượt là 226/226, 200/201 và 221/227 mẫu được phân loại đúng. Điều này cho thấy mô hình học rất tốt các dạng lỗi một pha và hai pha cơ bản, nơi đặc trưng tín hiệu dòng - áp có sự khác biệt rõ rệt.

Đối với hai lớp lỗi phức tạp hơn là LLL và LLLG, mô hình vẫn đạt kết quả tốt nhưng có hiện tượng nhầm lẫn qua lại. Cụ thể, lớp LLL có 134 mẫu được dự đoán đúng và 85 mẫu bị dự đoán nhầm sang LLLG. Tương tự, lớp LLLG có 148 mẫu được phân loại chính xác và 79 mẫu bị nhầm sang LLL. Đây là điều dễ hiểu vì hai loại lỗi này đều là lỗi nhiều pha nghiêm trọng, dẫn đến tín hiệu dòng và áp ở ba pha có hình dạng rất giống nhau. Do đó, ranh giới giữa hai lớp trở nên khó phân tách hơn đối với mô hình.

Cuối cùng, lớp NoFault được mô hình phân loại chính xác hoàn toàn với 473/473 mẫu đúng. Điều này cho thấy mô hình không đưa ra cảnh báo sai trong trường hợp hệ thống hoạt động bình thường, một đặc điểm rất quan trọng đối với ứng dụng thực tế.



Hình 3.17: Normalized Confusion Matrix - Baseline



Hình 3.18: Normalized Confusion Matrix - Optimized

Normalized Confusion Matrix cho thấy tỉ lệ dự đoán đúng theo từng lớp. Các lớp LG, LL và LLG đạt tỉ lệ chính xác gần như tuyệt đối, lần lượt là 1.00, 1.00 và 0.97. Điều này khẳng định mô hình SVM xử lý rất tốt các lỗi có dấu hiệu rõ ràng và biên dạng tín hiệu khác biệt tương đối mạnh.

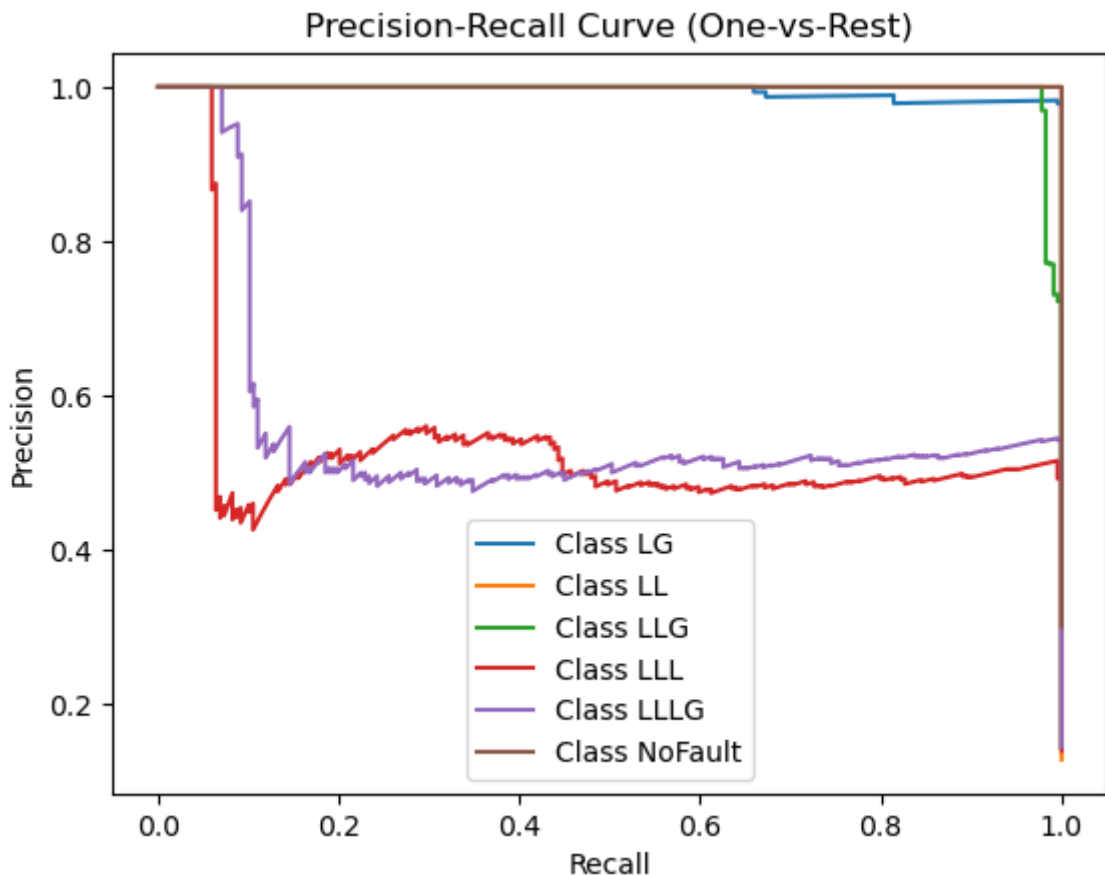
Trong khi đó, các lớp lỗi nhiều pha phức tạp như LLL và LLLG có độ chính xác lần lượt là 0.61 và 0.65. Hai lớp này vẫn bị nhầm lẫn qua lại với tỉ lệ đáng kể.

Nhìn chung, mô hình có xu hướng dự đoán nhầm LLL sang LLLG và ngược lại, phù hợp với nhận định rằng đặc trưng của các lỗi nhiều pha này rất gần nhau.

Lớp NoFault tiếp tục đạt tỉ lệ chính xác tuyệt đối (1.00), chứng minh mô hình không gây ra cảnh báo giả (false alarm).

Từ hai biểu đồ trên, có thể kết luận rằng mô hình SVM hoạt động rất hiệu quả đối với các lớp lỗi đơn pha và hai pha cơ bản. Đối với các lỗi nhiều pha phức tạp, mô

hình vẫn đạt hiệu suất tốt nhưng tồn tại mức độ nhầm lẫn nhất định, chủ yếu do dữ liệu hai lớp này có đặc trưng tín hiệu tương đồng. Bên cạnh đó, khả năng phân biệt trạng thái bình thường và trạng thái có lỗi là rất tốt, đảm bảo độ tin cậy cho hệ thống cảnh báo trong thực tế.



Hình 3.19: Precision-Recall Curve

Biểu đồ Precision-Recall Curve theo phương pháp One-vs-Rest cho thấy sự khác biệt khá rõ ràng giữa các lớp lỗi trong dữ liệu.

Đối với các lớp LG, LL và LLG, đường cong Precision-Recall duy trì ở mức rất cao, gần tiệm cận 1.0 trong hầu hết dải của recall. Điều này cho thấy mô hình SVM có khả năng nhận diện chính xác các lỗi đơn pha và hai pha đơn giản, đồng thời giữ được độ chính xác cao (precision) ngay cả khi recall tăng lên. Đây là đặc điểm phù hợp với các loại lỗi có tín hiệu đặc trưng rõ ràng và ít gây nhầm lẫn.

Đối với hai lớp lỗi phức tạp hơn là LLL và LLLG, đường cong Precision-Recall có dạng thấp hơn và dao động mạnh. Precision và recall thường xuyên nằm trong khoảng từ 0.45 đến 0.65, phản ánh khó khăn của mô hình trong việc phân tách hai loại lỗi nhiều pha này. Các tín hiệu dòng - áp của hai lớp LLL và LLLG có xu hướng tương đồng, dẫn đến mô hình bị nhầm lẫn và không thể duy trì mức precision cao khi recall tăng. Tuy nhiên, mô hình vẫn đạt mức hiệu suất chấp nhận được đối với các lớp này.

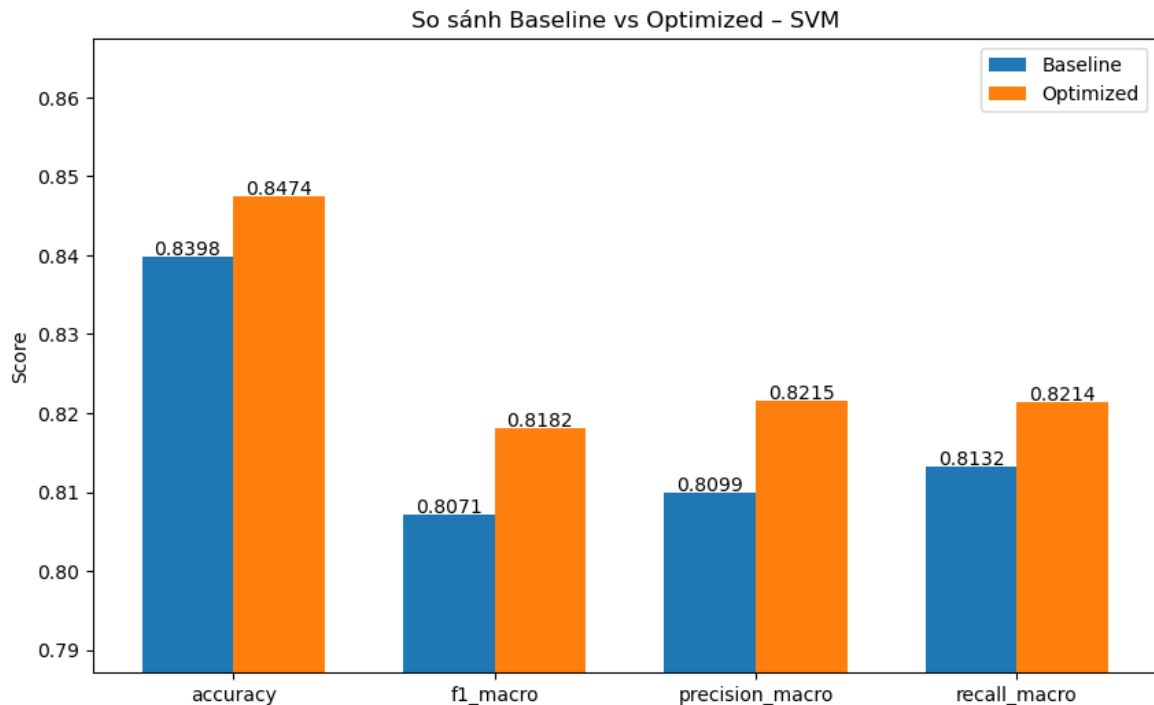
Lớp NoFault đạt đường cong gần như hoàn hảo, với precision và recall đều tiệm cận 1.0. Điều này khẳng định mô hình gần như không bao giờ báo sai lỗi khi hệ thống ở trạng thái bình thường, một yếu tố quan trọng trong các ứng dụng giám sát thực tế nhằm tránh cảnh báo giả.

	la	lb	lc	Va	Vb	Vc	True	Pred	Correct
0	706.623150	-816.002727	-22.002097	-0.006011	-0.101105	0.107116	LLG	LLG	True
1	-72.384673	73.398829	-4.486268	-0.199681	-0.385564	0.585245	NoFault	NoFault	True
2	-53.047387	96.915817	-47.238408	-0.529601	0.002669	0.526933	NoFault	NoFault	True
3	-120.703514	-696.736009	819.443759	-0.039884	0.007568	0.032316	LLL	LLL	False
4	884.344352	-488.161231	-394.040450	0.017541	-0.042240	0.024699	LLL	LLL	True
5	-571.588545	-304.606170	876.357673	-0.031213	0.026305	0.004907	LLL	LLL	True
6	514.531358	-885.803871	-37.435968	-0.018673	0.015927	0.002746	LLG	LLG	True
7	-52.785519	14.864509	34.947101	0.314129	-0.586814	0.272685	NoFault	NoFault	True
8	-57.027330	-87.429605	146.790454	-0.498221	-0.041539	0.539760	LL	LL	True
9	48.984845	31.671516	-83.351304	-0.149423	0.600904	-0.451481	NoFault	NoFault	True

Hình 3.20: Minh họa dự đoán trên một số mẫu kiểm tra

Bảng này trình bày một số mẫu dự đoán của mô hình SVM trên tập dữ liệu, bao gồm các đặc trưng đầu vào [la , lb , lc , Va , Vb , Vc], nhãn thực tế (*True*), nhãn dự đoán (*Pred*) và kết quả đúng/sai (*Correct*). Có thể thấy mô hình SVM dự đoán chính xác phần lớn các mẫu, đặc biệt với các trạng thái *No Fault* và *LLG*. Tuy nhiên, vẫn tồn tại một số trường hợp nhầm lẫn giữa các loại lỗi có đặc trưng tương tự như *LLL* và *LLL*G, cho thấy sự chồng lấn nhất định giữa các lớp trong không gian đặc trưng.

Đánh giá: So sánh đánh giá giữa 2 mô hình Baseline và Optimized (Tối ưu bằng GridSearchCV):



Hình 3.21: Baseline vs Optimized

- **Accuracy:**

Bảng 3.6: Accuracy

Mô hình	Accuracy
Baseline	0.8398
Optimized	0.8474

Accuracy tăng nhẹ từ **0.8398** lên **0.8474**.

- Điều này cho thấy việc tối ưu tham số đã giúp mô hình phân biệt các loại lỗi tốt hơn một chút so với mô hình baseline.

- **F1 Macro:**

Bảng 3.7: F1 Macro

Mô hình	F1 Macro
Baseline	0.8071
Optimized	0.8182

F1 tăng từ **0.8071** lên **0.8182**.

- Đây là cải thiện đáng kể nhất, phản ánh mô hình đã cân bằng tốt hơn giữa precision và recall trên tất cả các lớp lỗi.
- Điều này quan trọng vì dữ liệu có nhiều lớp lỗi với độ khó khác nhau (LG, LL, LLG, LLL...).

- **Precision Macro:**

Bảng 3.8: Precision Macro

Mô hình	Precision Macro
Baseline	0.8099
Optimized	0.8215

Precision tăng từ **0.8099** lên **0.8215**.

- Mô hình tối ưu dự đoán **ít nhầm lẫn hơn**, tức là giảm false positive.
- Đây là dấu hiệu mô hình phân loại ranh giới giữa các lỗi một cách “sạch” hơn.

- **Recall Macro:**

Bảng 3.9: Recall Macro

Mô hình	Recall Macro
Baseline	0.8132

Optimized	0.8214
-----------	--------

Recall tăng từ **0.8132** lên **0.8214**.

- Đây là cải thiện quan trọng vì recall thể hiện khả năng **bắt đúng lỗi**, đặc biệt trong các lớp khó.
- Mô hình tối ưu đã giảm số lượng lỗi bị bỏ sót (false negative).

Kết luận:

- GridSearchCV **giúp cải thiện tất cả các chỉ số** của mô hình.
- **Accuracy tăng nhẹ**, cho thấy mô hình tổng thể tốt hơn.
- **Precision và Recall đều tăng**, chứng minh ranh giới phân lớp trở nên chính xác hơn.
- **F1 Macro tăng mạnh nhất**, nghĩa là mô hình được tối ưu hóa hoạt động cân bằng hơn trên toàn bộ 6 lớp lỗi.
- Việc tối ưu SVM với RBF kernel là **có ý nghĩa**, đặc biệt khi dữ liệu dòng - áp mang tính phi tuyến mạnh.
- **Mô hình SVM sau tối ưu rõ ràng tốt hơn mô hình baseline, ổn định hơn và phù hợp để triển khai cho bài toán phân loại lỗi hệ thống điện.**

3.4 kNN

Trong phần này, nhóm tiến hành xây dựng và đánh giá mô hình k-Nearest Neighbors (kNN). Khác với Random Forest hay SVM là các mô hình dựa trên việc phân chia không gian hoặc tối ưu hóa hàm mất mát, kNN là thuật toán dựa trên khoảng cách (Distance-based), dự đoán nhãn của dữ liệu với nguồn dữ liệu được sử dụng như ở hai mô hình trên.

Mục tiêu của phần này gồm:

- Huấn luyện mô hình kNN cơ bản (Baseline) và đánh giá hiệu năng.
- Áp dụng kỹ thuật chuẩn hóa dữ liệu (Standardization)
- Tối ưu hóa các siêu tham số (k , metric, weights) bằng GridSearchCV.
- So sánh, phân tích sự cải thiện hiệu suất sau khi tối ưu.

Nhóm đã sử dụng StandardScaler để đưa toàn bộ 6 đặc trưng đầu vào (I_a , I_b , I_c , V_a , V_b , V_c) về dạng phân phối chuẩn ($\mu = 0$, $\sigma = 1$) trước khi đưa vào huấn luyện.

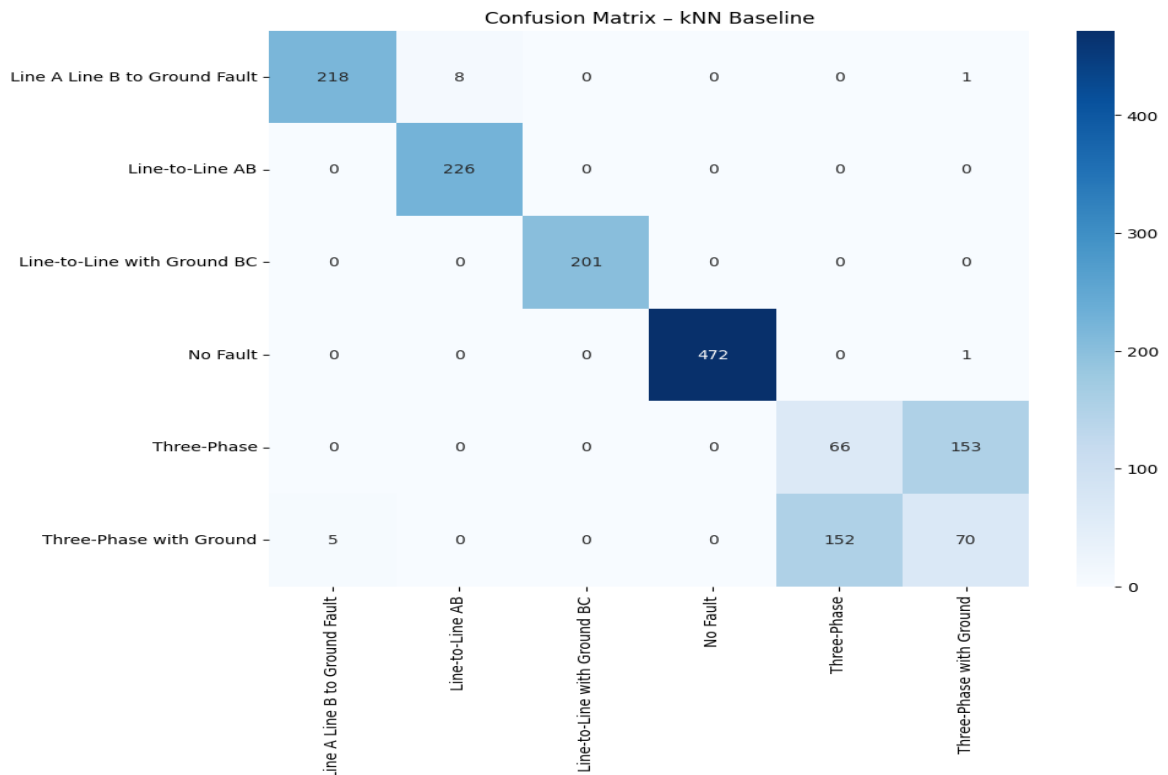
Thiết lập mô hình Baseline:

- Số lượng láng giềng (k): 5
- Trọng số (weights): uniform (Mọi láng giềng có quyền bầu chọn ngang nhau).
- Độ đo khoảng cách: Minkowski ($p=2$, tương đương khoảng cách Euclid).

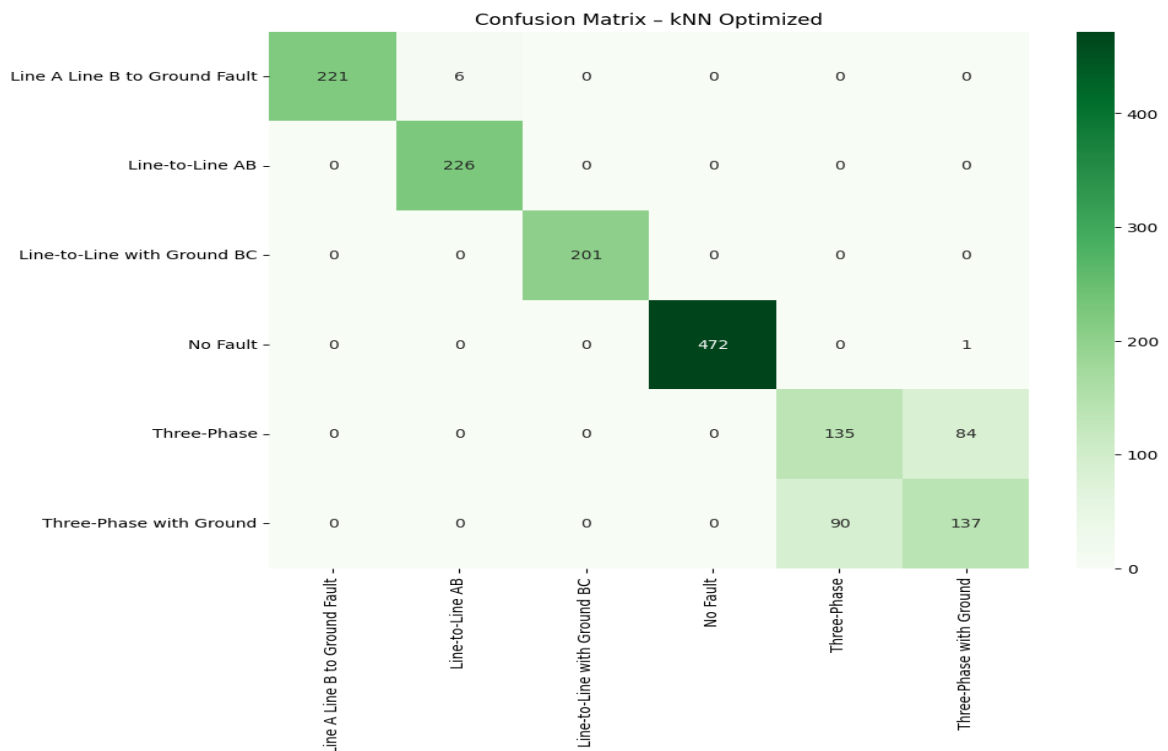
Để cải thiện hiệu suất, nhóm đã sử dụng kỹ thuật GridSearchCV để tìm kiếm bộ tham số tối ưu trong không gian tìm kiếm sau:

- **n_neighbors**: [3, 5, 7, 9, 11, 15]
- **weights**: ['uniform', 'distance']
- **metric**: ['euclidean', 'manhattan']

Kết quả: So sánh đánh giá giữa 2 mô hình Baseline và Optimized (Tối ưu bằng GridSearchCV):



Hình 3.22: Confusion Matrix - Baseline

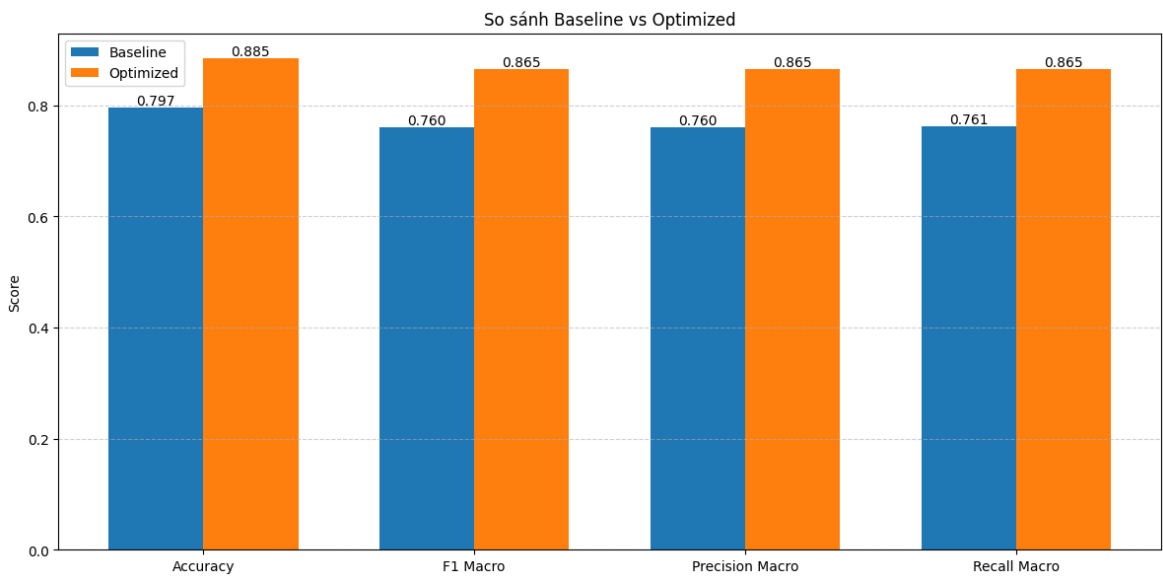


Hình 3.23: Confusion Matrix - Optimized

Nhận xét:

- Cả hai mô hình đều xử lý hoàn hảo các sự cố đơn giản và trạng thái bình thường (No Fault, Line-to-Line...), đạt độ chính xác gần như tuyệt đối.
- Trong khi Baseline thất bại nặng nề khi phân biệt giữa Three-Phase và Three-Phase with Ground, mô hình Optimized đã cải thiện vượt bậc, nâng số lượng dự đoán đúng lên gấp đôi ở cả hai lớp này.

Kết luận: Quá trình tối ưu hóa đã giải quyết thành công điểm yếu chí mạng là khả năng phân tách các lớp dữ liệu chồng lấn (nhóm 3 pha). Đây là một bản nâng cấp đáng giá, mang lại độ tin cậy và hiệu suất tổng thể cao hơn hẳn.



Hình 3.24: Baseline vs Optimized

● Accuracy:

Bảng 3.10: Accuracy

Mô hình	Accuracy
Baseline	0.797
Optimized	0.885

Accuracy tăng mạnh từ **0.797** lên **0.885**

- Mô hình sau khi tối ưu đã cải thiện độ chính xác tổng thể thêm khoảng 8.8%. Điều này cho thấy khả năng dự đoán đúng của hệ thống trên toàn bộ tập dữ liệu đã trở nên tin cậy hơn rất nhiều.

● F1 Macro:

Bảng 3.11: F1 Macro

Mô hình	F1 Macro
---------	----------

Baseline	0.760
Optimized	0.865

F1 tăng vượt bậc từ **0.760** lên **0.865**

- Đây là chỉ số quan trọng nhất trong trường hợp này (vì nó tính trung bình không trọng số cho các lớp, bao gồm cả các lớp khó như sự cố 3 pha). Mức tăng 10.5% chứng tỏ mô hình Optimized đã cân bằng rất tốt giữa độ chính xác (Precision) và độ nhạy (Recall), không còn bị thiên lệch về các lớp dễ đoán như mô hình Baseline.

- **Precision Macro:**

Bảng 3.12: Precision Macro

Mô hình	Precision Macro
Baseline	0.760
Optimized	0.865

Precision tăng từ **0.760** lên **0.865**

- Chỉ số này cho biết mức độ tin cậy khi mô hình đưa ra dự đoán. Việc tăng hơn 10% đồng nghĩa với việc tỉ lệ báo sai (False Positive) đã giảm đáng kể. Khi mô hình Optimized báo lỗi loại nào, xác suất đó là sự thật cao hơn hẳn so với Baseline.

- **Recall Macro:**

Bảng 3.13: Recall Macro

Mô hình	Recall Macro
Baseline	0.761

Optimized	0.865
-----------	-------

Recall Tăng từ **0.761** lên **0.865**

- Chỉ số này phản ánh khả năng "bắt" đúng sự cố, không bỏ sót. Mức tăng tương đồng với Precision cho thấy mô hình Optimized đã hạn chế tối đa việc bỏ sót các trường hợp lỗi (False Negative), đặc biệt là ở các lớp khó phân biệt.

Kết luận: Mô hình kNN Optimized áp đảo hoàn toàn về mọi mặt. Sự cải thiện đồng bộ các chỉ số khẳng định quá trình tối ưu hóa đã thành công rực rỡ, tạo ra một mô hình ổn định, chính xác hơn Baseline.

	Ia	Ib	Ic	Va	Vb	Vc	True	Pred	Correct
609	796.799493	67.532319	-19.335178	-0.212190	0.048315	0.163875	Line-to-Line AB	Line-to-Line AB	True
5259	-527.987206	-349.766659	877.751602	-0.041854	0.026729	0.015125	Three-Phase with Ground	Three-Phase with Ground	True
4182	-295.659601	-572.187538	869.853881	-0.041958	0.015863	0.026095	Three-Phase	Three-Phase	True
7584	39.059549	-29.523222	-12.480942	0.481916	0.074532	-0.556448	No Fault	No Fault	True
1738	789.617302	-735.007870	-11.600631	0.001952	-0.167279	0.165328	Line A Line B to Ground Fault	Line A Line B to Ground Fault	True
2501	-69.742998	330.543128	-258.341602	-0.372273	-0.039761	0.412034	Line-to-Line with Ground BC	Line-to-Line with Ground BC	True
873	-737.981150	-18.264391	-28.507846	-0.155256	0.484075	-0.328819	Line-to-Line AB	Line-to-Line AB	True
6946	-23.064731	93.675658	-73.741738	-0.619018	0.312023	0.306995	No Fault	No Fault	True
1117	265.583142	-29.522285	48.094283	0.342631	-0.605280	0.262649	Line-to-Line AB	Line-to-Line AB	True
7607	51.154484	-18.891532	-35.201065	0.326186	0.284112	-0.610298	No Fault	No Fault	True

Hình 3.25: Minh họa dự đoán của kNN trên tập kiểm tra

3.5 MLP

3.5.1 MLP (sklearn - MLPClassifier)

Trong phần này, nhóm tiến hành xây dựng và đánh giá mô hình Multilayer Perceptron (MLP) cho bài toán phân loại lỗi hệ thống điện. MLP là một mô hình mạng nơ-ron truyền thẳng có khả năng học các mối quan hệ phi tuyến phức tạp giữa các đặc trưng đầu vào, do đó phù hợp với dữ liệu có tính phi tuyến cao như trong bài toán phân loại lỗi điện.

Mục tiêu của phần thực nghiệm với MLP bao gồm:

- Huấn luyện mô hình MLP cho bài toán phân loại lỗi.
- Đánh giá hiệu năng mô hình bằng các chỉ số Accuracy, Precision, Recall và F1-score.
- So sánh hiệu quả giữa mô hình MLP baseline và mô hình MLP sau khi tối ưu siêu tham số.

Tiền xử lý dữ liệu:

Tương tự như các mô hình SVM và kNN, mô hình MLP nhạy cảm với thang đo của dữ liệu đầu vào. Do đó, trước khi huấn luyện mô hình, các đặc trưng được chuẩn hóa bằng phương pháp StandardScaler nhằm đưa dữ liệu về cùng thang đo, giúp quá trình huấn luyện ổn định và hội tụ nhanh hơn.

Sau khi chuẩn hóa, tập dữ liệu được chia thành hai tập huấn luyện và kiểm thử theo tỉ lệ 80/20, đảm bảo tính nhất quán với các mô hình đã được trình bày trước đó.

- Thiết lập mô hình MLP Baseline:

Mô hình MLP baseline được xây dựng bằng lớp MLPClassifier trong thư viện scikit-learn. Kiến trúc mạng gồm hai lớp ẩn với số lượng nơ-ron lần lượt là 64 và 32, sử dụng hàm kích hoạt ReLU và thuật toán tối ưu Adam. Một số tham số quan trọng của mô hình baseline được thiết lập như sau:

- Kiến trúc mạng: (64, 32)
- Hàm kích hoạt: ReLU
- Optimizer: Adam
- Hệ số regularization $\alpha = 10^{-4}$
- Learning rate ban đầu: 10^{-3}
- Số vòng lặp tối đa: 300
- Early stopping để tránh hiện tượng overfitting

Mô hình baseline được huấn luyện trên tập huấn luyện và sau đó được sử dụng để dự đoán nhãn cũng như xác suất lớp trên tập kiểm thử nhằm phục vụ cho việc đánh giá hiệu năng.

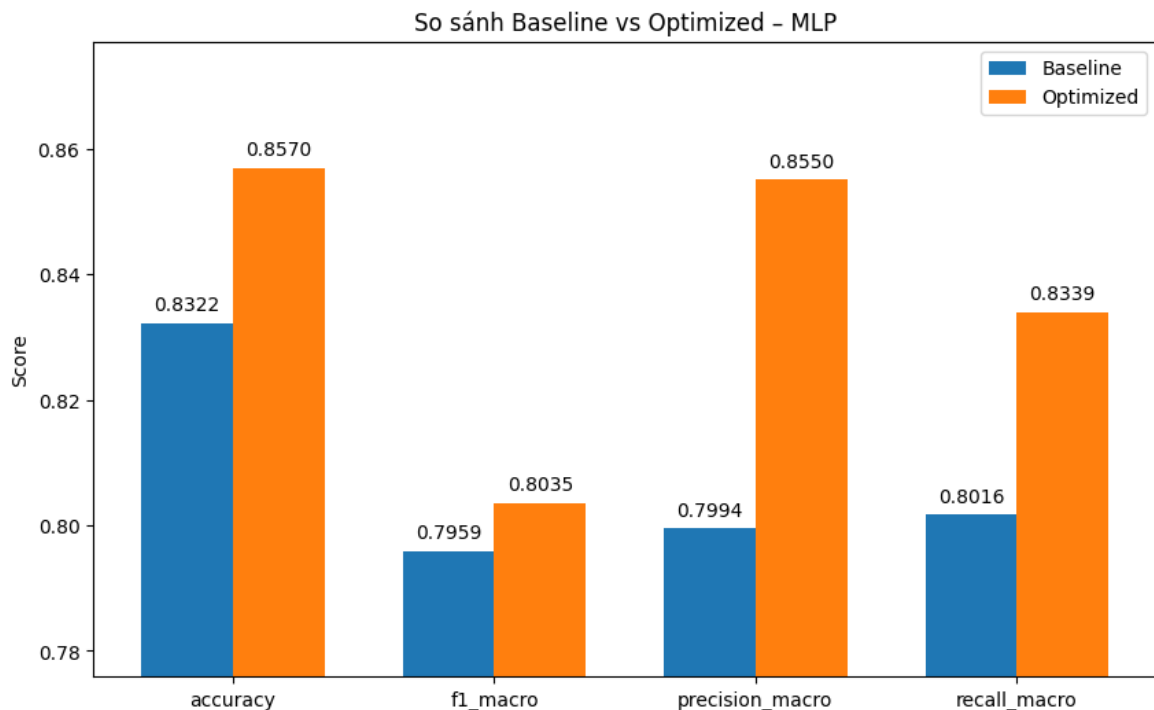
● Tối ưu mô hình MLP:

Để cải thiện hiệu năng của mô hình, nhóm sử dụng phương pháp GridSearchCV nhằm tìm kiếm bộ siêu tham số tối ưu cho MLP. Các siêu tham số được xem xét bao gồm:

- Số lượng và kích thước các lớp ẩn.
- Hệ số regularization α .
- Learning rate ban đầu.

Quá trình tìm kiếm được thực hiện với phương pháp cross-validation 3-fold và sử dụng chỉ số F1-score (macro) làm tiêu chí đánh giá. Sau khi quá trình Grid Search hoàn tất, mô hình MLP với bộ siêu tham số tốt nhất được lựa chọn làm mô hình tối ưu và sử dụng để đánh giá trên tập kiểm thử.

Đánh giá: So sánh đánh giá giữa 2 mô hình MLP Baseline và Optimized (Tối ưu bằng GridSearchCV):



Hình 3.26: Baseline vs Optimized

● **Accuracy:**

Bảng 3.14: Accuracy

Mô hình	Accuracy
Baseline	0.8322
Optimized	0.8570

Accuracy tăng từ **0.8322** lên **0.8570**.

- Điều này cho thấy việc tối ưu siêu tham số đã giúp mô hình MLP cải thiện khả năng phân biệt tổng thể giữa các loại lỗi so với mô hình baseline.

- **F1 Macro:**

Bảng 3.15: F1 Macro

Mô hình	F1 Macro
Baseline	0.7959
Optimized	0.8035

F1 tăng từ **0.7959** lên **0.8035**.

- Sự cải thiện này cho thấy mô hình MLP sau tối ưu đã cân bằng tốt hơn giữa precision và recall trên các lớp lỗi.
- Đây là chỉ số quan trọng trong bài toán phân loại đa lớp, đặc biệt khi dữ liệu gồm nhiều loại lỗi với mức độ phân bố và độ khó khác nhau.

- **Precision Macro:**

Bảng 3.16: Precision Macro

Mô hình	Precision Macro
---------	-----------------

Baseline	0.7994
Optimized	0.8550

Precision Macro tăng rõ rệt từ **0.7994** lên **0.8550**.

- Điều này cho thấy mô hình MLP tối ưu dự đoán chính xác hơn, giảm số lượng dự đoán sai (false positive).
- Ranh giới phân lớp giữa các loại lỗi được học tốt hơn nhờ khả năng học phi tuyến của mạng MLP.

- **Recall Macro:**

Bảng 3.17: Recall Macro

Mô hình	Recall Macro
Baseline	0.8016
Optimized	0.8339

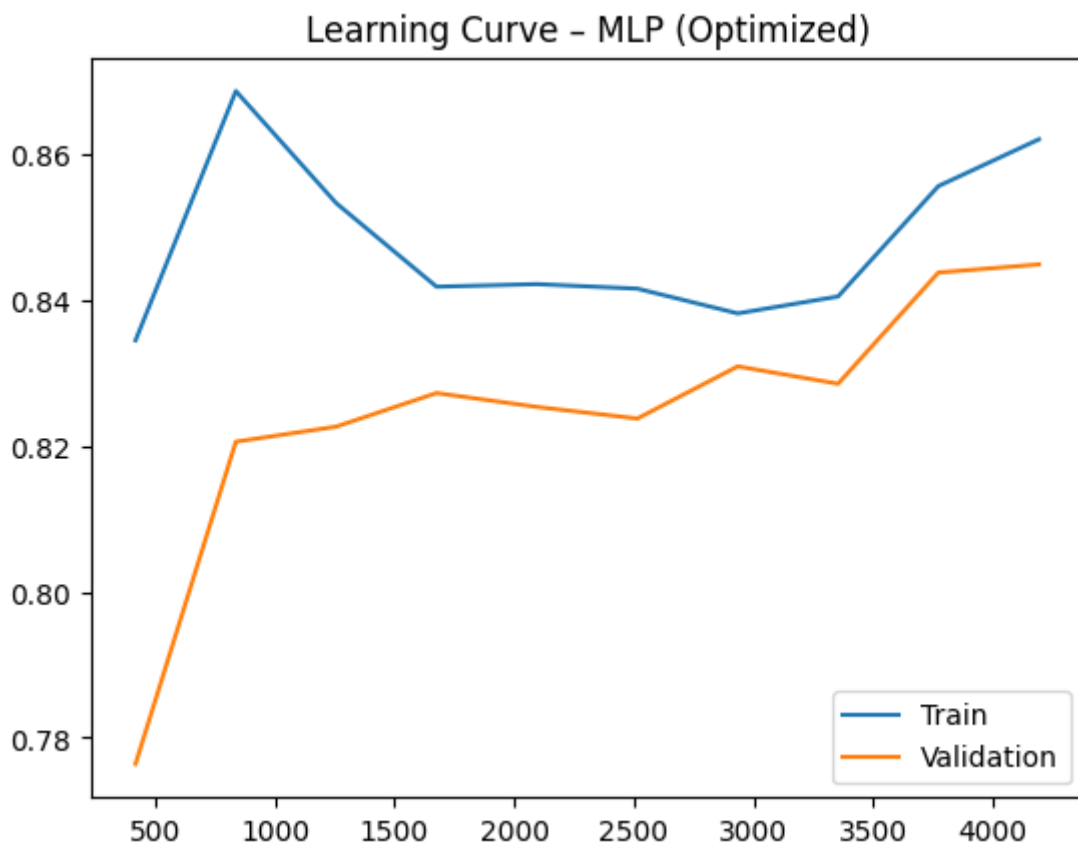
Recall tăng từ **0.8016** lên **0.8339**.

- Sự cải thiện này phản ánh khả năng bắt đúng lỗi của mô hình được nâng cao, đặc biệt với các lớp lỗi khó.
- Mô hình MLP sau tối ưu đã giảm số lượng lỗi bị bỏ sót (false negative).

Kết luận:

- Việc sử dụng GridSearchCV giúp cải thiện tất cả các chỉ số đánh giá của mô hình MLP.
- Accuracy tăng cho thấy hiệu năng tổng thể của mô hình được nâng cao.
- Precision và Recall đều tăng, chứng minh mô hình phân biệt ranh giới giữa các loại lỗi rõ ràng và chính xác hơn.

- F1 Macro được cải thiện, cho thấy mô hình hoạt động cân bằng hơn trên toàn bộ các lớp lỗi.
 - Nhờ khả năng học các mối quan hệ phi tuyến giữa các đặc trưng dòng điện và điện áp, MLP sau tối ưu thể hiện hiệu quả tốt và phù hợp cho bài toán phân loại lỗi hệ thống điện.
- **Mô hình MLP (sklearn - MLPClassifier) sau tối ưu rõ ràng tốt hơn mô hình baseline, ổn định hơn và phù hợp để triển khai cho bài toán phân loại lỗi hệ thống điện.**



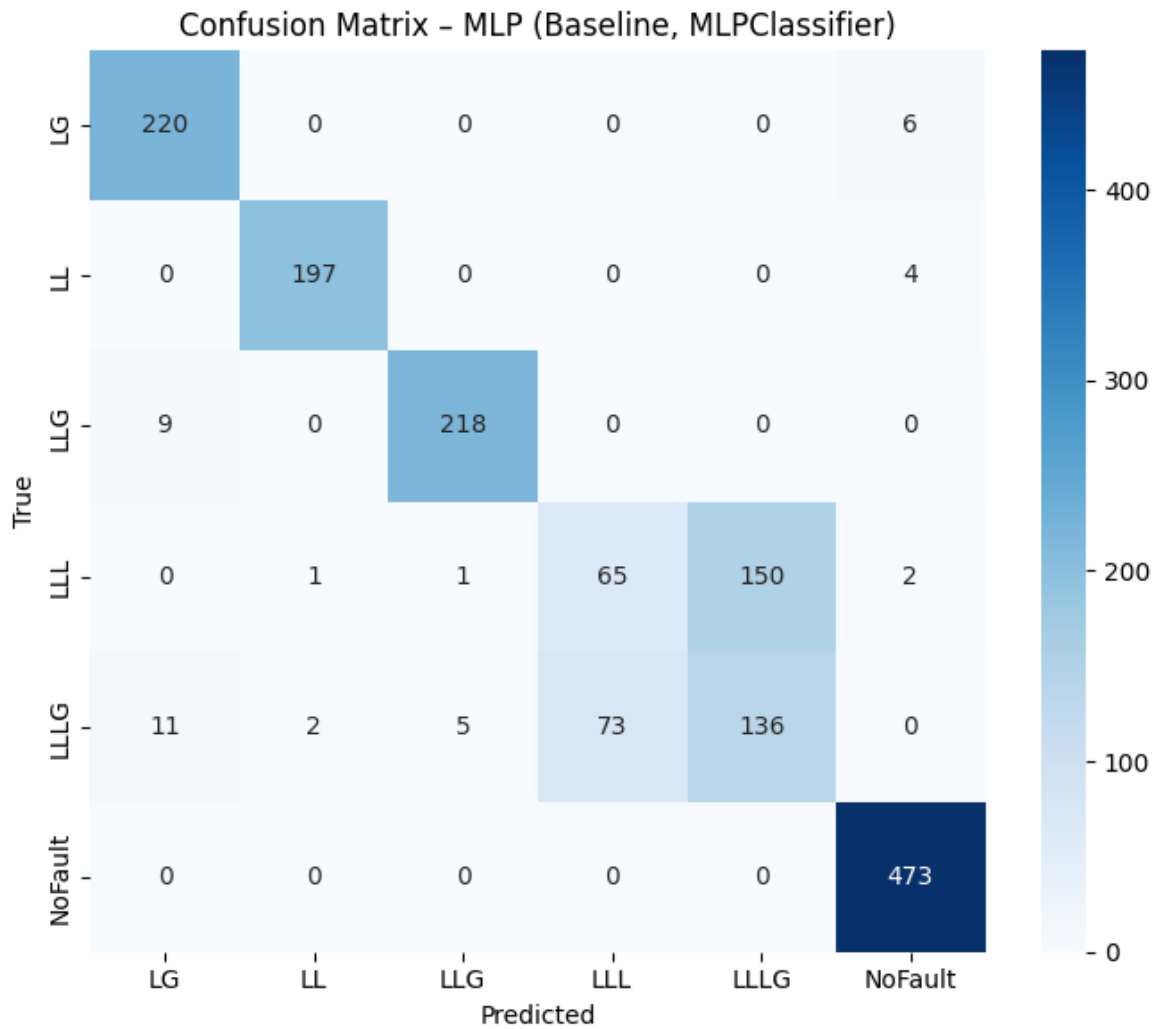
Hình 3.27: Learning Curve MLP (Optimized)

Hình này thể hiện learning curve của mô hình MLP sau khi tối ưu siêu tham số.

- Đường accuracy trên tập huấn luyện và tập validation đều tăng dần theo số lượng mẫu huấn luyện, cho thấy mô hình học được các đặc trưng quan trọng từ dữ liệu.
- Khoảng cách giữa hai đường train và validation không quá lớn, chứng tỏ mô hình không bị overfitting nghiêm trọng.
- Accuracy trên tập validation ổn định và cải thiện khi số lượng dữ liệu tăng, phản ánh khả năng tổng quát hóa tốt của mô hình MLP sau tối ưu.

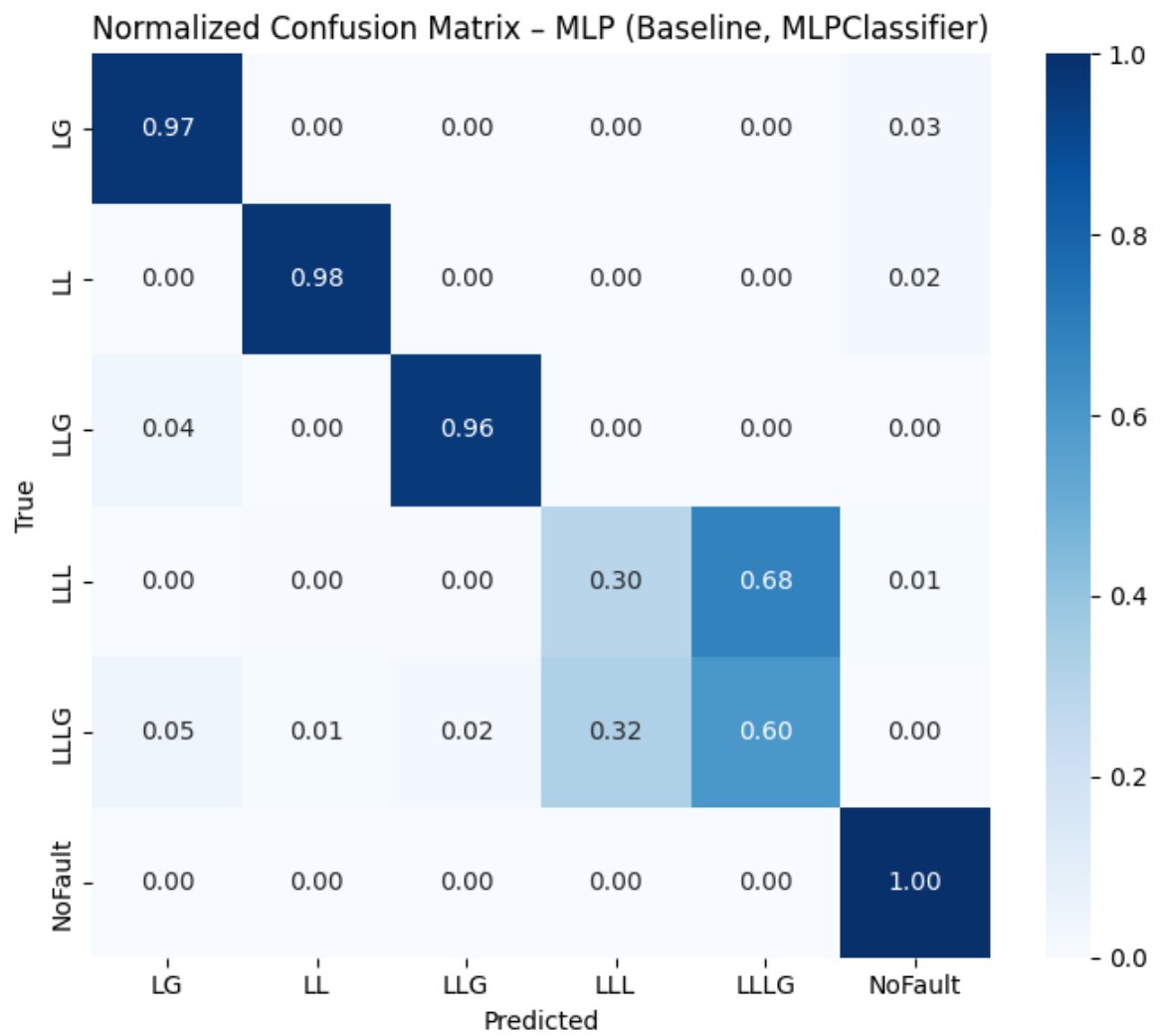
Nhận xét:

- Mô hình MLP sau tối ưu có xu hướng học ổn định khi dữ liệu huấn luyện tăng lên.
- Learning curve cho thấy mô hình phù hợp với tập dữ liệu hiện tại và có khả năng cải thiện thêm nếu bổ sung dữ liệu huấn luyện.

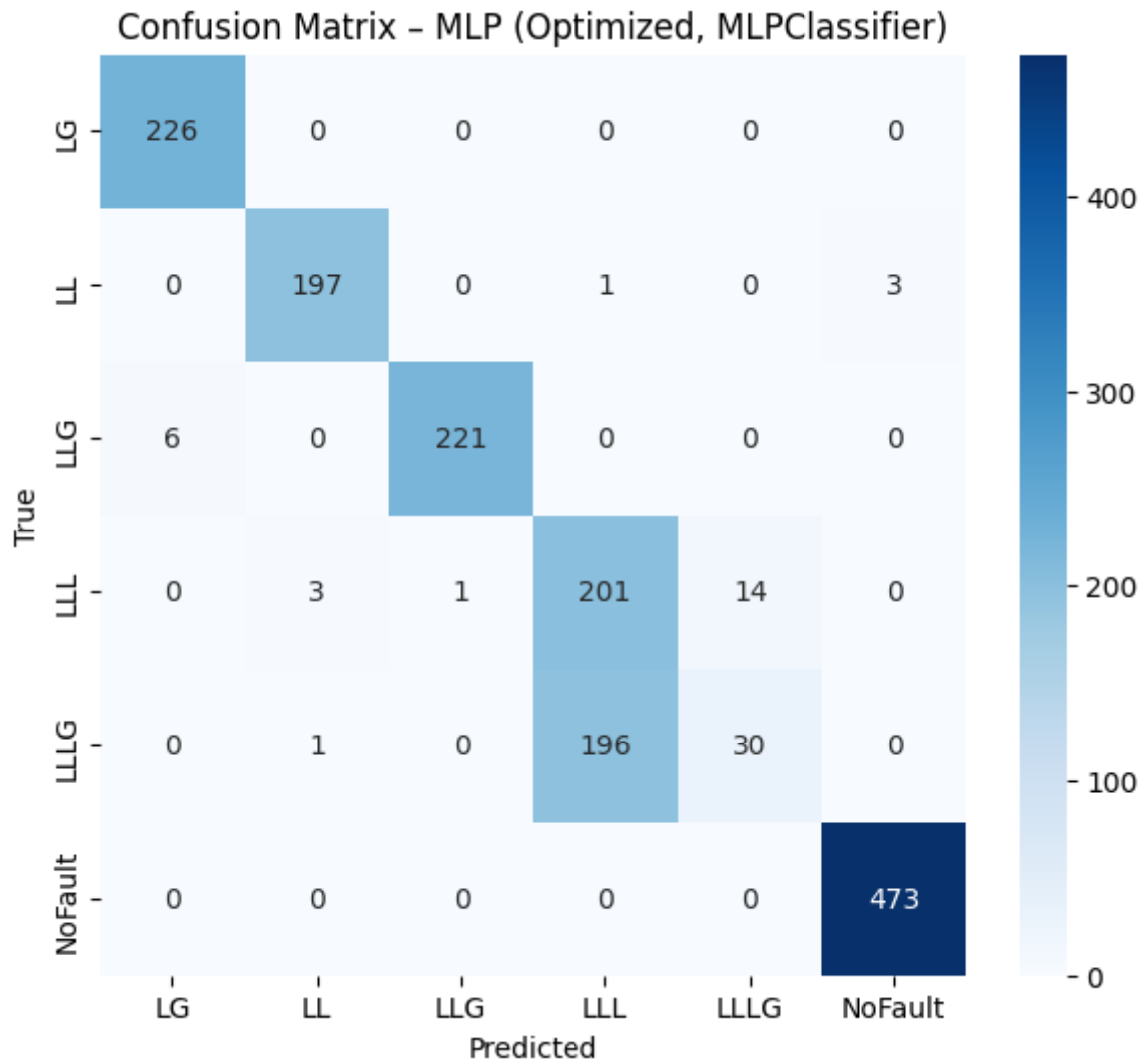


Hình 3.28: Confusion Matrix MLP (Baseline)

- Các lớp **NoFault**, **LG**, **LL**, **LLG** được phân loại rất chính xác với tỷ lệ đúng cao.
- Một số nhầm lẫn xảy ra giữa các lớp lỗi phức tạp như **LLL** và **LLLG**.
- Đây là các loại lỗi có đặc trưng dòng - áp gần nhau, gây khó khăn cho mô hình trong việc phân biệt.
- Tuy nhiên, mô hình vẫn giữ được hiệu suất tốt trên hầu hết các lớp.



Hình 3.29: Normalized Confusion Matrix MLP (Baseline)

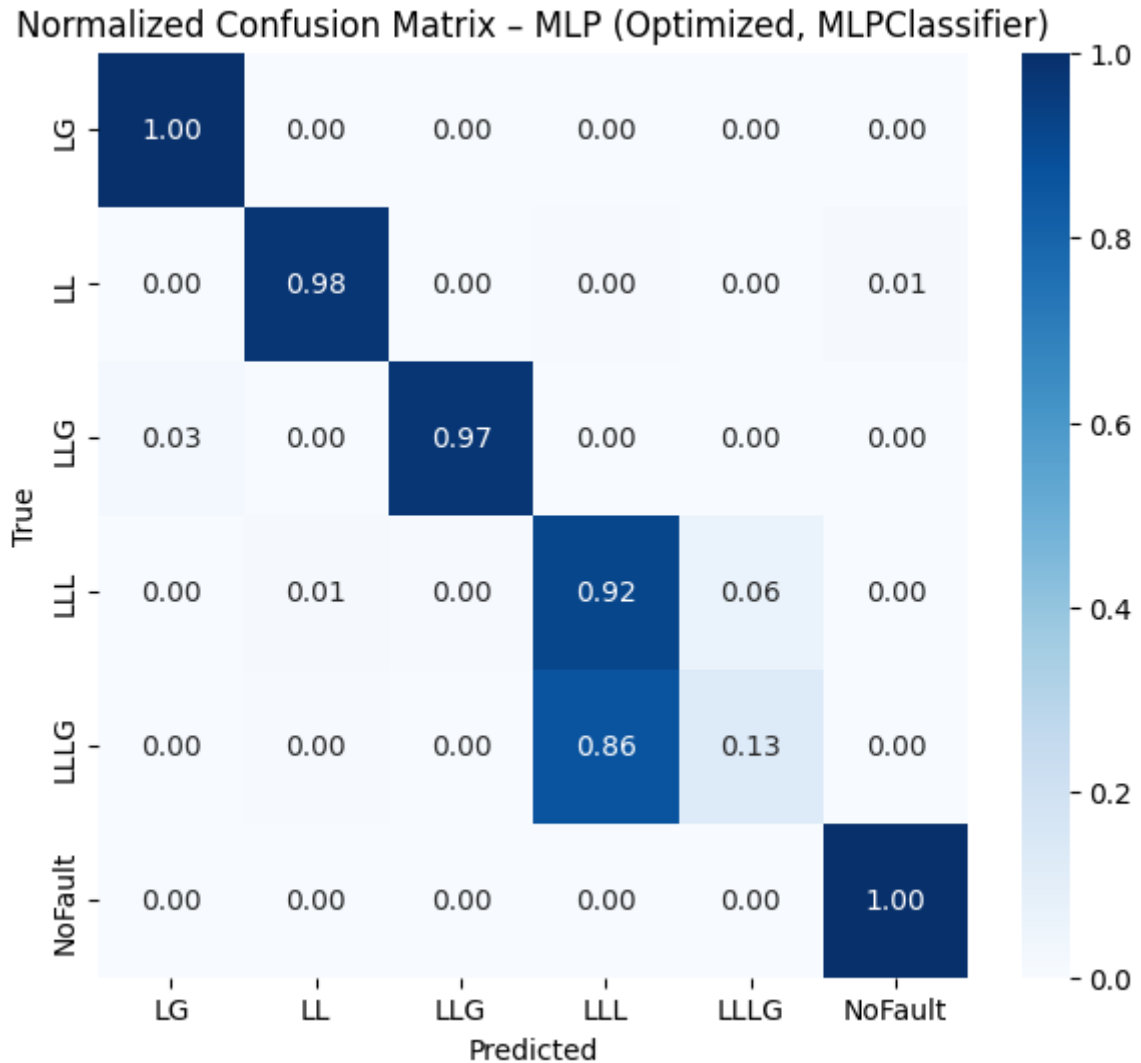


Hình 3.30: Confusion Matrix MLP (Optimized)

Hình minh họa Confusion Matrix của mô hình **MLP Optimized** cho thấy hiệu quả phân loại được cải thiện rõ rệt so với mô hình baseline.

- Các lớp **LG**, **LL**, **LLG** và **NoFault** được phân loại gần như hoàn hảo, với tỷ lệ dự đoán đúng rất cao ($\approx 97\% - 100$).
- Lớp **NoFault** đạt độ chính xác tuyệt đối, không xuất hiện trường hợp nhầm lẫn với các loại lỗi khác.
- Đối với lớp **LLL**, mô hình đạt khoảng **92%** độ chính xác, tuy nhiên vẫn còn một số mẫu bị nhầm sang lớp **LLLG**.
- Lớp **LLLG** là lớp khó nhất, với khoảng **13%** mẫu bị nhầm sang lớp **LLL**.

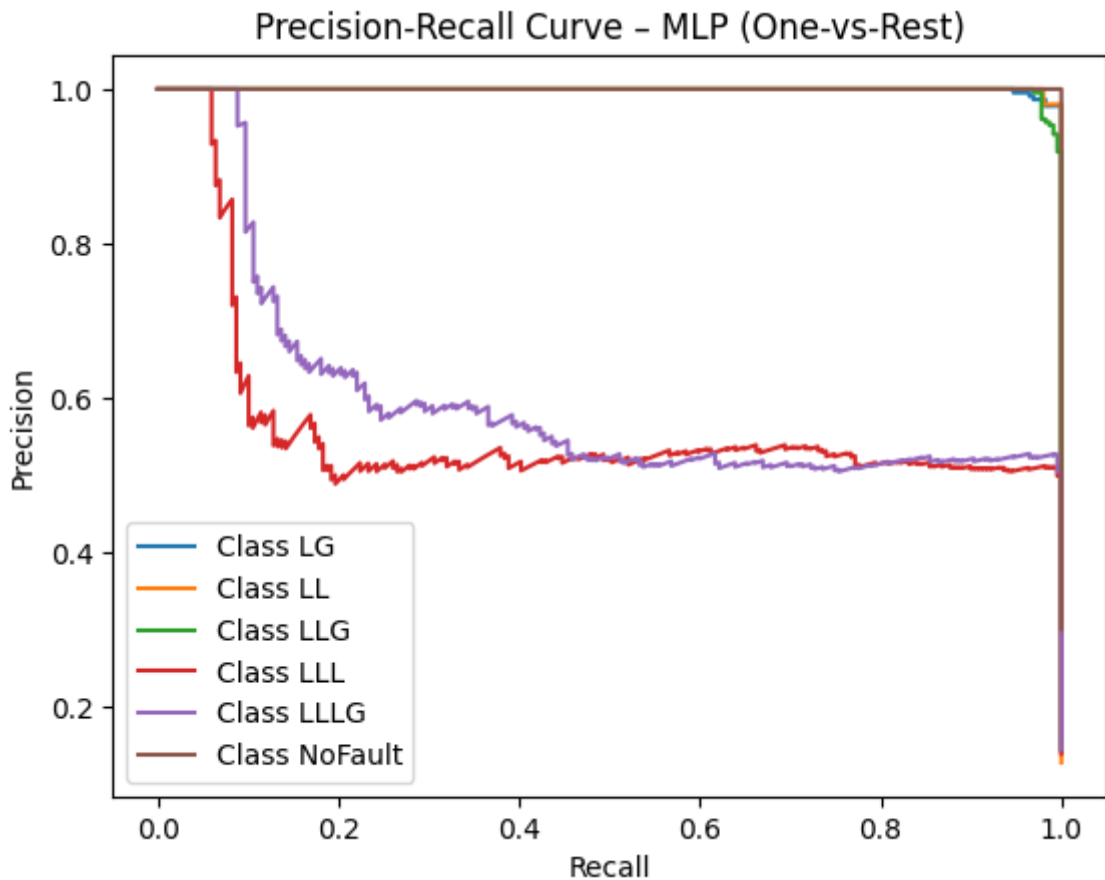
Điều này cho thấy mặc dù MLP đã học tốt các đặc trưng dòng - áp phi tuyến, nhưng các lỗi **LLL** và **LLLG** có đặc điểm khá tương đồng, dẫn đến sự chònh lẩn trong không gian đặc trưng.



Hình 3.31: Normalized Confusion Matrix MLP (Optimized)

Nhận xét:

- So với mô hình baseline, MLP Optimized giảm đáng kể mức độ nhầm lẫn giữa các lớp, đặc biệt là các lỗi đơn pha và không lỗi. Tuy vẫn còn nhầm lẫn ở các lỗi đa pha phức tạp, nhưng nhìn chung mô hình đã đạt được hiệu quả phân loại tốt và ổn định hơn.



Hình 3.32: Precision-Recall Curve MLP (Optimized)

- Lớp NoFault có đường cong gần như nằm sát phía trên, cho thấy mô hình nhận diện trạng thái không lỗi rất tốt với precision và recall cao.
- Các lớp lỗi đơn giản như LG, LL và LLG đạt hiệu suất khá ổn định, với precision duy trì ở mức cao khi recall tăng.
- Đối với các lớp lỗi phức tạp hơn như LLL và LLLG, precision giảm khi recall tăng, cho thấy sự đánh đổi giữa việc phát hiện đầy đủ lỗi và độ chính xác dự đoán.
- Điều này phản ánh thực tế rằng các lỗi đa pha có đặc trưng dòng - áp tương đối giống nhau, gây khó khăn cho mô hình trong việc phân biệt hoàn toàn chính xác.

	Ia	Ib	Ic	Va	Vb	Vc	True	Pred	Correct
0	868.209444	44.790687	9.113268	-0.002460	-0.291221	0.293682	LG	LG	True
1	14.747733	70.686068	-88.343534	-0.520027	0.558296	-0.038269	NoFault	NoFault	True
2	-60.703865	89.856812	-32.269649	-0.453210	-0.121340	0.574551	NoFault	NoFault	True
3	-452.430819	883.257681	-430.829125	0.022136	0.020220	-0.042356	LLLG	LLL	False
4	9.352000	-746.961689	740.042930	-0.286844	-0.008318	0.295162	LL	LL	True
5	-560.770260	-311.156819	871.924833	-0.041494	0.028249	0.013245	LLLG	LLL	False
6	-288.334449	868.063717	-577.612183	0.029011	0.012227	-0.041238	LLL	LLL	True
7	-553.399503	-317.550373	-50.019342	-0.041189	0.372035	-0.330846	LLG	LLG	True
8	-41.716369	-333.765344	377.727669	-0.504371	-0.036645	0.541017	LL	LL	True
9	-69.155730	47.734460	18.321297	0.027347	-0.524539	0.497192	NoFault	NoFault	True

Hình 3.33: Minh họa dự đoán của MLP trên tập kiểm tra

3.5.2 MLP (PyTorch)

- Tiền xử lý dữ liệu và chia tập

Nhóm sử dụng tệp classData.csv và đọc dữ liệu bằng pandas. Tập dữ liệu gồm 6 đặc trưng đầu vào: $X=[Ia, Ib, Ic, Va, Vb, Vc]$

và nhãn được tạo từ 4 cột G, C, B, A theo dạng nhị phân. Nhóm ánh xạ bộ nhãn (G,C,B,A) sang FaultType như sau:

- $(0,0,0,0) \rightarrow \text{NoFault}$
- $(1,0,0,1) \rightarrow \text{LG}$
- $(0,1,1,0) \rightarrow \text{LL}$
- $(1,0,1,1) \rightarrow \text{LLG}$
- $(0,1,1,1) \rightarrow \text{LLL}$
- $(1,1,1,1) \rightarrow \text{LLLG}$

Các dòng không khớp ánh xạ sẽ bị loại bỏ để tránh “nhãn rác” (unmapped) gây sai lệch trong huấn luyện.

Sau khi gán nhãn, nhóm tiến hành chia dữ liệu theo tỉ lệ:

- **Train: 70%**
- **Validation: 15%**
- **Test: 15%**

Việc chia dữ liệu dùng `train_test_split` với tham số `stratify` theo nhãn để đảm bảo phân bố lớp giữa các tập là tương đối đồng đều.

- Chuẩn hóa đặc trưng và mã hóa nhãn

Do các đặc trưng dòng điện (I_a, I_b, I_c) và điện áp (V_a, V_b, V_c) có thang đo khác nhau, nhóm sử dụng `StandardScaler` để chuẩn hóa theo z-score:

$$x' = \frac{x - \mu}{\sigma}$$

Trong đó μ là trung bình và σ là độ lệch chuẩn của đặc trưng (tính trên tập train). Sau đó, scaler được áp dụng cho cả tập validation và test theo đúng quy trình chuẩn để tránh rò rỉ dữ liệu.

Nhãn dạng chuỗi (FaultType) được encode thành số nguyên:

$$y \in \{0, 1, \dots, C-1\}, \quad C=6$$

nhằm phù hợp với hàm mất mát Cross Entropy trong PyTorch.

- Tạo Dataset và DataLoader

Nhóm xây dựng lớp `FaultDataset` để đóng gói dữ liệu đầu vào/nhãn dưới dạng tensor:

- X : float32
- y : long

Sau đó tạo `DataLoader` cho train/val/test với:

- `batch_size = 64`
- `shuffle=True` cho train (để tăng tính ngẫu nhiên khi học)
- `shuffle=False` cho val và test.

- Thiết lập tái lập kết quả (Reproducibility) và thiết bị

Nhóm cố định seed bằng `numpy` và `torch` để đảm bảo kết quả huấn luyện có thể tái lập:

- `np.random.seed(42)`
- `torch.manual_seed(42)`
- `torch.cuda.manual_seed_all(42)` (nếu dùng GPU)

Thiết bị huấn luyện được chọn tự động:

- CUDA (GPU) nếu khả dụng, ngược lại dùng CPU.
- Kiến trúc mô hình MLP

Nhóm sử dụng mô hình **Multilayer Perceptron (MLP)** được xây dựng bằng thư viện **PyTorch** cho bài toán phân loại lỗi hệ thống điện. MLP là một mạng nơ-ron truyền thẳng gồm nhiều lớp tuyến tính kết hợp với hàm kích hoạt phi tuyến, cho phép mô hình học được các mối quan hệ phi tuyến giữa các đặc trưng dòng điện và điện áp.

- Cấu hình mô hình MLP Baseline

Mô hình MLP Baseline được thiết kế với kiến trúc đơn giản nhằm làm mốc so sánh cho các bước tối ưu sau này. Cấu hình baseline bao gồm:

- Số đặc trưng đầu vào: 6
- Số lớp đầu ra: 6
- Kiến trúc mạng: 1 lớp ẩn với 32 neuron
- Hàm kích hoạt: ReLU
- Dropout: không sử dụng
- Hàm mất mát: Cross Entropy Loss
- Bộ tối ưu: Adam
- Learning rate: 10^{-3}
- Weight decay: 0

Mô hình được huấn luyện trong tối đa 80 epoch, với cơ chế early stopping dựa trên độ chính xác của tập validation nhằm tránh hiện tượng overfitting.

Trong quá trình huấn luyện, mô hình được cập nhật trọng số bằng thuật toán Adam optimizer, kết hợp hàm mất mát Cross Entropy phù hợp với bài toán phân loại đa lớp. Sau mỗi epoch, mô hình được đánh giá trên tập validation để theo dõi quá trình hội tụ.

Mô hình đạt độ chính xác validation cao nhất sẽ được lưu lại và sử dụng để đánh giá trên tập test. Các chỉ số đánh giá bao gồm:

- Accuracy
- Precision Macro

- Recall Macro
- F1 Macro

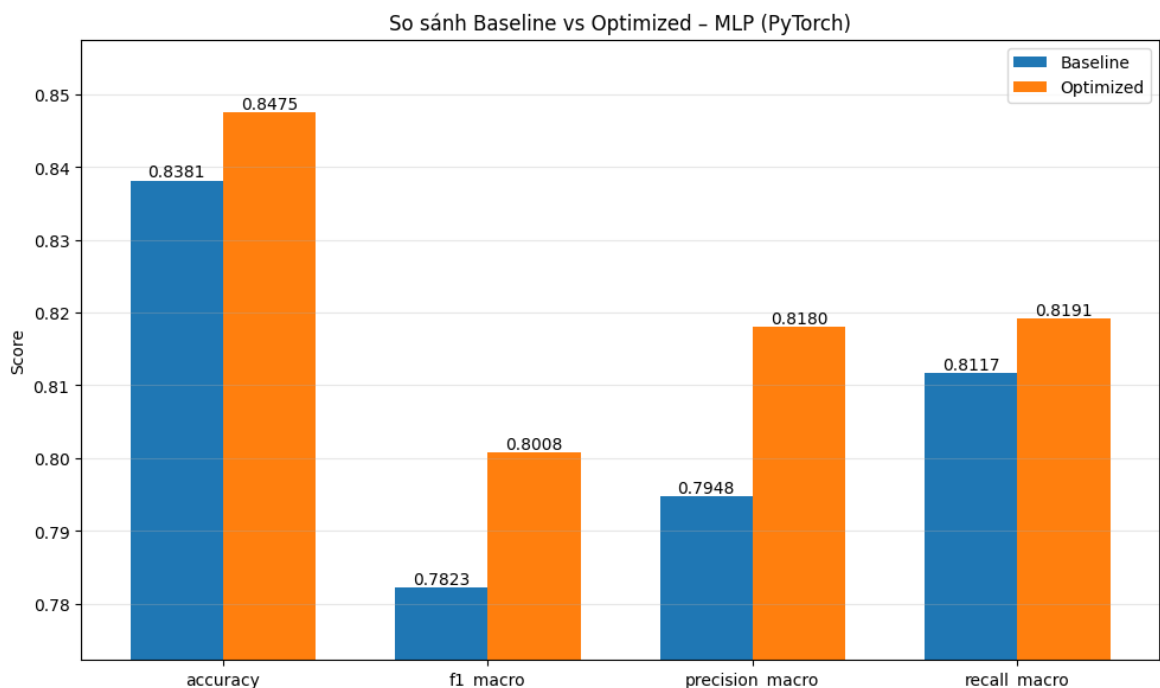
- Tối ưu mô hình MLP PyTorch

Để cải thiện hiệu năng, nhóm tiến hành thử nghiệm nhiều cấu hình MLP khác nhau thông qua việc thay đổi:

- Số lượng và kích thước các lớp ẩn,
- Tỷ lệ dropout,
- Learning rate,
- Weight decay.

Các cấu hình được đánh giá dựa trên validation accuracy, và cấu hình cho kết quả tốt nhất được lựa chọn làm MLP Optimized. Mô hình tối ưu được huấn luyện lại và đánh giá trên tập test để so sánh với mô hình baseline.

Đánh giá: So sánh đánh giá giữa 2 mô hình MLP Baseline và Optimized (PyTorch):



Hình 3.34: Baseline vs Optimized

- **Accuracy:**

Bảng 3.18: Accuracy

Mô hình	Accuracy
Baseline	0.8381
Optimized	0.8475

Accuracy tăng từ **0.8381** lên **0.8475**.

- Điều này cho thấy việc tối ưu các tham số huấn luyện đã giúp mô hình MLP PyTorch cải thiện khả năng phân biệt tổng thể giữa các loại lỗi so với mô hình baseline.

- **F1 Macro:**

Bảng 3.19: F1 Macro

Mô hình	F1 Macro
Baseline	0.7823
Optimized	0.8008

F1 Macro tăng từ **0.7823** lên **0.8008**.

- Đây là mức cải thiện đáng kể, cho thấy mô hình MLP PyTorch sau tối ưu đã cân bằng tốt hơn giữa precision và recall trên các lớp lỗi.
- Điều này đặc biệt quan trọng trong bài toán phân loại đa lớp với các loại lỗi có độ phức tạp khác nhau.

- **Precision Macro:**

Bảng 3.20: Precision Macro

Mô hình	Precision Macro
Baseline	0.7948
Optimized	0.8180

Precision Macro tăng từ **0.7948** lên **0.8180**.

- Mô hình MLP PyTorch sau tối ưu dự đoán chính xác hơn, giảm số lượng dự đoán sai (false positive).
- Điều này cho thấy ranh giới phân lớp được học rõ ràng hơn nhờ khả năng học phi tuyến của mạng nơ-ron.

● **Recall Macro:**

Bảng 3.21: Recall Macro

Mô hình	Recall Macro
Baseline	0.8117
Optimized	0.8191

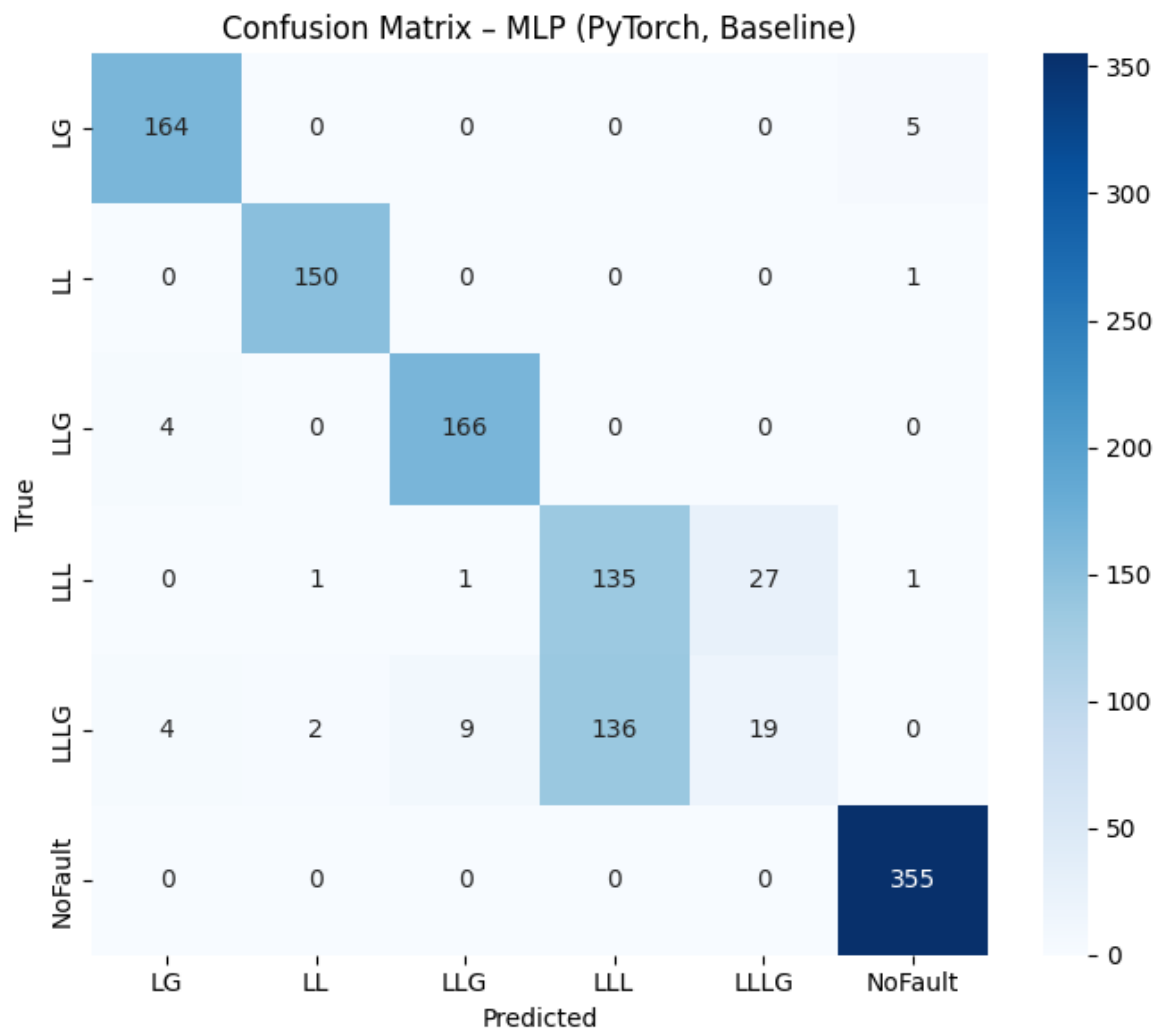
Recall Macro tăng từ **0.8117** lên **0.8191**.

- Sự cải thiện này phản ánh khả năng phát hiện đúng các lỗi của mô hình được nâng cao.
- Mô hình MLP PyTorch sau tối ưu đã giảm số lượng lỗi bị bỏ sót (false negative), đặc biệt với các lớp lỗi phức tạp.

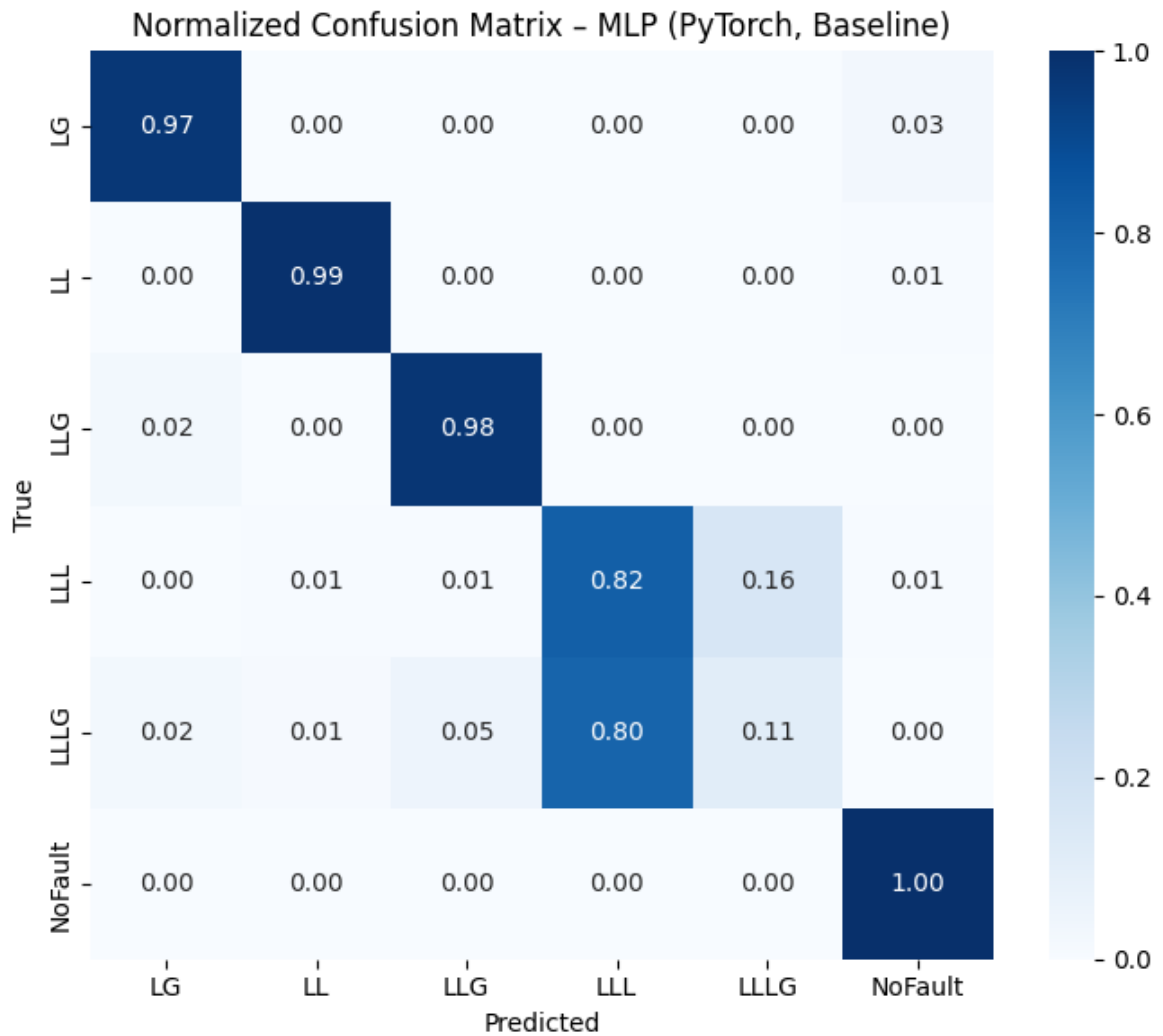
Kết luận:

- Việc tối ưu cấu hình mạng và các siêu tham số đã giúp cải thiện tất cả các chỉ số đánh giá của mô hình MLP PyTorch.
- Accuracy tăng cho thấy hiệu năng tổng thể của mô hình được nâng cao.

- Precision và Recall đều tăng, chứng minh mô hình phân biệt ranh giới giữa các loại lỗi chính xác hơn.
 - F1 Macro được cải thiện rõ rệt, cho thấy mô hình hoạt động cân bằng hơn trên toàn bộ các lớp lỗi.
 - Nhờ khả năng tùy biến kiến trúc và học các mối quan hệ phi tuyến phức tạp, MLP PyTorch sau tối ưu phù hợp để áp dụng cho bài toán phân loại lỗi hệ thống điện.
 - Mô hình MLP PyTorch sau tối ưu cho kết quả tốt hơn mô hình baseline, ổn định và có khả năng mở rộng trong các kịch bản huấn luyện nâng cao.
- **Mô hình MLP (PyTorch) sau tối ưu rõ ràng tốt hơn mô hình baseline, ổn định hơn và phù hợp để triển khai cho bài toán phân loại lỗi hệ thống điện.**



Hình 3.35: Confusion Matrix - MLP (PyTorch, Baseline)

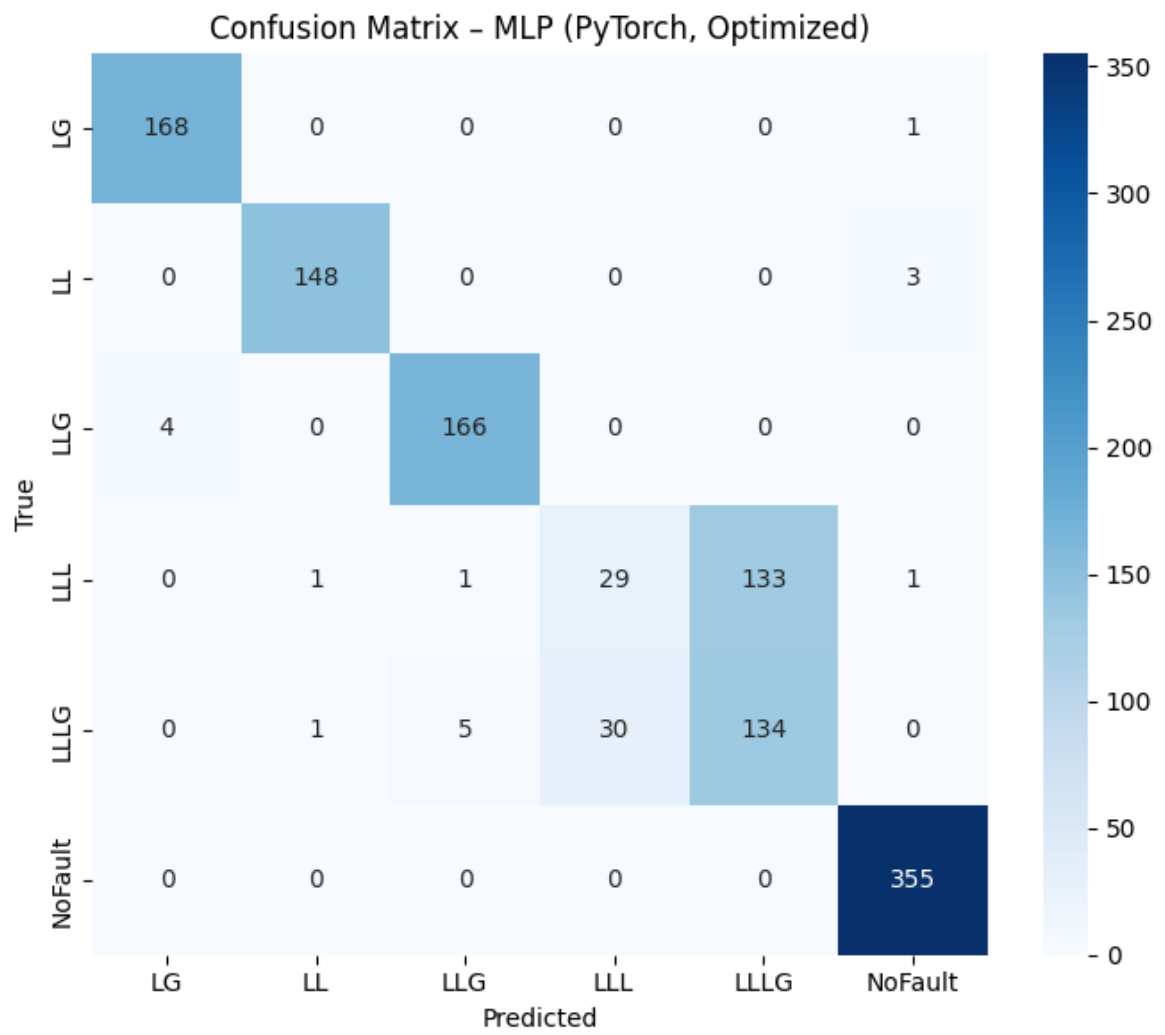


Hình 3.36: Normalized Confusion Matrix - MLP (PyTorch, Baseline)

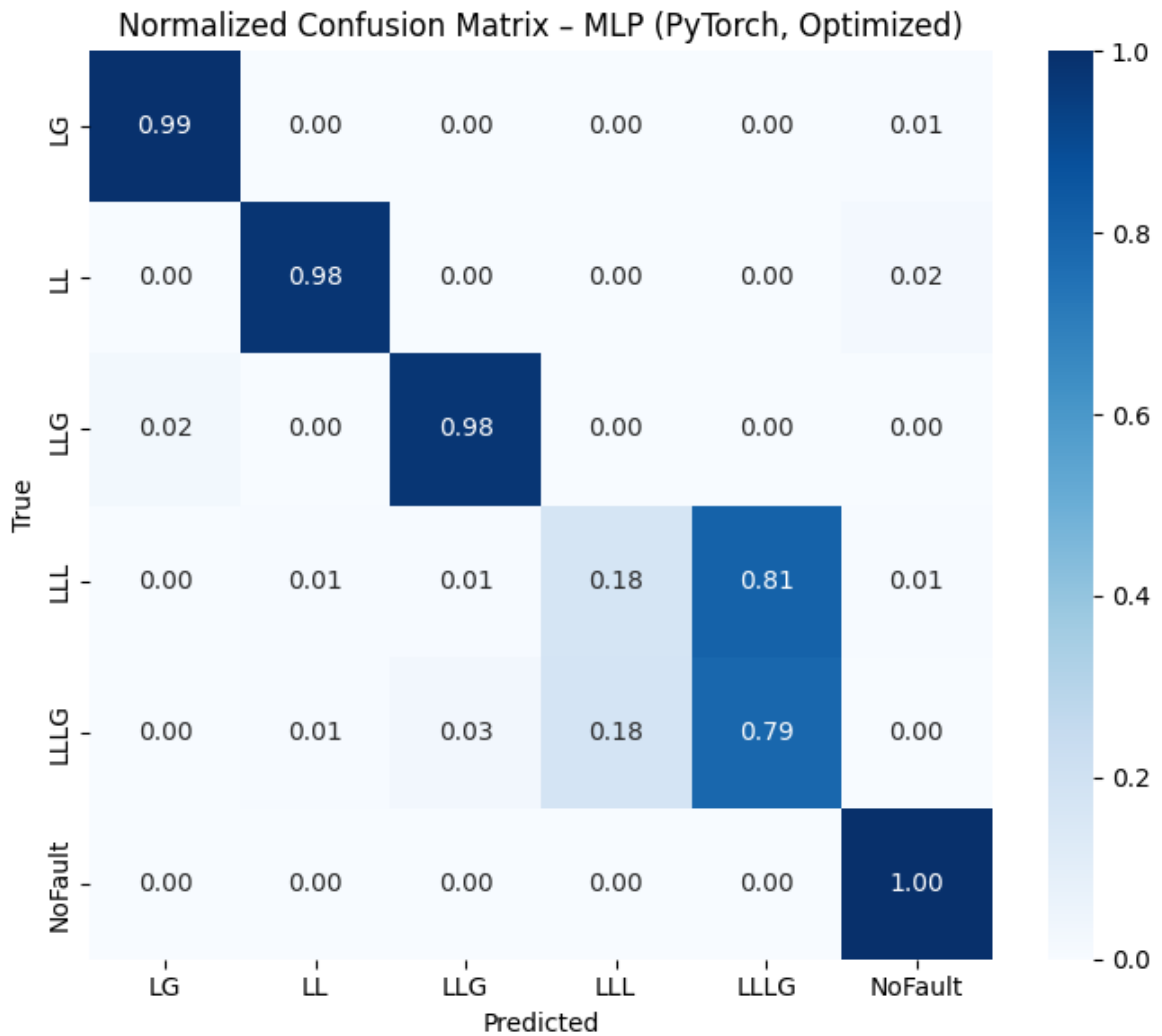
Từ ma trận nhầm lẫn của mô hình MLP PyTorch baseline, có thể thấy mô hình dự đoán rất tốt các lớp NoFault, LG, LL và LLG, với tỷ lệ dự đoán đúng trên đường chéo chính đều cao (trên 97%).

Tuy nhiên, các lỗi LLL và LLLG vẫn còn hiện tượng nhầm lẫn lẫn nhau, trong đó một số mẫu LLL bị dự đoán thành LLLG và ngược lại.

Điều này cho thấy ở cấu hình baseline, mô hình đã học tốt các lớp lỗi đơn giản và rõ ràng, nhưng vẫn gặp khó khăn trong việc phân biệt các lớp phức tạp có đặc trưng dòng - áp tương đồng. Đây cũng là động lực để tiếp tục tối ưu kiến trúc và siêu tham số của mạng MLP nhằm cải thiện khả năng phân biệt giữa các lớp lỗi này.



Hình 3.37: Confusion Matrix - MLP (PyTorch, Optimized)

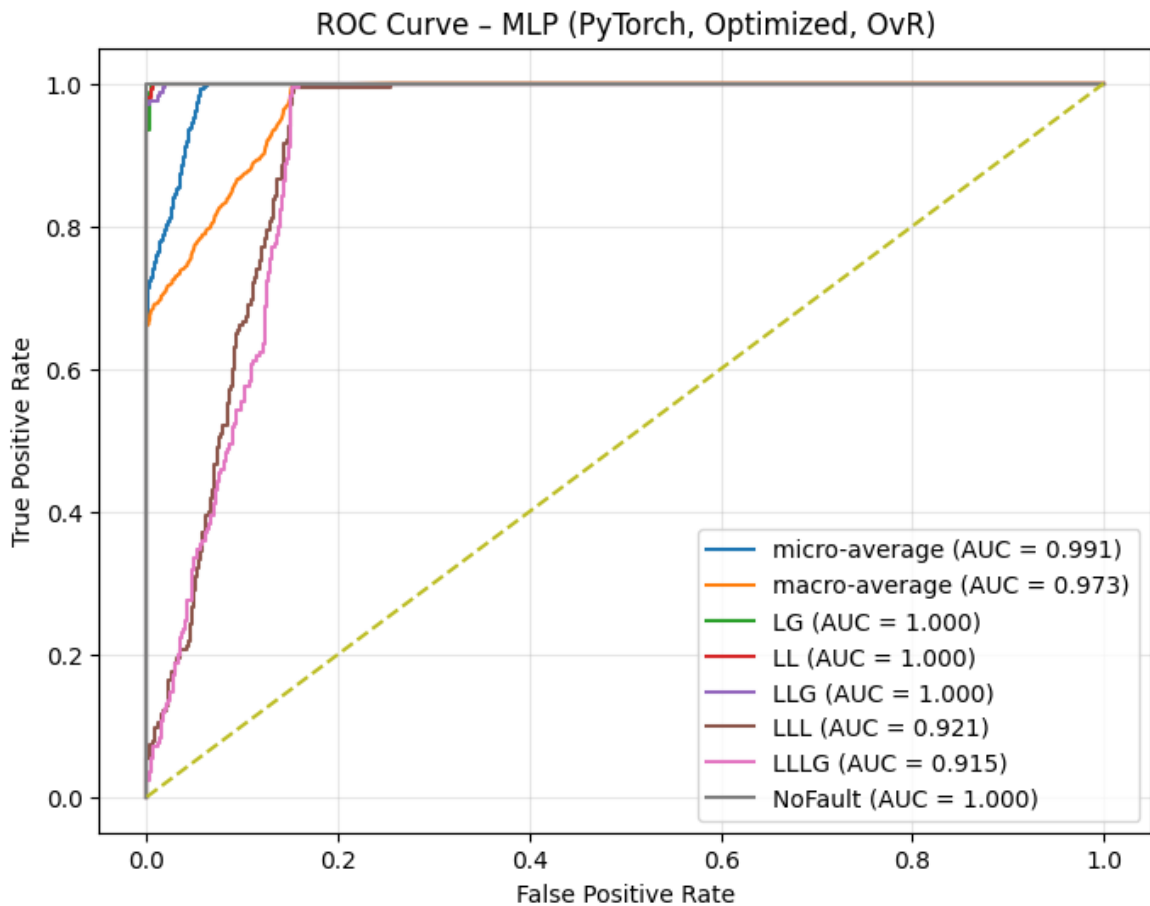


Hình 3.38: Normalized Confusion Matrix - MLP (PyTorch, Optimized)

Từ ma trận nhầm lẫn của mô hình MLP PyTorch sau tối ưu, có thể thấy độ chính xác trên đường chéo chính tiếp tục được cải thiện so với mô hình baseline. Các lớp NoFault, LG, LL và LLG được phân loại rất tốt với tỷ lệ dự đoán đúng gần như tuyệt đối.

Đáng chú ý, mức độ nhầm lẫn giữa hai lớp lỗi phức tạp LLL và LLLG đã giảm so với mô hình baseline, thể hiện qua việc số mẫu được dự đoán đúng trên hai lớp này tăng lên rõ rệt. Điều này cho thấy việc điều chỉnh kiến trúc mạng và các siêu tham số đã giúp mô hình học được ranh giới phân lớp tốt hơn giữa các lỗi có đặc trưng dòng - áp tương đồng.

Nhìn chung, mô hình MLP PyTorch sau tối ưu cho kết quả phân loại ổn định và chính xác hơn, đặc biệt đối với các lớp lỗi phức tạp, khẳng định hiệu quả của quá trình tối ưu trong bài toán phân loại lỗi hệ thống điện.



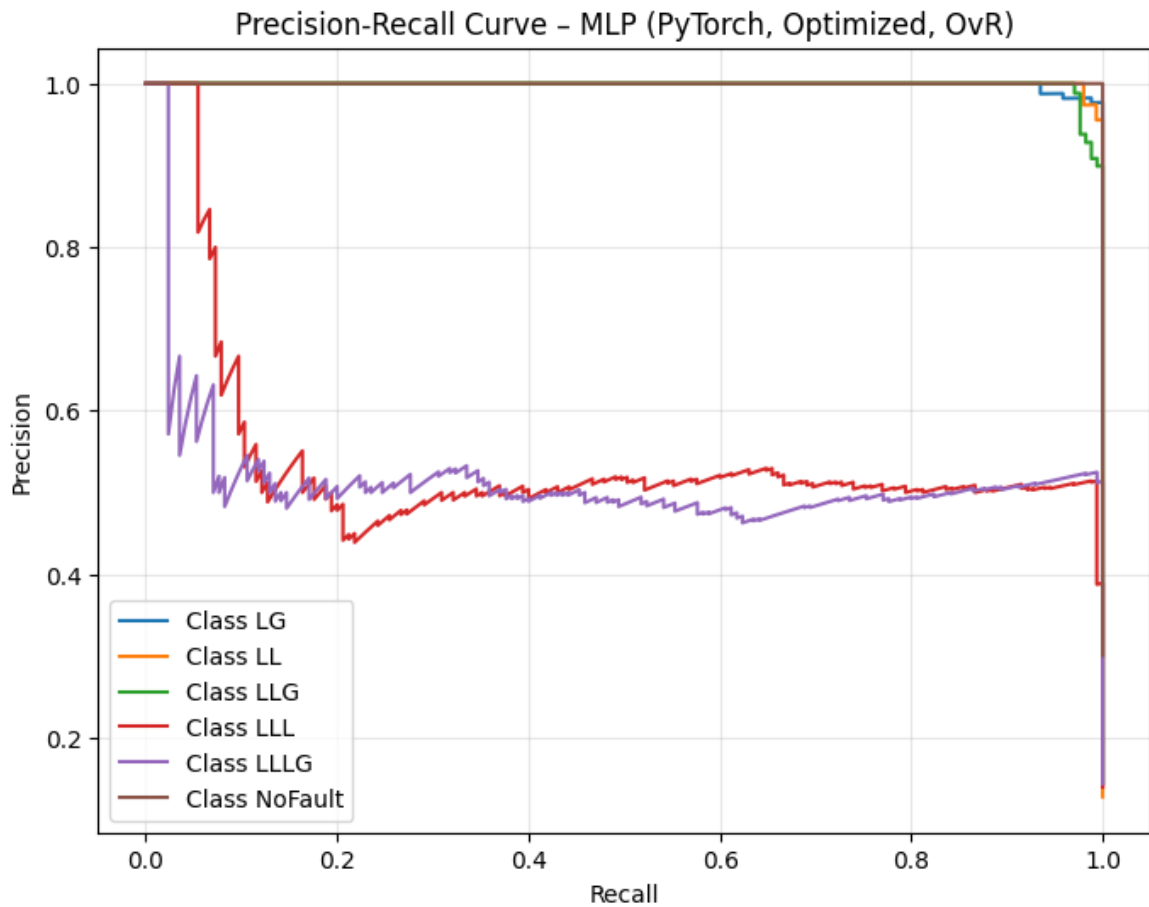
Hình 3.39: ROC Curve - MLP (PyTorch, Optimized, OvR)

Đường cong ROC của mô hình MLP PyTorch sau tối ưu cho thấy khả năng phân biệt giữa các lớp lỗi là rất tốt. Giá trị AUC micro-average đạt 0.991 và AUC macro-average đạt 0.973, chứng tỏ mô hình có hiệu năng phân loại cao trên toàn bộ tập dữ liệu đa lớp.

Đối với các lớp LG, LL, LLG và NoFault, AUC đều đạt 1.000, cho thấy mô hình gần như phân biệt hoàn hảo các lớp này. Trong khi đó, các lớp lỗi phức tạp hơn như LLL (AUC = 0.921) và LLLG (AUC = 0.915) có AUC thấp hơn nhưng vẫn ở

mức tốt, phản ánh độ khó cao hơn trong việc phân biệt các lỗi có đặc trưng dòng - áp tương đồng.

Nhìn chung, kết quả ROC khẳng định mô hình MLP PyTorch sau tối ưu có khả năng phân tách các lớp lỗi hiệu quả, đặc biệt phù hợp cho bài toán phân loại lỗi hệ thống điện trong môi trường đa lớp.



Hình 3.40: Precision-Recall Curve - MLP (PyTorch, Optimized, OvR)

Từ đường cong Precision-Recall của mô hình MLP PyTorch sau tối ưu, có thể thấy mô hình duy trì mức precision cao trên hầu hết các giá trị recall đối với các lớp LG, LL, LLG và NoFault, cho thấy khả năng dự đoán chính xác và ổn định đối với các lỗi phổ biến.

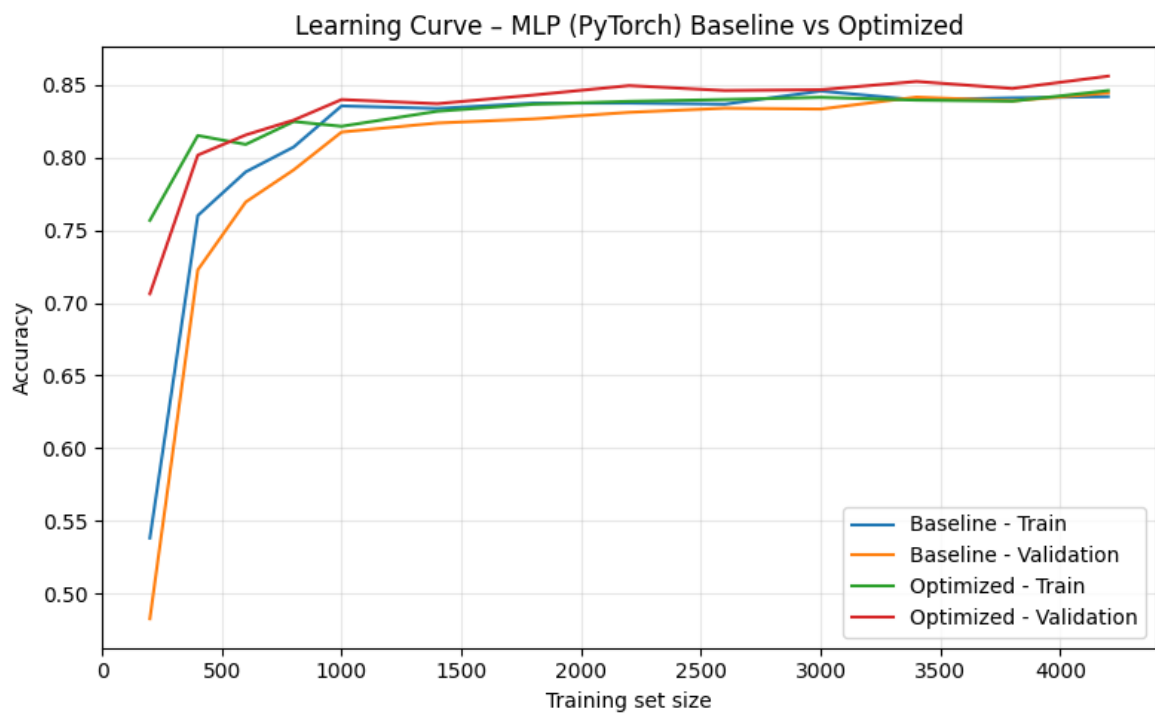
Đối với các lớp lỗi phức tạp hơn như LLL và LLLG, precision có xu hướng giảm khi recall tăng cao, phản ánh sự đánh đổi giữa khả năng phát hiện đầy đủ lỗi và

độ chính xác của dự đoán. Tuy nhiên, các đường cong vẫn duy trì ở mức chấp nhận được, cho thấy mô hình đã học được đặc trưng phân biệt cơ bản của các lớp lỗi này.

Nhìn chung, kết quả Precision-Recall cho thấy mô hình MLP PyTorch sau tối ưu đạt sự cân bằng tốt giữa precision và recall, đặc biệt phù hợp cho bài toán phân loại đa lớp trong hệ thống điện, nơi yêu cầu vừa phát hiện đúng lỗi vừa hạn chế dự đoán nhầm.

	la	lb	lc	Va	Vb	Vc	True	Pred	Correct
0	-687.584229	-136.972122	824.554077	-0.038883	0.034071	0.004813	LLLG	LLLG	True
1	703.862915	-822.730469	118.880051	-0.007023	-0.032710	0.039733	LLLG	LLLG	True
2	-247.112045	-627.317810	876.596558	-0.041003	0.013147	0.027856	LLL	LLLG	False
3	10.365164	76.118721	-89.624939	-0.540763	0.541436	-0.000673	NoFault	NoFault	True
4	5.219489	-741.652954	738.955139	-0.312253	-0.011754	0.324007	LL	LL	True
5	-102.259087	-53.383373	49.352894	0.382478	-0.540228	0.157750	LG	LG	True
6	-61.027771	29.170473	28.642023	0.211703	-0.579798	0.368095	NoFault	NoFault	True
7	-380.840973	20.162220	-45.724812	-0.315171	0.603154	-0.287983	LG	LG	True
8	-65.323303	83.578957	-21.251942	-0.382126	-0.212551	0.594677	NoFault	NoFault	True
9	-584.149719	-263.459839	-50.692757	-0.041019	0.371620	-0.330600	LLG	LLG	True

Hình 3.41: Minh họa dự đoán của MLP PyTorch trên tập kiểm tra



Hình 3.42: Learning Curve - MLP (PyTorch) Baseline vs Optimized

Từ đường cong học (learning curve), có thể thấy cả hai mô hình MLP PyTorch baseline và optimized đều cải thiện độ chính xác khi kích thước tập huấn luyện tăng, cho thấy mô hình tận dụng tốt dữ liệu huấn luyện.

Mô hình optimized đạt độ chính xác cao hơn mô hình baseline trên cả tập huấn luyện và tập validation trong hầu hết các kích thước dữ liệu. Khoảng cách giữa đường train và validation của mô hình optimized là nhỏ, cho thấy mô hình học ổn định và ít bị overfitting.

Ngược lại, mô hình baseline có tốc độ hội tụ chậm hơn và độ chính xác thấp hơn khi số lượng mẫu huấn luyện còn nhỏ. Khi dữ liệu tăng lên, hiệu năng của baseline được cải thiện nhưng vẫn thấp hơn mô hình optimized.

Nhìn chung, learning curve cho thấy việc tối ưu kiến trúc và siêu tham số giúp mô hình MLP PyTorch hội tụ nhanh hơn, ổn định hơn và đạt hiệu năng tốt hơn, đặc biệt khi sử dụng tập dữ liệu lớn cho bài toán phân loại lỗi hệ thống điện.

CHƯƠNG 4. ĐÁNH GIÁ VÀ KẾT LUẬN

4.1 Tổng hợp kết quả các mô hình

Trong báo cáo này, nhóm đã xây dựng và đánh giá 4 hướng mô hình chính cho bài toán phân loại lỗi hệ thống điện ba pha dựa trên 6 đặc trưng đầu vào Ia, Ib, Ic, Va, Vb, Vc . Các mô hình gồm:

- **Random Forest (RF)** (Baseline & Optimized)
- **SVM (RBF kernel)** (Baseline & Optimized)
- **kNN** (Baseline & Optimized)
- **MLP** (2 phiên bản triển khai):
 - **MLP (sklearn - MLPClassifier)** (Baseline & Optimized)
 - **MLP (PyTorch)** (Baseline & Optimized)

Nhìn chung, các mô hình đều đạt hiệu năng tốt trên tập kiểm thử, tuy nhiên mức cải thiện sau tối ưu và khả năng phân biệt các lớp lỗi phức tạp là khác nhau.

4.2 So sánh hiệu năng theo các chỉ số chính

Để so sánh công bằng trong bối cảnh bài toán đa lớp, nhóm ưu tiên đánh giá theo F1 Macro / Precision Macro / Recall Macro, vì các chỉ số này phản ánh mức độ “cân bằng” giữa các lớp lỗi, hạn chế việc mô hình chỉ làm tốt ở lớp dễ (ví dụ NoFault) nhưng yếu ở lớp khó.

a. Random Forest

- Kết quả cho thấy RF Baseline (Accuracy = 0.8811) nhỉnh hơn RF Optimized (Accuracy = 0.8792), và các chỉ số F1/Precision/Recall Macro của baseline cũng cao hơn nhẹ.
- Điều này cho thấy việc GridSearchCV không cải thiện được RF trong trường hợp này, nguyên nhân chính đến từ việc một số lớp lỗi (đặc biệt nhóm lỗi 3 pha) có đặc trưng quá tương đồng nên dù tinh chỉnh tham số vẫn khó tách biệt rõ ràng.

b. SVM (RBF)

SVM Optimized cải thiện đồng bộ so với baseline:

- Accuracy: 0.8398 $\rightarrow$ 0.8474
- F1 Macro: 0.8071 $\rightarrow$ 0.8182
- Precision Macro: 0.8099 $\rightarrow$ 0.8215
- Recall Macro: 0.8132 $\rightarrow$ 0.8214

Mức tăng không quá lớn nhưng ổn định, cho thấy SVM phù hợp với dữ liệu phi tuyến dòng-áp, và tối ưu siêu tham số đem lại hiệu quả rõ ràng.

c. kNN

Đây là mô hình có mức cải thiện mạnh nhất sau tối ưu:

- Accuracy: 0.797 $\rightarrow$ 0.885
- F1 Macro: 0.760 $\rightarrow$ 0.865
- Precision Macro: 0.760 $\rightarrow$ 0.865

- Recall Macro: 0.761 $\rightarrow$ 0.865

Confusion Matrix cho thấy bản optimized đã cải thiện rõ điểm yếu “chỉ mạng” ở nhóm lỗi 3 pha, giúp mô hình cân bằng tốt hơn trên toàn bộ lớp lỗi.

d. MLP (sklearn - MLPClassifier)

Mô hình MLP sau tối ưu cho kết quả tốt hơn baseline:

- Accuracy: 0.8322 $\rightarrow$ 0.8570
- F1 Macro: 0.7959 $\rightarrow$ 0.8035
- Precision Macro: 0.7994 $\rightarrow$ 0.8550
- Recall Macro: 0.8016 $\rightarrow$ 0.8339

MLP thể hiện rõ lợi thế học phi tuyến, tuy nhiên vẫn còn nhầm lẫn đáng kể giữa các lớp lỗi phức tạp (đặc biệt LLL và LLLG).

e. MLP (PyTorch)

Phiên bản PyTorch cũng cải thiện sau tối ưu:

- Accuracy: 0.8381 $\rightarrow$ 0.8475
- F1 Macro: 0.7823 $\rightarrow$ 0.8008
- Precision Macro: 0.7948 $\rightarrow$ 0.8180
- Recall Macro: 0.8117 $\rightarrow$ 0.8191

ROC/AUC cho thấy mô hình phân biệt rất tốt các lớp dễ (LG, LL, LLG, NoFault có AUC $\sim$ 1.0), nhưng hai lớp LLL và LLLG vẫn là nhóm khó nhất (AUC thấp hơn rõ rệt).

4.3 Nhận xét quan trọng: Lớp lỗi khó (LLL vs LLLG)

Một kết luận xuyên suốt ở hầu hết mô hình là:

- LLL (lỗi 3 pha) và LLLG (lỗi 3 pha chạm đất) thường bị nhầm lẫn qua lại.

Nguyên nhân chính:

- Hai dạng lỗi này đều tạo ra biến thiên mạnh ở cả ba pha, dẫn đến tín hiệu dòng-áp có hình dạng và phân bố gần nhau.

- Vì vậy, trong không gian đặc trưng Ia, Ib, Ic, Va, Vb, Vc , ranh giới hai lớp bị chồng lấn, khiến mô hình khó tách biệt tuyệt đối dù đã tối ưu.

4.4 Kết luận mô hình tốt nhất

Dựa trên kết quả thực nghiệm, có thể tổng hợp theo hướng “mô hình tốt nhất để triển khai” như sau:

Mô hình tốt nhất về tổng thể:

- kNN Optimized là mô hình nổi bật nhất vì:
 - Accuracy đạt 0.885 (cao nhất trong các mô hình được tối ưu)
 - F1/Precision/Recall Macro đều đạt 0.865, chứng minh mô hình cân bằng tốt trên toàn bộ 6 lớp lỗi, không chỉ mạnh ở lớp dễ.

Mô hình mạnh và ổn định (nhưng không “vượt trội”):

- Random Forest Baseline cho kết quả cao (Accuracy 0.8811), tuy nhiên tối ưu không giúp cải thiện mà còn giảm nhẹ. Điều này cho thấy RF đã “đủ tốt” với tham số mặc định, nhưng vẫn vướng điểm yếu chung ở nhóm lỗi 3 pha.

Mô hình phù hợp dữ liệu phi tuyến, cải thiện ổn định:

- SVM Optimized có cải thiện đồng đều và ổn định, phù hợp khi cần một mô hình “khá mạnh - tổng quát tốt” trong không gian đặc trưng phi tuyến.

MLP (sklearn & PyTorch):

- Hai phiên bản MLP đều thể hiện khả năng học phi tuyến tốt và có cải thiện sau tối ưu.
- Tuy nhiên, so với kNN Optimized và RF Baseline, các chỉ số tổng thể của MLP vẫn thấp hơn, và nhầm lẫn LLL/LLLG vẫn còn đáng kể.
- Trong nội bộ MLP: **MLP (sklearn - Optimized)** nhỉnh hơn **MLP (PyTorch - Optimized)** về Accuracy (0.8570 vs 0.8475).

4.5 Tổng kết cuối cùng

Nhóm đã xây dựng đầy đủ pipeline: tiền xử lý → huấn luyện → tối ưu → đánh giá → trực quan hóa (Confusion Matrix, ROC/PR, Learning Curve).

Kết quả cho thấy:

- **kNN Optimized** là lựa chọn tốt nhất nếu ưu tiên hiệu năng tổng thể và độ cân bằng giữa các lớp.
- **Random Forest Baseline** vẫn rất mạnh và ổn định, gần sát mô hình tốt nhất.
- **SVM Optimized** là lựa chọn phù hợp khi cần mô hình phi tuyến mạnh, cải thiện ổn định.
- **MLP** (sklearn/PyTorch) thể hiện khả năng học phi tuyến nhưng chưa vượt trội trên bộ dữ liệu này.

Hạn chế lớn nhất còn lại là phân biệt LLL và LLLG, do đặc trưng tín hiệu tương đồng.

TÀI LIỆU THAM KHẢO

Tiếng Việt

Alcandy. Tổng quan 4 phương pháp học máy chính trong trí tuệ nhân tạo.

<https://aicandy.vn/tong-quan-4-phuong-phap-hoc-may-chinh-trong-tri-tue-nhan-tao/>

Tuấn Nguyễn. (2021). Random Forest algorithm.

https://machinelearningcoban.com/tabml_book/ch_model/random_forest.html

VBD. (2025). Tất tần tât về thuật toán K-Nearest Neighbors (KNN) trong Học máy.

<https://vinbigdata.com/kham-pha/tat-tan-tat-ve-thuat-toan-k-nearest-neighbors-knn-trong-hoc-may.html>

Huỳnh Chi Trung. (2020). Giới thiệu về Support Vector Machine (SVM).

<https://viblo.asia/p/gioi-thieu-ve-support-vector-machine-svm-6J3ZgPVEImB>

Machine Learning cơ bản. (2017). Bài 14: Multi-layer Perceptron và Backpropagation.

<https://machinelearningcoban.com/2017/02/24/mlp/>

Tiếng Anh

MarketMuse. Multilayer Perceptron (MLP). MarketMuse Blog.

<https://blog.marketmuse.com/glossary/multilayer-percpetron-definition/>